

Bittersweet Signatures: Bringing LWR to a Picnic for Hardware-Friendly MPC-in-the-Head

– Full Version –

Brieuc Balon¹ , Gianluca Brian^{2,3} , Sebastian Faust² , Carmit Hazay⁴ ,
Elena Micheli² , and François-Xavier Standaert¹ 

¹ UCLouvain, ICTEAM, Crypto Group, Louvain-la-Neuve, Belgium

² Technical University of Darmstadt, Germany

³ ETH Zurich, Zurich, Switzerland

⁴ Bar-Ilan University, Israel

Abstract. Post-quantum signature schemes are becoming increasingly important due to the threat of quantum computers to classical cryptographic schemes. Among the approaches considered in the literature, the MPC-in-the-head paradigm introduced by Ishai et al. (STOC’07) provides an innovative solution for constructing zero-knowledge proofs by exploiting Multi-Party Computation (MPC). This technique has proven to be a versatile tool in order to design efficient cryptographic schemes, including post-quantum signatures. Building on the MPC-in-the-head paradigm, we introduce Bittersweet signatures, a new class of signature schemes based on the Learning With Rounding (LWR) assumption. Their main advantage is conceptual simplicity: by exploiting (almost) key-homomorphic pseudorandom functions (PRFs), a cryptographic object that preserves pseudorandomness while allowing linear operations on keys, we obtain a very regular design offering nice opportunities for parallel implementations. Theoretically, analyzing Bittersweet signatures requires addressing significant challenges related to the (carry) leakage that almost key-homomorphic operations lead to. Concretely, Bittersweet signatures natively lead to competitive signature sizes, trading moderate software performance overheads for hardware performance gains when compared to state-of-the-art MPC-in-the-head schemes (e.g., relying on code-based assumptions), while admittedly lagging a bit behind recent algorithms based on the VOLE-in-the-head or Threshold-Computation-in-the-head frameworks. Besides, their scalability and algebraic structure makes them promising candidates for leakage-resilient implementations. The new abstractions we introduce additionally suggest interesting research directions towards further optimization and generalization.

1 Introduction

1.1 State of the Art

For more than a decade now, designing signature schemes that can withstand quantum computers has been a major research area. These efforts culminated with the selection of three first algorithms for standardization by the NIST

in July 2022: Dilithium (<https://pq-crystals.org/dilithium/>) and Falcon (<https://falcon-sign.info/>), whose security relies on the hardness of solving lattice problems, and SPHINCS⁺, whose security relies on cryptographic hash functions (<https://sphincs.org/>). Nevertheless, the design of post-quantum signature schemes remains of fundamental importance for several reasons. First, increasing the diversity of the hardness assumptions these algorithms rely on is needed to avoid “putting all eggs in the same basket”. This has prompted the NIST to open a call for new proposals, leading to the submission of 50 candidates in June 2023.¹ Second, the performances and cost of these algorithms remains a topic of concern, especially if security against implementation attacks is required [65, 11, 38, 66]. Finally, the state of the art regarding signatures with advanced functionalities such as threshold and multisignatures [29], group signatures [37] and ring signatures [77], currently remains relatively sparse.

As reflected by the submissions to the 2023 NIST call, post-quantum signatures can be based on various assumptions (e.g., besides lattices and hash functions used by Dilithium, Falcon, and SPHINCS⁺, Isogenies, code-based assumptions, and multivariate polynomials have been considered) and rely on different methodologies. Among those, one popular approach, on which we focus in this work, is to rely on zero-knowledge proofs using the MPC-in-the-head paradigm, introduced by Ishai *et al.* [62]. This paradigm refers to a technique that uses Multi-Party Computation (MPC) in the context of Zero-Knowledge (ZK) proofs, where a set of parties compute a function over their inputs while keeping those inputs private. In the proof system, the prover then simulates an MPC protocol in his mind (or “in the head”) and commits to the produced views. The verifier then asks to open a subset of these views and verifies that they are consistent and that the output of the computation is correct.

While initially seen as a primarily theoretical contribution, this paradigm has been shown to be a versatile tool to construct (among others) efficient signature schemes, starting with the work of Giacomelli *et al.* [58]. On the one hand, it has motivated several designs relying on symmetric cryptographic primitives like Picnic [36, 35], BBQ [40], Banquet [18], Limbo [41] and Rainier [46]. On the other hand, it has also been exploited to leverage various other assumptions. For example, LegRoast relies on the one-wayness of the Legendre PRF [26], several schemes rely on the syndrome decoding problem [60, 52, 28, 34], Feneuil *et al.* proposed schemes based on the subset sum problem [53], Adj *et al.* proposed a scheme based on the MinRank problem [3], Benadjila *et al.* proposed a scheme based on the multivariate quadratic problem [21], and Bettaieb *et al.* proposed one based on the permuted kernel problem [24]. To a large extent, these new designs followed the first motivations mentioned above (*i.e.*, avoiding putting all eggs in the same basket and improving performances, mostly ignoring implementation attacks so far). As a result, and to the best of our knowledge, there are few works considering the combination of lattice-based assumptions and the MPC-in-the-head paradigm (exceptions are discussed in related works).

¹ <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>.

1.2 Our Contribution

The early MPC-in-the-head schemes relied on various instantiations of the underlying MPC protocol, with dishonest majority [58, 36, 67] and honest majority [8, 27], introducing different trade-offs between prover runtime and proof length. More recent schemes considered new abstractions that capture simpler cryptographic building blocks, such as game-based cryptographic primitives that support homomorphic computations [61] and puncturable PRFs [34]. In this paper, we consider yet another instantiation of the MPC-in-the-head paradigm based on (almost) key-homomorphic PRFs [31], which preserve pseudorandomness while allowing operations on the keys in an (almost) homomorphic way.

Key-homomorphic PRFs are useful in the MPC-in-the-head (signature) setting because they enable non-interactive computations, which considerably simplifies the views on which a prover has to commit. The concrete PRF we use for this purpose is based on the Learning With Rounding (LWR) assumption [13], and therefore goes against the conventional wisdom that signatures based on the MPC-in-the-head paradigm are best combined with non-lattice-based assumptions. We nevertheless show that this choice comes with various advantages, which we reflect with the Bittersweet signatures’ name.² Among others, working with this object enables a clean formalization of the virtual MPC protocol for evaluating the PRF on a public value (as needed for our signature scheme). It also leads to a regular design facilitating parallel implementations where the (multi-party) computations can be performed on each share independently.

As a first step towards formalizing the use of key-homomorphic primitives for MPC-in-the-head signatures, we investigate a simple setting where all-but-one share are revealed at every iteration of the protocol. The main technical challenge we need to solve for this purpose is to deal with the additional (carry) leakage that the almost key-homomorphism leads to. That is, since the share-based MPC computations of the PRF we use are only approximately correct, the errors may leak additional information about the secret (via carry propagation). Like many other post-quantum signature schemes, we rely on the Fiat-Shamir with abort transform in order to mitigate this carry leakage [69]. We also discuss how heuristic instantiation choices can make this carry leakage arbitrarily small and how to capture it directly with (new) hardness assumptions. More precisely, we show in [Section A](#) that a “LWR with carry leakage” can be reduced to standard LWR (given some increase of the parameters), and that it is also connected to a form of “LWR with misuse”, which may be of independent interest.

A more detailed overview of our technical contribution is given in [Section 2](#).

We finally illustrate the potential of our results by discussing the concrete instantiation of Bittersweet signatures. This allows us to exhibit the gaps between parameters suggested by (current) security proofs and parameters enabling efficient implementations, as observed for other schemes like Picnic [36, 35]. It also allows us to exhibit how more aggressive optimizations can gradually reduce the size of Bittersweet signatures and lead to competitive instances. We

² Suggesting an initially strange but ultimately good-tasting mix of ingredients.

additionally confirm the interest of regular designs for hardware implementation purposes. We show that almost key-homomorphism leads to a straightforward way to increase the number of shares since the same circuit can be applied to all the shares without interaction, which is easily leveraged by hardware implementations for computationally-intensive operations (e.g., hash functions, multiplications). As a result, we can demonstrate hardware performance gains for moderate software performance overheads when compared to state-of-the-art MPC-in-the-head schemes (e.g., relying on code-based assumptions), while the performances of Bittersweet are admittedly lagging a bit behind the ones recent schemes based on the VOLE-in-the-head or Threshold-Computation-in-the-head frameworks. Incidentally, our results also lead to one of the few efficient signature schemes relying on unstructured lattice assumptions in the literature.

1.3 Future Work

The conceptual simplicity of Bittersweet signatures and the generic investigation of almost key-homomorphic primitives for MPC-in-the-head signatures that we initiate come with a nice potential for further developments towards leakage-resilient implementations, and opens interesting research avenues:

Leakage-resilience. It has already been shown that the PRF on which Bittersweet signatures rely can be efficiently masked [48]. Hence, implementations with strong leakage-resilience are a natural target for future work. The (almost linear) scaling trends of masked key-homomorphic primitives should, for example, lead to a better efficiency vs. (side-channel) security tradeoff than MPC-in-the-head signatures based on symmetric cryptographic primitives [78, 10]. The large moduli used by Bittersweet signatures could additionally be turned into an asset if combined with advanced (e.g., inner product) masking schemes [50].

Generalizations. Our investigations focus on a PRF operating in $\mathbb{Z}_{2^q}$, additive secret sharing and a protocol opening all-but-one share per iteration. But nothing theoretically prevents generalizations to other fields, secret sharing schemes, and opening strategies. Such generalizations could be interesting for performance [54, 55], leakage-resilience [72] and possibly help to design new signatures with advanced functionalities (e.g., threshold or multisignatures). Whether our results can be extended to the LWE context and whether other key-homomorphic PRFs can lead to better results are also interesting open questions.

Optimizations. As many post-quantum signature schemes based on the MPC-in-the-head paradigm, Bittersweet signatures require to commit on a lot of randomness. The parallel nature of the Bittersweet design nevertheless raises the question of whether one could get rid of (some of) these commitments by leveraging properties of LWR (e.g., if the LWR function was itself sufficiently committing). Such optimizations would lead to further performance improvements.

LWR with carry leakage and misuse. The formal investigation of these new variants of standard hardness assumptions that we initiate in [Section A](#) finally raises the question whether they could find applications in other contexts.

1.4 Related Works

Combining the MPC-in-the-head paradigm with lattice-based assumption was already proposed by other works like [22, 19, 25], yet in a quite different context than ours (*i.e.*, not as a building block of key-homomorphic PRFs).

Using a rounding operation as a source of non-linearity to build low-complexity cryptographic objects is reminiscent of using alternating moduli [30, 44].

A recent line of work focuses on VOLE-in-the-head signatures [17, 76, 39, 16], whose key building blocks are multi-round interactive protocols based on Vector Oblivious Linear Evaluation (VOLE) correlations. This paradigm is tailored for ZK protocols in the VOLE-hybrid model, aiming to compile them into a publicly verifiable protocol. However, it may require more than three rounds of interaction and non-black-box compilation techniques within the underlying VOLE protocol. As a result, adapting their techniques to our context would not be straightforward, and yields another interesting direction for future work.

MPC-based signatures rely on symmetric primitives (GGM/Merkle trees) in order to derive randomness for the parties. As the number of emulated parties grows, this part of the designs may lead to significant overheads. To address this, [33] uses a block cipher-based Punctured PRF (PPRF) instead of a hash-based one, which reduces the execution time, while [16] redesigns the GGM-PPRF to further shrink the signature size. We avoid these optimizations which would be less beneficial to Bittersweet signatures, for which the main performance bottleneck stems from algebraic operations. Adopting [16] would further introduce a rejection probability tied to the challenge space, complicating the design.

Finally, and besides the aforementioned thresholding approach [54, 55], the hypercube optimization has been proposed to amplify the soundness of any MPC protocol that uses additive secret sharing [74]. Such an optimization is conceptually applicable to Bittersweet, but its interest is reduced because it combines poorly with another (more significant) optimization we leverage, namely replacing the last (unopened) shares by smaller error vectors in the signature.

2 Technical Overview

A successful paradigm for designing signature schemes is based on identification schemes (which can also be viewed as a generalization of Sigma protocols). Specifically, in the first step, a challenge-response 3-round protocol is constructed for a given hard relation. Then, in the next step, a round collapsing mechanism is applied to the above protocol, using the signed message as the challenge. The MPC-in-the-head framework can be seen as a generalization of the above paradigm. In this context, the hard relation is translated into a distributed functionality, which is then realized with multiple parties running an MPC protocol. As a result, the choice of this MPC protocol will have a crucial impact on the level of concrete efficiency of the signature scheme. At a high level, the design of such a signature scheme involves the following four main steps:

1. *Selecting the hard relation* between public and secret keys;

2. *Choosing the MPC scheme* used to compute the relation;
3. *Constructing the Sigma protocol* for the hard relation;
4. *Transforming the Sigma protocol into a signature scheme*.

A well-known example of an MPC-in-the-head signature scheme that further serves as our starting point is the Picnic scheme [36] that follows the next stepping stones toward constructing their finalized (non-optimized) scheme.

1. *Selecting the hard relation*: Picnic’s underlying relation is given by a pseudorandom function (PRF). Namely, the public value is the couple (x, y) of input and output of the PRF, the secret value is the key k of the PRF F , and the pairs $((x, y), k)$ in the relation are those satisfying $y = F_k(x)$. Concretely, this PRF is instantiated with the LowMC block cipher [6].
2. *Choosing the MPC scheme*: The scheme of Giacomelli *et al.*, on which Picnic relies, exploits any linear function decomposition [58]. This is a notion introduced to capture protocol virtualization as done in MPC-in-the-head. For their particular instantiation, they consider a (2,3)-decomposition, which operates by performing secret sharing of the computations carried on the gates defining the arithmetic representation of F . First, the secret key k is additively shared into 3 shares k_1, k_2, k_3 . Then, linear operations are conducted on each share locally, while multiplication gates require interaction. The information involved in the computation of each output share y_1, y_2, y_3 is finally stored in the corresponding views $\text{view}_1, \text{view}_2, \text{view}_3$.
3. *Constructing the Sigma protocol*: Picnic is based on the ZKBoo protocol of Giacomelli *et al.* [58], which works as follows. The prover first runs the MPC protocol described above on the key k , resulting in a triple of key shares k_1, k_2, k_3 , output shares y_1, y_2, y_3 , and views $\text{view}_1, \text{view}_2, \text{view}_3$. Next, it commits to all the key shares and views and sends the commitments and output shares to the verifier. The verifier picks a couple of random indices i_1, i_2 and sends them to the prover. As a response, the prover reveals the key shares and views corresponding to the parties indexed by i_1, i_2 . Finally, the verifier can check three properties: (1) the correctness of two output shares based on the opened key shares and views, (2) the validity of two commitments, and (3) that the output shares reconstruct to the PRF output y . This process can then be repeated t times with the same long-term key k and different random coins in order to reduce the soundness error.
4. *Transforming the Sigma protocol into a signature scheme*: In the latest versions of Picnic [35], this is done via the Fiat-Shamir heuristic, both in the classic and quantum setting. Namely, the public key is the input-output PRF pair (x, y) , the signing key is the PRF key k , and a signature consists of a Sigma protocol transcript. To make it non-interactive, the challenge is computed by hashing the content of the public key, the first round of every iteration of the Sigma protocol and the message to be signed. The security proof is in the quantum random oracle model. It relies on several features of the Sigma protocol: completeness, unpredictable commitments, special soundness, zero-knowledge and quantum computationally unique responses.

Informally, the size and performances of Picnic signatures significantly depend on the number of multiplication gates within the circuit representation of the PRF F (since it affects the running time of the MPC protocol and the size of the views). This concern motivates our interest in key-homomorphic PRFs as a building block for MPC-in-the-head based signatures [31]. Key-homomorphic PRFs allow the computation on each key share independently, i.e., satisfy $F_k(x) = F_{k_1}(x) + \dots + F_{k_d}(x)$ for every d -additive sharing $k_1, \dots, k_d$ of the key k . In other words, they imply a “non-interactive MPC” scenario where each party contributes a single message to the MPC protocol. Consequently, key homomorphism eliminates the need for large views or interaction between the virtual parties, allowing us to increase the number of parties almost for free.

While this intuition holds regardless of the details of the key-homomorphic PRF, its specific choice affects security and efficiency. This is because each signature contains many PRF evaluations, one for each party and each iteration of the Sigma protocol. Therefore, we adopt the key-homomorphic PRF introduced by Boneh *et al.* [31], which is based on the Learning with Rounding (LWR) assumption [13]. Our primary motivation for choosing this scheme is its conceptual simplicity and, looking ahead to future work, its potential for leakage-resilient implementations via masking [48]. In this scheme, the PRF key is a random vector $\mathbf{k} \in \mathbb{Z}_Q^n$, the input is a random matrix $\mathbf{X} \in \mathbb{Z}_Q^{m \times n}$, and the output is computed as $\mathbf{y} = F_{\mathbf{k}}(\mathbf{X}) = \lfloor \mathbf{X}\mathbf{k} \rfloor_P \in \mathbb{Z}_P^m$, where $\lfloor x \rfloor_P = \left\lfloor x \cdot \frac{P}{Q} \right\rfloor$.³

A bit more precisely, this scheme only satisfies a weaker variant of key-homomorphism, known in the literature as *almost* key homomorphism. For a function F and key k , it ensures that $|F_k(x) - (F_{k_1}(x) + \dots + F_{k_d}(x))| \leq \delta$ for every additive sharing $k_1, \dots, k_d$ of the key, where δ is a small error. For our LWR-based PRF, this error scales with $\log(d)$ and intuitively describes the carry information that is preserved when *summing-before-rounding* and lost when *rounding-before-summing*. Introducing a small reconstruction error translates into several technical and conceptual challenges in the analysis and design of the scheme, forming the core of our technical contribution. In the following paragraphs, we outline these challenges and explain how we address them.

A modified verification check and its impact on the security of the hard relation. To ensure completeness, we include a small error tolerance in the verification check of the output shares, therefore modifying check (3) of Picnic (cf. [Item 3](#)). In other words, instead of checking the consistency of the output shares as $y_1 + \dots + y_d = y$, we check that $\mathbf{y} - (\mathbf{y}_1 + \dots + \mathbf{y}_d) \leq \delta$. Besides, and beyond completeness, we also target special soundness, a form of extractability for Sigma protocols (see [Definition 2](#) for a formal definition). To achieve it, we introduce a tolerance in the hard relation as well, defining it as

³ Formally, our analysis only requires *weak* PRF and we do not need to hash the input with a random oracle as Boneh *et al.* in order to get a PRF. We nevertheless use a hash function in our concrete instances, for performance reasons.

$\mathcal{R} = \{((\mathbf{X}, \mathbf{y}), \mathbf{k}) : |\mathbf{y} - F_{\mathbf{k}}(\mathbf{X})| \leq \delta\}$. Since the distance is measured component-wise and the vector $\mathbf{y} - F_{\mathbf{k}}(\mathbf{X})$ has m components, this relation is $(2\delta + 1)^m$ -times easier to break compared to a strict equality check (see [Lemma 1](#) for the security bound). Interestingly, we reduce this factor to 3^m when considering a setting where all-but-one shares are revealed in each protocol iteration, modifying check (3) accordingly. Intuitively, this is because having $d - 1$ additive shares out of d can be seen as having 1 additive share out of 2. In practice, in check (3), the verifier therefore aggregates the opened key shares $\mathbf{k}_{i_1}, \dots, \mathbf{k}_{i_{d-1}}$ into one, and then verifies almost key-homomorphism with the only unopened output share, *i.e.*, the verification check becomes $|\mathbf{y} - (F_{\mathbf{k}_{i_1} + \dots + \mathbf{k}_{i_{d-1}}}(\mathbf{X}) + \mathbf{y}_{i_d})| \leq 1$.

Average-case completeness and zero-knowledge. The standard zero-knowledge security definition quantifies over all statement-witness pairs $((\mathbf{X}, \mathbf{y}), \mathbf{k})$ in the relation, which implies that privacy holds even if the distinguisher knows the witness. However, this creates an issue with our simulation due to having almost key homomorphism: while the carry is deterministic for the distinguisher, it remains probabilistic for the simulator. To address this, we restrict ourselves to pairs generated through an external distribution, denoted as $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$. This notion, referred to as average-case zero-knowledge [\[23\]](#), resolves the simulation issue by ensuring that the distinguisher has no longer access to the witness. An orthogonal problem that arises with completeness further requires the switch to the average case definition. The problem arises because a statement-witness pair is in the relation if the absolute value difference between $\mathbf{y}$ and $F_{\mathbf{k}}(\mathbf{X})$ is less than 1. However, honestly-generated proofs for pairs satisfying $|\mathbf{y} - F_{\mathbf{k}}(\mathbf{X})| = 1$ might not be accepted. This is because the verifier compares $\mathbf{y}$ with $\tilde{\mathbf{y}}$, which it computes locally based on the decommitments and the unopened output shares. Since the reconstruction of $\tilde{\mathbf{y}}$ can also differ from $F_{\mathbf{k}}(\mathbf{X})$ by a factor 1, by the triangle inequality the difference between $\mathbf{y}$ and $\tilde{\mathbf{y}}$ can be 2 for some components, resulting in rejection. Therefore, we restrict ourselves to the set of instances in which $\mathbf{y} = F_{\mathbf{k}}(\mathbf{X})$. Looking ahead, this will not pose a problem for the security of our signature scheme because it is defined over honestly generated keys, and therefore, it suffices to generate the key pairs with $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$. Furthermore, the security of Fiat-Shamir signatures is often proved for Sigma protocols with constraints on the statement-witness distribution (cast as identification schemes), and thus will adapt to our security definition. See [Section 5](#) for the details.

The carry leakage information and the abort condition. The main technical challenge of this work concerns estimating how much information the carry leaks. Intuitively, this is because every iteration j of the Sigma protocol provides an eavesdropper with a noisy version $\mathbf{z}$ of $\mathbf{X}\mathbf{k}$, computable from the opened key shares $\mathbf{k}_{i_1}, \dots, \mathbf{k}_{i_{d-1}}$ and the unopened output share $\mathbf{y}_{i_d}$ as

$$\mathbf{z} = \mathbf{X}(\mathbf{k}_{i_1} + \dots + \mathbf{k}_{i_{d-1}}) + \frac{Q}{P}\mathbf{y}_{i_d} \in \left[\mathbf{X}\mathbf{k} - \frac{Q}{P} + 1, \mathbf{X}\mathbf{k} \right].$$

By averaging such samples over enough iterations, an eavesdropper could eventually isolate $\mathbf{X}\mathbf{k}$ and learn the secret. Although the scheme is already correct

as presented, this type of attack compromises its security. Interestingly, we show that an abort-based approach allows us to control the amount of information leaked.⁴ The abort event can be triggered in any iteration by the equality of any component of two vectors (the non-rounded output $\mathbf{X}\mathbf{k}$ and the sum of the non-rounded output shares $\mathbf{X}\mathbf{k}_{i_1} + \dots + \mathbf{X}\mathbf{k}_{i_{d-1}}$), in a fixed α -sized subset of their bits. This subset corresponds to the most significant α bits cut out by the rounding function, i.e., the ones between $r_S = \log(Q) - \log(P)$ and $r_E = \log(Q) - \log(P) + \alpha$. This condition ensures that the remaining transcripts do not reveal more than an LWR instance with α extra bits, as it intuitively guarantees that information on the (secret) LSBs of $\mathbf{y}$ cannot be forwarded to its (public) MSBs due to carry propagation. Since the abort condition excludes a $\approx t \cdot m \cdot 2^{-\alpha}$ fraction of the transcripts, we can increase the parameter α to make the abort probability arbitrarily small. We discuss in [Section 7](#) that combining such an increase with reasonable heuristics can significantly reduce the size of Bittersweet signatures. We also propose a formal approach to the carry leakage in [Section A](#), where we introduce two variants of the LWR problem and show that, under certain conditions, they can be reduced to the standard LWR.

Overall security. From the above arguments, our Sigma protocol satisfies average-case completeness and average-case honest-verifier zero-knowledge. Using similar arguments as for Picnic, we also prove that it achieves unpredictable commitments, special soundness, and quantum computationally unique responses, which are properties required for applying the Fiat-Shamir transform. At this point, the security of the signature follows from the vast literature on the post-quantum security of the Fiat-Shamir with abort transform. For correctness, we rely on the analysis of Devevey *et al.* [43], which, to our knowledge, is the only formal treatment of the subject. Regarding unforgeability, most analyses follow a modular framework: first prove unforgeability against no-message attack and then amplify to standard unforgeability. For the former, we consider the analysis of Don *et al.* [47], which, to the best of our knowledge, remains the state-of-the-art.⁵ Finally, we extend unforgeability against no-message attack to the standard notion via the recent result of Barbosa *et al.* [14]. See details in [Section 6](#).

3 Preliminaries

3.1 Notation

Given a set $\mathcal{S}$, $|\mathcal{S}|$ denotes its number of elements. If $N \in \mathbb{N}$, $\mathcal{S}^N$ denotes the N -times cartesian product, and $[N]$ denotes the set $\{1, \dots, N\}$. Given an element $z \in \mathbb{Z}_Q$ for some positive integer Q , $|z|$ denotes $\min\{z \bmod Q, -z \bmod Q\}$. Given a vector $\mathbf{v} \in \mathbb{Z}_Q^N$, $|\mathbf{v}|$ denotes the maximal component of $\mathbf{v}$ in absolute

⁴ A similar technique is used in the analysis of the NIST candidate Dilithium.

⁵ Although their results are not directly defined in the abort framework, the properties involved in their proofs do not seem to worsen with the introduction of aborts. We carefully discuss the adaptation of these results we need in [Section C.5](#).

value, and the notation $0 \in \mathbf{v}$ refers to the event where $\mathbf{v}$ has at least one zero component. Given two integers $Q > P$ and two rings $\mathcal{S}_Q, \mathcal{S}_P$ such that $|\mathcal{S}_Q| = Q$ and $|\mathcal{S}_P| = P$, the *rounding function* $\lfloor \cdot \rfloor_P : \mathcal{S}_Q \rightarrow \mathcal{S}_P$ is defined as $\lfloor x \rfloor_P = \left\lfloor x \frac{P}{Q} \right\rfloor$. When referring to the rounding of a vector, we consider its application component-wise. Given two integers p, i such that $i < p$, and a value $x \in \mathbb{Z}_{2^p}$, we denote with $x[i]$ the i -th bit of x . In particular, $x[0]$ is the least significant bit, and $x[p-1]$ is the most significant bit. Throughout the paper, we always consider $x[i]$ as an element of $\mathbb{Z}$ rather than $\mathbb{Z}_2$ or $\mathbb{Z}_{2^p}$. For two values i, j , we denote by $i..j$ the range of indices from i (included) to j (excluded), *i.e.* $i..j = \{i, i+1, \dots, j-1\}$. If $i = 0$ (resp., $j = p$), we omit it and simply write $..j$ (resp., $i..$). When we sum two ring elements, we always assume that the sum is the one for which the ring is defined. For example, if $x, y \in \mathbb{Z}_Q$, then we write $x + y$ for $x + y \bmod Q$, while if $x, y \in \mathbb{Z}_P$, then $x + y$ denotes $x + y \bmod P$.

3.2 The Computational Model

Our computational model assumes classical queries to the signing oracle as outlined in NIST’s call for Additional Digital Signature Schemes.⁶ However, we evaluate the security of our scheme in the Quantum Random Oracle Model (QROM), which extends the classical Random Oracle Model (ROM) to include adversaries making quantum superposition queries to the oracle.

3.3 Standard Cryptographic Primitives

We defer the definitions of signature schemes, pseudorandom generators, weak pseudo-random functions, and hash functions to [Section B](#).

3.4 Sigma Protocols

Let $\mathcal{X}, \mathcal{W} \subseteq \{0, 1\}^*$ be sets, and $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ be an efficiently decidable relation for the language $\mathcal{L} = \{x : \exists w \text{ s.t. } (x, w) \in \mathcal{R}\}$. A *Sigma protocol* Σ for $\mathcal{R}$ is an interactive protocol between two parties, called the *prover* and the *verifier*. The prover is modeled as a couple of probabilistic polynomial-time algorithms $\Sigma.\text{Prove} = (\Sigma.\text{Prove}_1, \Sigma.\text{Prove}_2)$, which are assumed to share state. The verifier $\Sigma.\text{Verify}$ is a deterministic polynomial-time algorithm that outputs a decision bit. Sigma protocols always consist of three rounds. In the first round, the prover generates a first message $\text{firstmes} \in \mathcal{N}_{\text{com}}$ as $\text{firstmes} \leftarrow^* \Sigma.\text{Prove}_1(x, w)$. Next, in the second round, the verifier answers with a uniformly random generated challenge $ch \leftarrow^* \mathcal{N}_{\text{ch}}$. In the third round, the prover publishes a response $\text{resp} \leftarrow^* \Sigma.\text{Prove}_2(ch)$. Finally, the verifier outputs $\Sigma.\text{Verify}(x, \text{firstmes}, ch, \text{resp}) \in \{0, 1\}$. We call *transcript* of the protocol the triple $(\text{firstmes}, ch, \text{resp})$.

We now introduce three key properties of Sigma protocols: *completeness*, *honest-verifier zero-knowledge*, and *s-special soundness*. The first ensures that

⁶ Mentioned on page 14 of <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf>

honestly generated proofs for statements in the language are always verified. Honest-verifier zero-knowledge guarantees that the transcript of an execution where the verifier issues a random challenge (and thus behaves honestly) reveals no more information on the witness beyond what could be derived from the statement alone. Finally, *s-special soundness* allows the reconstruction of a witness for a given statement from s transcripts, provided they meet specific criteria. These criteria include validity of the transcript, identical first messages, and distinct challenges. Similarly to [23], we consider the *average-case* variant of completeness and zero-knowledge, meaning that we restrict to statements generated according to precise distributions.⁷ This relaxes completeness to only hold for statements generated via the distribution, and therefore, not all the honestly generated proofs for statements in the language are correctly verified. Moreover, this weakens the adversaries considered in the experiment for zero-knowledge: their choices cannot depend on the internal coins used for sampling the statement, but only on the statement itself. We make these properties formal below, where the experiments refer to Fig. 1. We denote with $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ distributions that output values in $\mathcal{X} \times \mathcal{W}$, and with $\mathcal{D}_{\mathcal{X}}$ distributions that output values in $\mathcal{X}$.

Definition 1 (Completeness). *We say that $\Sigma = (\Sigma.\text{Prove}, \Sigma.\text{Verify})$ is $\epsilon_{\Sigma.\text{Comp}}$ -average-case complete for a distribution $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ if*

$$\mathbb{P}[\Sigma.\text{Comp}_{\Sigma} = 0] \leq \epsilon_{\Sigma.\text{Comp}}.$$

Definition 2 (Special Soundness). *Let Σ be a Sigma protocol for a relation $\mathcal{R}$. We say that $\Pi_{\mathcal{M}}$ has $(s, \epsilon_{\Sigma.\text{SS}})$ -special soundness if there exists an algorithm E such that for every quantum-polynomial-time adversaries A it holds that*

$$\mathbb{P}[\Sigma.\text{SS}(A, E) = 0] \leq \epsilon_{\Sigma.\text{SS}}.$$

Looking forward, we prove the unforgeability of the FS signatures *with abort* under a *computational* variant of HVZK. To this end, we merge the techniques used in [59] and [14]. The former proves unforgeability of FS signatures under a *computational* variant of HVZK named multi-HVZK, albeit only in the abort-free scenario. In the second work, unforgeability is proved in the setting *with aborts*, though under a *statistical* variant of HVZK named accepting HVZK.

Therefore, our proof requires a new variant of HVZK. As stated before, it is an *average-case* notion; this aligns with the definitions given in [59] and [14], which are formulated directly for identification schemes. To accommodate signatures with aborts, we adopt the *accepting* property from [14], requiring simulatability only for accepting transcripts. Additionally, we incorporate the *multi*-HVZK variant from [59], where HVZK holds across multiple transcripts.⁸

⁷ This is not a problem for the security of the resulting Fiat-Shamir signature. In the literature, the latter is usually built on top of identification schemes, which correspond to Sigma protocols equipped with a statement-witness generation algorithm.

⁸ As noted in [59], this stronger variant follows from statistical HVZK, but not from computational HVZK in the context of identification or average-case schemes.

$\Sigma.\text{Comp}()$	$\Sigma.\text{Tr}(x, w)$
$(x, w) \leftarrow \$ \mathcal{D}_{\mathcal{X} \times \mathcal{W}}$	$\text{firstmes} \leftarrow \$ \Sigma.\text{Prove}_1(x, w)$
$(\text{firstmes}, ch, \text{resp}) \leftarrow \$ \Sigma.\text{Tr}(x, w)$	$ch \leftarrow \$ \mathcal{N}_{\text{ch}}$
$b \leftarrow \Sigma.\text{Verify}(x, \text{firstmes}, ch, \text{resp})$	$\text{resp} \leftarrow \$ \Sigma.\text{Prove}_2(ch)$
if $(x, w) \in \mathcal{R}$ AND $b = 0$	return $(\text{firstmes}, ch, \text{resp})$
return 0	
return 1	$\Sigma.\text{View}_{\text{Real}, N}(x, w)$
$\Sigma.\text{SS}(\mathbf{A}, \mathbf{E})$	for $i \in [N]$
$(x, \text{firstmes}, (ch_c, \text{resp}_c)_{c \in [s]}) \leftarrow \$ \mathbf{A}()$	$\text{tr}_i \leftarrow \$ \Sigma.\text{Tr}(x, w)$
$w \leftarrow \$ \mathbf{E}(x, \text{firstmes}, (ch_c, \text{resp}_c)_{c \in [s]})$	return $(x, \text{tr}_1, \dots, \text{tr}_N)$
if $\exists c \neq c' \in [s]: ch_c = ch_{c'}$	$\Sigma.\text{View}_{\text{Sim}, N}(x)$
return 1	for $i \in [N]$
for $c \in [s]$	$\text{tr}_i \leftarrow \$ \Sigma.\text{Sim}(x)$
$b_c \leftarrow \Sigma.\text{Verify}(x, \text{firstmes}, ch_c, \text{resp}_c)$	return $(x, \text{tr}_1, \dots, \text{tr}_N)$
if $b_c = 0$	
return 1	
if $(x, w) \in \mathcal{R}$	
return 1	
return 0	

Fig. 1. The experiments relevant to the definitions in Section 3.4

Definition 3 (Average-Case Accepting Multi-HVZK). Let $N \in \mathbb{N}$ and $\epsilon_{\Sigma.\text{HVZK}} \in [0, 1]$. Let $\mathcal{X}, \mathcal{Y}$ be two sets, $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ be a relation, and $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be a distribution over $\mathcal{R}$. Let $\mathcal{D}_{\mathcal{X}}$ be the distribution that samples $(x, w) \leftarrow \$ \mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and outputs x . A Sigma protocol $\Sigma = (\Sigma.\text{Prove}, \Sigma.\text{Verify})$ for the relation $\mathcal{R}$ and the distribution $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ is average-case accepting $(N, \epsilon_{\Sigma.\text{HVZK}})$ -multi-HVZK ($(N, \epsilon_{\Sigma.\text{HVZK}})$ -acamHVZK for short) if there exists a polynomial-time simulator $\Sigma.\text{Sim}$ such that, for every stateful quantum polynomial-time adversary $\mathbf{A}$,

$$|\mathbb{P}[\mathbf{A}(\Sigma.\text{View}_{\text{Real}, N}(\mathcal{D}_{\mathcal{X} \times \mathcal{W}})) = 1 \mid \text{Success}] - \mathbb{P}[\mathbf{A}(\Sigma.\text{View}_{\text{Sim}, N}(\mathcal{D}_{\mathcal{X}})) = 1]|$$

is at most $\epsilon_{\Sigma.\text{HVZK}}$, where **Success** is the event where $\text{resp}_i \neq \perp$ for every $i \in [N]$.

Next, we present less standard properties for Sigma protocols: *commitment unpredictability* and *quantum computationally unique responses* (QCUR) [47]. The first concerns the probability of guessing the Sigma protocol's first message. The second translates the classical notion of computationally unique responses to the quantum world, stating that it is hard to detect a measurement over a superposition of successful responses to the same first message and challenge.

Definition 4 (Commitment Unpredictability). Let Σ be a Sigma protocol with statement space $\mathcal{X}$ and witness space $\mathcal{W}$, and let $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be a distribution

that outputs values in $\mathcal{X} \times \mathcal{W}$. We say that Σ has $\epsilon_{\text{ComGuess}}$ -unpredictable commitments with respect to $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ if

$$\mathbb{E} \left(\max_{\text{firstmes}_0 \in \mathcal{N}_{\text{com}}} \Pr[\text{firstmes} = \text{firstmes}_0] \right) \leq \epsilon_{\text{ComGuess}},$$

where the average is taken over $(x, w) \leftarrow \mathcal{D}_{\mathcal{X} \times \mathcal{W}}$, and the probability inside is taken over $\text{firstmes} \leftarrow \Sigma.\text{Prove}(x, w)$.

Formalizing quantum computationally unique responses requires the introduction of an additional definition, *i.e.*, the one of *collapsing relation*. This notion concerns generic relations and is used to argue about the verification stage of the Sigma protocol. Informally, it states that given a relation between two sets $\mathcal{S}_1$ and $\mathcal{S}_2$, if an adversary produces two superpositions S_1 and S_2 that satisfy the relation, then it cannot determine whether S_1 was measured.

Definition 5 (Collapsing Relation [47]). Let $\text{Rel} : \mathcal{S}_1 \times \mathcal{S}_1 \rightarrow \{0, 1\}$ be a relation over two sets $\mathcal{S}_1$ and $\mathcal{S}_2$ and define the following two games for a two-stage adversary $A = (A_1, A_2)$:

Game ₁ (A)	Game ₂ (A)
$(\text{state}, S_1, S_2) \leftarrow A_1$	$(\text{state}, S_1, S_2) \leftarrow A_1$
$r \leftarrow \text{Rel}(S_1, S_2)$	$r \leftarrow \text{Rel}(S_1, S_2)$
$S_1 \leftarrow \mathcal{M}(S_1)$	
$S_2 \leftarrow \mathcal{M}(S_2)$	$S_2 \leftarrow \mathcal{M}(S_2)$
$b \leftarrow A_2(\text{state}, S_1, S_2)$	$b \leftarrow A_2(\text{state}, S_1, S_2)$
return (b, r)	return (b, r)

In the games above, $\mathcal{M}$ denotes a measurement in the computational basis, and applying Rel to quantum registers is done by computing the relation coherently and measuring it. A relation Rel is called ϵ_{Coll} -collapsing from $\mathcal{S}_1$ to $\mathcal{S}_2$ if for every quantum-polynomial-time adversary A , it holds that

$$|\Pr[\text{Game}_1(A) = (1, 1)] - \Pr[\text{Game}_2(A) = (1, 1)]| \leq \epsilon_{\text{Coll}}.$$

Definition 6 (Quantum Computationally Unique Responses [47]). Let Σ be a Sigma protocol with first message space $\mathcal{N}_{\text{com}}$, challenge space $\mathcal{N}_{\text{ch}}$ and response space $\mathcal{N}_{\text{resp}}$. We say that the protocol Σ has ϵ_{QCUR} -quantum computationally unique responses if the verification predicate $\Sigma.\text{Verify}(x, \cdot, \cdot, \cdot) : \mathcal{N}_{\text{com}} \times \mathcal{N}_{\text{ch}} \times \mathcal{N}_{\text{resp}} \rightarrow \{0, 1\}$, seen as a relation between $\mathcal{N}_{\text{com}} \times \mathcal{N}_{\text{ch}}$ and $\mathcal{N}_{\text{resp}}$, is ϵ_{QCUR} -collapsing from $\mathcal{N}_{\text{resp}}$ to $\mathcal{N}_{\text{com}} \times \mathcal{N}_{\text{ch}}$.

3.5 Hard-Instance Generators

Let $\lambda \in \mathbb{N}$ be a security parameter, let $\mathcal{R}$ be a relation, and let $f(1^\lambda)$ be an efficient algorithm. Intuitively, f is a *hard instance generator* if it satisfies two properties. First, it outputs pairs in $\mathcal{R}$ with overwhelming probability. Second, given a public value p generated with f , it should be hard to construct a secret value s' such that $(p, s') \in \mathcal{R}$. We next report the formal definition from [81].

Definition 7 (Hard Instance Generator). f is called a $(\epsilon_{\text{Rel}}, \epsilon_{\text{HIG}})$ -hard instance generator for a relation $\mathcal{R}$ if the following conditions are met:

- $\mathbb{P}[(p, s) \notin \mathcal{R} \mid (p, s) \leftarrow \$ f] \leq \epsilon_{\text{Rel}};$
- for every quantum polynomial-time adversary $\mathbf{A}$, then $\mathbb{P}[f.\text{HIG}(\mathbf{A}) = 1] \leq \epsilon_{\text{HIG}}$, where $f.\text{HIG}(\mathbf{A})$ is the following experiment:

```

 $f.\text{HIG}(\mathbf{A})$ 
 $\hline$ 
 $(p, s) \leftarrow \$ f(1^\lambda)$ 
 $s' \leftarrow \$ \mathbf{A}(p)$ 
if  $(p, s') \in \mathcal{R}$ 
  return 1
return 0

```

By inspection, the notion of hard instance generator is closely related to that of *one-way function*. Notably, it is also possible to instantiate hard instance generators using weak PRFs. While this result is implicitly mentioned in the literature, the version we provide below explicitly incorporates the parameters of the PRF and the associated relation. In our case, the weak PRF takes values in $\mathcal{Y} = \mathbb{Z}_P^m$. Therefore, we tailor the following lemma to this scenario. However, it is easy to extend the result to generic sets $\mathcal{Y}$ where a distance is defined.

Lemma 1. Let $F: \mathcal{K} \times \mathcal{X} \rightarrow \mathbb{Z}_P^m$ be a $(N, \epsilon_{\text{wPRF}})$ -weak pseudorandom function for $N = 1$, and with ϵ_{wKR} -weak security against key recovery. Let f be the algorithm that samples k and x at random in $\mathcal{K}$ and $\mathcal{X}$, and outputs the pair $((x, F_k(x)), k)$. Consider the relation $\mathcal{R}_\delta = \{((x, y), k): |F_k(x) - y| \leq \delta\} \subseteq (\mathcal{X} \times \mathbb{Z}_P^m) \times \mathcal{K}$. Then f is a $(\epsilon_{\text{Rel}}, \epsilon_{\text{HIG}})$ hard-instance generator for the relation $\mathcal{R}$, where $\epsilon_{\text{Rel}} = 0$ and

$$\epsilon_{\text{HIG}} = \epsilon_{\text{wPRF}} + (2\delta + 1)^m \cdot \frac{|\mathcal{K}|}{P^m} \cdot \epsilon_{\text{wKR}}.$$

Proof (sketch). First, we observe that f always produces pairs $((x, y), k) \in \mathcal{R}$. This proves the first part of the definition of HIG with $\epsilon_{\text{Rel}} = 0$. For the second part of the definition, we consider the following hybrid experiments:

$\text{Hyb}^{(1)}(\mathbf{A})$ is defined as $f.\text{HIG}(\mathbf{A})$, but the PRF output y is sampled uniformly at random instead of being computed as $y = F_k(x)$.

$\text{Hyb}^{(2)}(\mathbf{A})$ is defined as $\text{Hyb}^{(1)}(\mathbf{A})$, but instead of checking that the key k' generated from $\mathbf{A}$ satisfies $|F_{k'}(x) - y| \leq \delta$, it outputs 1 only if $F_{k'}(x) = y$.

Then, we prove that for every quantum polynomial-time adversary $\mathbf{A}$ there exists a quantum polynomial-time adversary $\tilde{\mathbf{A}}$ such that

$$\left| \Pr[f.\text{HIG}(\mathbf{A}) = 1] - \Pr[\text{Hyb}^{(1)}(\mathbf{A}) = 1] \right| \leq \epsilon_{\text{wPRF}} \quad (1)$$

$$\Pr[\text{Hyb}^{(1)}(\mathbf{A}) = 1] = (2\delta + 1)^m \cdot \Pr[\text{Hyb}^{(2)}(\tilde{\mathbf{A}}) = 1] \quad (2)$$

$$\Pr[\text{Hyb}^{(2)}(A) = 1] \leq \frac{|\mathcal{K}|}{|\mathbb{Z}_P^m|} \cdot \epsilon_{\text{wKR}}. \quad (3)$$

Notably, the combination of Equation 1, 2 and 3 proves the lemma, so it remains to explain why these equations hold. Equation 1 derives from a standard reduction over the pseudorandomness of F . Equation 2 is true because guessing a key k' yielding one value in the interval $[y - \delta^m, y + \delta^m]$ is $(2\delta + 1)^m$ times easier than finding one exactly matching y . Equation 3 is true because finding a key k' such that $F_{k'}(x) = y$ is easier than finding the exact key k used for computing y from x . To quantify how easier it is, we use the fact that there are in average $\frac{|\mathcal{K}|}{|\mathbb{Z}_P^m|}$ keys k' for every possible (x, y) . Therefore, finding a key such that $F_{k'}(x) = y$ is $\frac{|\mathcal{K}|}{|\mathbb{Z}_P^m|}$ times easier than finding the exact key used for generating (x, y) . $\square$

The complete proof of the above lemma can be found in Section C.1.

3.6 Signatures from Sigma Protocols and Fiat-Shamir with Aborts

Similar to other well-known signatures, our scheme is constructed from a Sigma protocol via the Fiat-Shamir transform [56] with aborts [69] framework, which we present in Fig. 2. This construction is proved correct and post-quantum secure under various combinations of assumptions on the underlying Sigma protocol. In this work, we rely on the results of Devevey *et al.* [43], Fehr *et al.* [47], and Barbosa *et al.* [14], and report them below. Looking ahead, Section 6 contains an instantiation of the theorems from this section with the parameters of the hard-instance generator and the sigma protocol derived in Section 4 and 5.

$\text{S.Gen}(1^\lambda)$	$\text{Sign}_{sk}(msg)$
$(x, w) \leftarrow \mathcal{D}_{\mathcal{X} \times \mathcal{W}}(1^\lambda)$	while $resp = \perp$ do
$(vk, sk) \leftarrow (x, (x, w))$	$(state, firstmes) \leftarrow \Sigma.\text{Prove}_1(sk)$
return (vk, sk)	$ch \leftarrow H(firstmes msg)$
	$resp \leftarrow \Sigma.\text{Prove}_2(state, ch)$
$\text{S.Verify}(vk, msg, \sigma)$	$\sigma \leftarrow (firstmes, resp)$
$(firstmes, resp) \leftarrow \sigma$	return σ
$ch \leftarrow H(firstmes msg)$	
return $\Sigma.\text{Verify}(vk, firstmes, ch, resp)$	

Fig. 2. The Signature Scheme $\Pi_{\text{Sign}} = (\text{S.Gen}, \text{Sign}, \text{S.Verify})$ constructed via Fiat-Shamir with Abort. $\Sigma = (\Sigma.\text{Prove}_1, \Sigma.\text{Prove}_2, \Sigma.\text{Verify})$ is any Sigma protocol for some relation $\mathcal{R}$. $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ is any distribution generating a pair of statement and witness in $\mathcal{R}$.

Analyzing correctness in the abort setting is a bit tedious [43], as many parameters come into play. The first one is the completeness parameter $\epsilon_{\Sigma.\text{Comp}}$ of the sigma protocol, as the signature verification is strongly related to the one of the sigma protocol. The probability of abort in an honest execution of the sigma

protocol ϵ_{HAbort} , together with the number of iterations κ of the Fiat-Shamir with aborts construction, are related to the probability that **Sign** does not abort. Moreover, a sufficiently high min-entropy of the first message ν guarantees that the challenges generated with the random oracle are uniformly random. This intuition is made formal in Theorem 6 of [43], which we next simplify.

Theorem 1 (Correctness, [43], restated). *Let H be a hash function modeled as a random oracle. Let Σ be a sigma protocol that meets the following properties. First, the probability of aborting in an honest execution should be at most ϵ_{HAbort} . Moreover, every non-aborting honest execution should be accepted, i.e. $\epsilon_{\Sigma, \text{Comp}}$ -average-case completeness holds with $\epsilon_{\Sigma, \text{Comp}} = \epsilon_{\text{HAbort}}$.⁹ Finally, for every statement and witness couple generated with $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$, the min entropy of the first message should be at least ν . Let Π_{Sign} be the signature scheme constructed from $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and Σ via the Fiat-Shamir with Abort transform (cf. Fig. 2). Then, for any (vk, sk) generated with $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and any message msg , Π_{Sign} satisfies ϵ_{Corr} -correctness with*

$$\epsilon_{\text{Corr}} = \frac{2^{-\nu}}{(1 - \epsilon_{\text{HAbort}})^3}.$$

The next result concerns the security of the signature scheme against no-message attacks. As the presence of aborts does not play a role here, the main challenges in the security analysis arise from the increased capabilities of the adversary in the QROM. In the classical ROM, unforgeability is typically reduced to the soundness of the underlying Sigma protocol by measuring the adversary's oracle queries. However, extending this approach to the quantum world presents additional hurdles, as it is impossible to measure superposition queries without disturbing them. The work of [47] provides a comprehensive treatment of this scenario, and we condense their Theorem 21 and 25 in Theorem 2 below.

The first requirement for unforgeability against no-message attacks (UF-NMA) is that the key pair is generated with an hard-instance generator. This guarantees, intuitively, that an adversary A can hardly recover a valid signing key from the public key only. UF-NMA is achieved in combination with another important property: any adversary capable of forging a signature given only the public key must, with high probability, also know the underlying signing key. In the quantum setting, this property can be reduced to the sigma protocol level up to a multiplicative factor $O(N_H^2)$ and an additive factor $\frac{1}{N_{HC}}$ (see Theorem 8 in [47]). This results in the sigma protocol property known as *proof of knowledge*. Theorem 21 in [47] formalizes the intuitions given up to this point. Namely, a sigma protocol with the proof of knowledge property and superpolynomial challenge space, combined with a hard-instance generator in the Fiat-Shamir heuristic, guarantees unforgeability against no-message attacks.

In Theorem 25 in [47], the authors investigate the assumptions implying that a sigma protocol is a proof of knowledge, made formal by considering an algorithm E extracting a witness from an interaction with the adversary. In the

⁹ Theorem 6 of [43] also holds for the more general case where $\epsilon_{\Sigma, \text{Comp}} \geq \epsilon_{\text{HAbort}}$.

proof of Theorem 25 in [47], E runs the malicious prover A to the point where it outputs the first message. Then, it samples a random challenge and sends it to A, obtaining a response by measuring A's corresponding register. Next, E rewinds A on the measured state, and chooses and sends to A a fresh random challenge, resulting in a response. If both transcripts verify, then E can compute the witness w using the extractor E_{SS} corresponding to the $(s, \epsilon_{\Sigma, SS})$ -special soundness property with $s = 2$. This strategy fails if the fresh challenge equals the first one and if any of the measurements is detected. This motivates the factor $1/C$ and the assumption that the scheme has ϵ_{QCUR} -quantum computationally unique responses. We refer to Theorem 21 and 25 in [47] for a formal treatment and to Section C.5 for a more detailed explanation of how Theorem 2 follows.

Theorem 2 (Unforgeability against no-message attack, [47], restated). *Let Σ be a Sigma protocol for a relation $\mathcal{R}$ with challenge space size C , $(s, \epsilon_{\Sigma, SS})$ -special soundness with $s = 2$ and $\epsilon_{\Sigma, SS} = 0$ (cf. Definition 2), and ϵ_{QCUR} -quantum computationally unique responses (cf. Definition 6).¹⁰ Let $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be a ϵ_{HIG} -hard instance generator according to Definition 7, yielding a pair of statements and witnesses in $\mathcal{R}$. Let Π_{Sign} be the signature scheme constructed from $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and Σ via the Fiat-Shamir with Abort transform (cf. Fig. 2). Then Π_{Sign} is $(\epsilon_{Sign, 0}, N_H)$ -unforgeable against no-message attack, with*

$$\epsilon_{Sign, 0} \leq \left(\epsilon_{HIG} \cdot N_H^6 + \frac{1}{C} + 2\epsilon_{QCUR} + \frac{1}{O(N_H^5) \cdot C} + \frac{1}{8N_H^3 C^3} \right)^{\frac{1}{3}}.$$

Proof (sketch). Theorem 2 is a direct application of two slightly adapted variants of Theorem 25 and Theorem 21 of [47]. Below, we detail the modification to each theorem and motivate why the resulting statements stay valid.

Theorem 25 assumes generic t -soundness (cf. [47], section 5), while we assume $(s, \epsilon_{\Sigma, SS})$ -special soundness. Since special soundness implies t -soundness, the theorem continues to hold. Looking ahead, we directly instantiate $(s, \epsilon_{\Sigma, SS}) = (2, 0)$, as our sigma protocol satisfies special soundness with these particular parameters. Moreover, we make explicit the parameters defining the proof of knowledge (cf. [47], definition 14), yielding $p = 1, d = 3$, and $\kappa = \frac{1}{C} + 2\epsilon_{QCUR}$. These values are derived from a step-by-step revision of the proof of Theorem 25 and of the underlying technical lemma ([47], Lemma 15). The latter proves that proofs of knowledge for static adversaries extend to adaptive adversaries, and through our revision we prove that this extension leaves the parameters p, d, κ unchanged.

Theorem 21 proves unforgeability against no message attacks for standard Fiat-Shamir signatures, while our variant refers to the Fiat-Shamir *with abort* transform (cf. Fig. 2). However, the possibility of abort only reduces the probability that an adversary can generate an accepting signature, therefore we can keep the same bounds. Since we apply Theorem 21 to the proof of knowledge

¹⁰ The authors prove this result for generic $(s, \epsilon_{\Sigma, SS})$ -special soundness. For simplicity, we only report the concrete case of $s = 2$ and $\epsilon_{\Sigma, SS} = 0$.

from Theorem 25, where $d = 3$, we compute the parameters for Theorem 21 under this assumption, yielding unforgeability with

$$\epsilon_{\text{Sign},0} \leq \left(\epsilon_{\text{HIG}} N_H^6 p(\eta) + \kappa(\eta) + \frac{1}{p(\eta)} \cdot \left(\frac{1}{8N_H^3 C^3} + \frac{1}{\mathcal{O}(N_H^5 C)} \right) \right)^{1/3}.$$

The parameter computation derives again from a detailed revision of the proof of Theorem 21 and the underlying Corollary (Corollary 16 in [47]). For the latter, we observe that the proof of knowledge property of $\text{FS}(\Sigma)$ holds with the same parameters p, d as in Σ , and with $e = 6, \mu(\eta) = \kappa(\eta) + \frac{1}{p(\eta)} \cdot \left(\frac{1}{8N_H^3 C^3} + \frac{1}{\mathcal{O}(N_H^5 C)} \right)$, where N_H is the number of queries to the random oracles made by the attacker to $\text{FS}(\Sigma)$, under the assumption that $d = 3$. $\square$

Lifting unforgeability against no-message attacks to standard unforgeability requires, first of all, an underlying zero-knowledge sigma protocol. This holds because the adversary can try to forge based on each of the N_{Sign} previous signatures, which in this case are sigma protocol transcripts. In turn, relating the signatures to sigma protocol conversations requires arguing about the randomness of the challenges produced by the RO. This is strictly related to the entropy of the input to the random oracle, which includes the first message, motivating the assumption of $\epsilon_{\text{ComGuess}}$ -unpredictable commitments.

At the proof level, using the ZK simulator implies reprogramming the random oracle accordingly, which poses several technical challenges in the quantum setting. The presence of aborts further complicates the analysis, since reprogramming introduces a bias toward first messages corresponding to non-aborting transcripts. Barbosa *et al.* [14] formalize this intuition in their Theorem 2, and we report a slightly simplified version of it below.¹¹

Theorem 3 (Unforgeability, [14], restated for Multi-HVZK). *Let $N_{\text{Sign}}, N_H \in \mathbb{N}$. Let $\mathcal{R}$ be a relation, and $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be a distribution yielding a pair of statements and witnesses in $\mathcal{R}$. Let Σ be an average-case accepting $(N_{\text{Sign}}, \epsilon_{\Sigma, \text{HVZK}})$ -multi-HVZK Sigma protocol for the relation $\mathcal{R}$ and the distribution $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$. Denote with ϵ_{HAbort} the probability of abort in an honest execution of Σ , and with $\epsilon_{\text{ComGuess}}$ the commitment-guessing probability, as defined in Definition 4. Let Π_{Sign} be the signature scheme constructed from $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and Σ via the Fiat-Shamir transform with Abort (cf. Fig. 2). Assume that the attacker $\mathbf{A}$ to unforgeability makes N_{Sign} classical queries to the signing oracle and N_H quantum queries to the random oracle H . Further, assume that Π_{Sign} is $(\epsilon_{\text{Sign},0}, N_H)$ -unforgeable against no-message attacks. Then Π_{Sign} is also $(\epsilon_{\text{Sign}, N_{\text{Sign}}}, N_H, N_{\text{Sign}})$ -unforgeable, with*

$$\begin{aligned} \epsilon_{\text{Sign}, N_{\text{Sign}}} \leq & \epsilon_{\text{Sign},0} + \frac{2N_{\text{Sign}} \sqrt{\epsilon_{\text{ComGuess}}}}{1 - \epsilon_{\text{HAbort}}} \sqrt{N_H + 1 + \frac{N_{\text{Sign}}}{1 - \epsilon_{\text{HAbort}}}} \\ & + 2(N_H + 1) \sqrt{\frac{N_{\text{Sign}} \epsilon_{\text{ComGuess}}}{1 - \epsilon_{\text{HAbort}}}} + \epsilon_{\Sigma, \text{HVZK}}. \end{aligned}$$

¹¹ The main difference is that their result is defined over generic events Γ , while in the following, we restrict to Γ being the event that is always true.

Proof (sketch). The proof is similar to the one in [14]. The only difference is the need to replace the real proofs with simulated ones all at once, which is possible thanks to the new definition of Average-Case Accepting Multi-HVZK. $\square$

We refer the reader to Section C.6 for a more detailed explanation of the differences between the work of [14] and the proof of Theorem 3.

4 Learning with Rounding

The *learning with rounding* (LWR) assumption is a deterministic variant of the well-known *learning with errors* (LWE) assumption, differing in how noise is added to the result. In the LWE case, a small error is added to the product, whereas in LWR, the product is instead *rounded*. Given two integers Q, P such that $Q > P$, the *rounding function* $\lfloor \cdot \rfloor_P : \mathbb{Z}_Q \rightarrow \mathbb{Z}_P$ is defined as $\lfloor x \rfloor_P := \left\lfloor x \cdot \frac{P}{Q} \right\rfloor$, where $\lfloor \cdot \rfloor$ denotes the floor function over rational numbers. We can extend the rounding function $\lfloor \cdot \rfloor_P$ to operate on vectors: if $\mathbf{x} \in \mathbb{Z}_Q^m$ is a vector, $\lfloor \mathbf{x} \rfloor_P$ simply applies the $\lfloor \cdot \rfloor_P$ to every component, yielding a vector in $\mathbb{Z}_P^m$. The inherent properties of the rounding function, including almost key-homomorphism, form the base for the technical results of this work. They are detailed in Section D.

For parameters $Q, P, n, m \in \mathbb{N}$ such that $Q > P$, and a vector $\mathbf{k} \in \mathbb{Z}_Q^n$, we define the LWR distribution (and the corresponding random distribution) as:

$\text{LWR}(Q, P, n, m) . \text{Real}(\mathbf{k})$	$\text{LWR}(Q, P, n, m) . \text{Rand}()$
$1 : \mathbf{X} \leftarrow \$_{\mathbb{Z}_Q^{m \times n}}, \mathbf{y} \leftarrow \lfloor \mathbf{X} \mathbf{k} \rfloor_P$	$\mathbf{X} \leftarrow \$_{\mathbb{Z}_Q^{m \times n}}, \mathbf{y} \leftarrow \$_{\mathbb{Z}_P^m}$
$2 : \text{return } (\mathbf{X}, \mathbf{y})$	$\text{return } (\mathbf{X}, \mathbf{y})$

When it is clear from the context, we just omit the parameters and write $\text{LWR} . \text{Real}(\mathbf{k})$ (*resp.*, $\text{LWR} . \text{Rand}()$) or just $\text{Real}(\mathbf{k})$ (*resp.*, $\text{Rand}()$). The LWR assumption is defined from these experiments, and states that it is hard to distinguish the rounded matrix product from a fresh random vector.

Definition 8 (LWR [13]). Let $Q, P, n, m \in \mathbb{N}$ such that $Q > P$. The *decisional LWR assumption* holds with advantage ϵ_{LWR} if, $\forall$ quantum PPT adversary $\mathbf{A}$,

$$|\mathbb{P}[\mathbf{A}(\text{LWR} . \text{Real}(\mathbf{k})) = 1 \mid \mathbf{k} \leftarrow \$_{\mathbb{Z}_Q^n}] - \mathbb{P}[\mathbf{A}(\text{LWR} . \text{Rand}()) = 1]| \leq \epsilon_{\text{LWR}}.$$

4.1 A LWR-Based Weak Pseudorandom Function

We now introduce the weak pseudorandom function (wPRF) used in our construction and analyze two key properties: its security as a weak pseudorandom function (wPRF) and as a hard-instance generator. We consider a natural LWR-based wPRF, $F_{\mathbf{k}} : \mathbb{Z}_Q^n \times \mathbb{Z}_Q^{m \times n} \rightarrow \mathbb{Z}_P^m(\mathbf{X})$ such that $\mathbf{y} \leftarrow \lfloor \mathbf{X} \mathbf{k} \rfloor_P$.

LWR implies that F is a weak PRF, with the remark that an increased number of wPRF queries corresponds to a matrix with more rows.

Lemma 2. Consider $F_{\mathbf{k}}$ with parameters $P, Q, n, m \in \mathbb{N}$ such that $P < Q$. Let $N \in \mathbb{N}$, and assume that the LWR assumption holds with probability ϵ_{LWR} over the same parameters, apart from the number of rows of the random matrix being $\tilde{m} = Nm$. Then F is a $(N, \epsilon_{\text{wPRF}})$ -weak PRF, with

$$\epsilon_{\text{wPRF}} \leq \epsilon_{\text{LWR}}.$$

Proof (sketch). To show that F is a weak PRF, we reduce its security to the LWR assumption. In wPRF.Real and wPRF.Rand the adversary receives N random matrices with either rounded products $\lfloor \mathbf{X}_i \mathbf{k} \rfloor$ or random outputs. By stacking these matrices (and outputs) row-wise, the distribution is identical to sampling a single large matrix and vector in the LWR experiments. $\square$

The full proof of Lemma 2 is deferred to Section C.2.

As a result, LWR implies that F is a hard instance generator.

Lemma 3. Consider $F_{\mathbf{k}}$ with parameters $P, Q, n, m \in \mathbb{N}$ such that $P < Q$. Assume that the LWR assumption holds with probability ϵ_{LWR} over the same parameters, apart from the number of rows of the random matrix being $\tilde{m} = Nm$, with $N \geq 2$. Let $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be the algorithm that samples $\mathbf{k}, \mathbf{X}$ at random in $\mathbb{Z}_Q^n \times \mathbb{Z}_Q^{m \times n}$, and outputs the pair $((\mathbf{X}, F_{\mathbf{k}}(\mathbf{X})), \mathbf{k})$. Consider the relation $\mathcal{R} = \{((\mathbf{X}, \mathbf{y}), \mathbf{k}) : |F_{\mathbf{k}}(\mathbf{X}) - \mathbf{y}| \leq 1\} \subseteq (\mathbb{Z}_Q^{m \times n} \times \mathbb{Z}_P^m) \times \mathbb{Z}_Q^n$. Then f is a $(\epsilon_{\text{Rel}}, \epsilon_{\text{HIG}})$ hard-instance generator for the relation $\mathcal{R}$, where $\epsilon_{\text{Rel}} = 0$ and

$$\epsilon_{\text{HIG}} \leq \epsilon_{\text{LWR}} \left(1 + 3^m \frac{Q^n}{P^m} \right) + 3^m \frac{Q^n}{P^{2mN}}.$$

Proof (sketch). This is a direct application of Lemma 1, where ϵ_{wPRF} is bounded according to Lemma 2, and ϵ_{wKR} is upper bounded in terms of ϵ_{wPRF} via Lemma 7. The full proof of Lemma 3 appears in Section C.3.

5 A Sigma Protocol Based on LWR

In this section, we introduce our Sigma protocol and prove its completeness, honest-verifier zero-knowledge, special soundness, commitment unpredictability, and quantum computationally unique responses. Our protocol follows a similar approach as previous MPC-in-the-head-based schemes like ZKBoo [58, 36] and ZKB++ [36], but is tailored to the almost key-homomorphic functionality introduced in Section 4.1. Its full description is given in Fig. 3.

We start with a high-level overview on its main features, including the prover's computation of the first message and response, the verification algorithm, and the abort condition. The first message is computed by creating key shares $\mathbf{k}_i$'s, and then returning their hash-based commitments h_i 's together with the local PRF evaluations $\mathbf{y}_i$, for every iteration $j \in [t]$. Upon receiving a challenge, the prover checks whether the abort event holds and opens the committed shares if not. The verifier then checks that the openings are computed correctly, that the

$\Sigma.\text{Prove}_1((\mathbf{X}, \mathbf{y}), \mathbf{k})$ for $j \in [t]$ $(\mathbf{k}_1^{(j)}, \dots, \mathbf{k}_d^{(j)}) \leftarrow \text{\$ Share}(\mathbf{k})$ for $i \in [d]$ $\mathbf{z}_i^{(j)} \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i^{(j)}, i)$ $\mathbf{y}_i^{(j)} \leftarrow \lfloor \mathbf{z}_i^{(j)} \rfloor_{2^{p-\alpha}}$ $\rho_i^{(j)} \leftarrow \text{\$ } \mathcal{P}_H$ $h_i^{(j)} \leftarrow H(\mathbf{k}_i^{(j)}, \rho_i^{(j)})$ $\mathbf{y}_{ext} \leftarrow \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^p}$ $state \leftarrow (\mathbf{X}, \mathbf{y}_{ext}, (\mathbf{k}_i^{(j)}, \rho_i^{(j)})_{i \in [d], j \in [t]})$ $firstmes \leftarrow (\mathbf{y}_i^{(j)}, h_i^{(j)})_{i \in [d], j \in [t]}$ return $firstmes, state$ $\Sigma.\text{Prove}_2(state, ch)$ $(\mathbf{X}, \mathbf{y}_{ext}, (\mathbf{k}_i^{(j)}, \rho_i^{(j)})_{i \in [d], j \in [t]}) \leftarrow state$ $(\mathcal{I}^{(j)})_{j \in [t]} \leftarrow ch$ if $\text{Abort}(state, ch) = 1$ $resp \leftarrow \perp$ else $resp \leftarrow (\mathbf{k}_i^{(j)}, \rho_i^{(j)})_{i \in \mathcal{I}^{(j)}, j \in [t]}$ return $resp$ $\text{Share}(\mathbf{k})$ $(\mathbf{k}_1, \dots, \mathbf{k}_{d-1}) \leftarrow \text{\$ } \mathcal{P}_K$ $\mathbf{k}_d = \mathbf{k} - \sum_{i \in [d-1]} G_K(\mathbf{k}_i)$ return $(\mathbf{k}_i)_{i \in [d]}$ $\text{Output}(\mathbf{X}, \tilde{\mathbf{k}}, i)$ if $i \in [d-1]$ $\mathbf{k} \leftarrow G_K(\tilde{\mathbf{k}})$ else $\mathbf{k} \leftarrow \tilde{\mathbf{k}}$ $\mathbf{z} \leftarrow \mathbf{X}\mathbf{k}$ return $\mathbf{z}$	$\Sigma.\text{Verify}((\mathbf{X}, \mathbf{y}), firstmes, ch, resp)$ $(\mathbf{y}_i^{(j)}, h_i^{(j)})_{i \in [d], j \in [t]} \leftarrow firstmes$ $(\mathcal{I}^{(j)})_{j \in [t]} \leftarrow ch$ if $resp = \perp$ return 0 $(\mathbf{k}_i^{(j)}, \rho_i^{(j)})_{i \in \mathcal{I}^{(j)}, j \in [t]} \leftarrow resp$ for $j \in [t]$ for $i \in \mathcal{I}^{(j)}$ if $H(\mathbf{k}_i^{(j)}, \rho_i^{(j)}) \neq h_i^{(j)}$ return 0 $\mathbf{z}_i^{(j)} \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i^{(j)}, i)$ if $\lfloor \mathbf{z}_i^{(j)} \rfloor_{2^{p-\alpha}} \neq \mathbf{y}_i^{(j)}$ return 0 $sum^{(j)} \leftarrow \sum_{i \in \mathcal{I}^{(j)}} \mathbf{z}_i^{(j)}$ $i^* \leftarrow [d] \setminus \mathcal{I}^{(j)}$ $\tilde{\mathbf{y}}^{(j)} = \lfloor sum^{(j)} \rfloor_{2^{p-\alpha}} + \mathbf{y}_{i^*}^{(j)}$ if $\mathbf{y} - \tilde{\mathbf{y}}^{(j)} \notin \{0, 1\}$ return 0 return 1 $\text{Abort}(state, ch)$ $(\mathbf{X}, \mathbf{y}_{ext}, (\mathbf{k}_i^{(j)}, \rho_i^{(j)})_{i \in [d], j \in [t]}) \leftarrow state$ $(\mathcal{I}^{(j)})_{j \in [t]} \leftarrow ch$ for $j \in [t]$ for $i \in \mathcal{I}^{(j)}$ $\mathbf{z}_i^{(j)} \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i^{(j)}, i)$ $sum^{(j)} \leftarrow \sum_{i \in \mathcal{I}^{(j)}} \mathbf{z}_i^{(j)}$ if $0 \in sum^{(j)}[r_S..r_E] - \mathbf{y}_{ext}[0..\alpha]$ return 1 return 0
--	--

Fig. 3. The Sigma protocol $\Sigma = (\Sigma.\text{Prove}_1, \Sigma.\text{Prove}_2, \Sigma.\text{Verify})$ and its subprocedures Share, Output and Abort. The statement is $(\mathbf{X}, \mathbf{y})$ and the witness is $\mathbf{k}$. The challenge $ch = (\mathcal{I}^{(j)})_{j \in [t]}$ is a tuple of t subsets of $[d]$ with $d-1$ elements each. In the subprocedures Share and Output, $G_K: \mathcal{P}_K \rightarrow \mathbb{Z}_{2^q}^n$ is a pseudorandom generator.

$\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ $\mathbf{k} \leftarrow \mathbb{Z}_{2^q}^n$ $\mathbf{X} \leftarrow \mathbb{Z}_Q^{m \times n}$ $\mathbf{y} = \lfloor \mathbf{X} \mathbf{k} \rfloor_{2^{p-\alpha}}$ return $((\mathbf{X}, \mathbf{y}), \mathbf{k})$

Fig. 4. The distribution $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ generating statements and witnesses for the Σ protocol. key shares correspond to the output shares, and that the output shares (almost) sum up to $\mathbf{y}$, up to a difference $\mathbf{1}$. The abort event can be triggered in any iteration by the equality of any component of two vectors, namely the non-rounded output and the sum of the non-rounded output shares, in a fixed α -sized subset of their bits. This subset corresponds to the most significant α bits cut out by the rounding function, i.e., the ones between $r_S = q - p$ and $r_E = q - p + \alpha$.

Theorem 4 (Completeness). *Let Σ be the sigma protocol described in Fig. 3, $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be the distribution described in Fig. 4, and consider the relation $\mathcal{R} = \{((\mathbf{X}, \mathbf{y}), \mathbf{k}) : |\mathbf{y} - \lfloor \mathbf{X} \mathbf{k} \rfloor_{2^{p-\alpha}}| \leq 1\}$. Assume that decisional LWR holds with parameters $(2^q, 2^p, n, m)$ and advantage ϵ_{LWR} , and that G_K is a $\epsilon_{\text{PRG}, K}$ -secure PRG. Then Σ is a $\epsilon_{\Sigma, \text{Comp}}$ -average-case complete sigma protocol for the relation $\mathcal{R}$ and for the distribution $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$, where*

$$\epsilon_{\Sigma, \text{Comp}} = \epsilon_{\text{HAbort}} = tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, K} + \epsilon_{\text{LWR}},$$

where ϵ_{HAbort} denotes the probability of aborting in an honest execution.

Proof (sketch). As long as the abort condition is not triggered and the statement-witness pair is generated with $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$, an honestly-generated proof always passes the verification checks. This makes completeness only depend on the probability of aborts. Recall that the abort condition occurs when, in any of the t iterations and for any of the m components, α bits of two vectors match. Assuming that G is a PRG and that LWR holds with parameters $(2^q, 2^p, n, m)$ ensures that one of these vectors, namely $\mathbf{y}_{\text{ext}}$, is pseudorandom. This reduces the problem to upper bounding the probability that $t \cdot m$ α -bit strings equal a random one, which is at most $t \cdot m \cdot 2^{-\alpha}$ by union bound. The full proof is deferred to Section E.1.

Proving average-case accepting multi-honest-verifier zero-knowledge poses significant challenges, which we tackle in Section E.2 and summarize in the next lines. In this setting, we wish to prove that an adversary observing a non-aborting transcript hardly gets information about the secret key. The main difficulty comes from the use of an almost-key homomorphic PRF, as potential reconstruction errors could leak information about the secret. Therefore, we need to carefully quantify the possible leakage. As part of the proof we present an algorithm, named Sim^* , able to simulate a full transcript without knowledge of the secret key. However, this comes with certain constraints: Sim^* needs α additional bits of the public key to function properly, and has a rather high probability of

aborting. Due to the first constraint, zero-knowledge reduces to LWR with parameters $(2^q, 2^p, n, m)$, even though the public key itself is only in $\mathbb{Z}_{2^{p-\alpha}}^n$. To deal with the second constraint, we use a common technique from the literature: we define an algorithm Sim that runs Sim^* ω times, drastically reducing the probability of aborts. This motivates the presence of ω in the statement of [Theorem 5](#). The factor $t\epsilon_{\text{Hide}}$ corresponds to the attack where any of the t unopened shares can be recomputed from the first message, leading to complete key leakage. The remaining terms correspond, as in [Theorem 4](#), to the probability of aborts, and the way they appear is influenced by how this probability changes across the hybrid experiments in the proof. Finally, all of the above only holds for every transcript considered separately; when joining all the N transcripts together, we have to take into account a factor N that multiplies everything else.

Theorem 5 (Average-Case Accepting Multi-HVZK). *Let $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be the distribution described in [Fig. 4](#), and let $\omega \in \mathbb{N}$. Assume that*

- *decisional LWR holds with parameters $(2^q, 2^p, n, m)$ and advantage ϵ_{LWR} ,*
- *$G_{\mathcal{K}}$ is a ϵ_{PRG} -secure PRG, and*
- *the canonical commitment scheme for $\mathcal{H}$ is ϵ_{Hide} -hiding.*

Consider the relation $\mathcal{R} = \{((\mathbf{X}, \mathbf{y}), \mathbf{k}) : |F_{\mathbf{k}}(\mathbf{X}) - \mathbf{y}| \leq 1\}$, where F is the weak PRF defined in [Section 4.1](#). Then Σ satisfies $(N, \epsilon_{\Sigma, \text{HVZK}})$ -acmHVZK for the relation $\mathcal{R}$ and the distribution $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$, where $N \in \mathbb{N}$ and

$$\epsilon_{\Sigma, \text{HVZK}} = Nt\epsilon_{\text{Hide}} + \frac{2Nt\epsilon_{\text{PRG}} + 4\epsilon_{\text{LWR}}}{1 - N \cdot t \cdot m \cdot 2^{-\alpha}} + 2\gamma^\omega,$$

where $\gamma \leq N \cdot t \cdot m \cdot 2^{-\alpha}$.

Proof (sketch). We split the experiments in smaller components, and then shift from the real experiment to the simulated one via hybrid arguments that gradually replace such components. Namely, the main components are the secret sharing algorithm Share , the PRG-based sampler Output , the commitment function $\mathcal{H}$, the challenge distribution $\mathcal{N}_{\text{ch}}$, and the LWR problems LWR, LWR' with different parameters. We summarize the key steps below.

1. We move the challenge generation at the beginning of the experiment, since the distribution $\mathcal{N}_{\text{ch}}$ does not depend on anything else. This allows to know the index i^* of the unopened share in each iteration from the start.
2. We replace the commitment values $\mathcal{H}(\mathbf{k}_{i^*})$ with commitment values of uniform, unrelated values. By reduction to the ϵ_{Hide} -hiding property of $\mathcal{H}$, applied for N messages and t iterations per message, it incurs in a distinguishing advantage $Nt\epsilon_{\text{Hide}}$.
3. Now we consider the event **Abort** that happens when the middle α bits of the reconstructed sum match the ones in the original $\mathbf{y}_{\text{ext}}$ vector in at least one component (i.e., when the **Abort** function outputs 1).
 - When $\mathbf{y}_{\text{ext}}$ comes from LWR.Rand , this event happens with probability $2^{-\alpha}$ for each component of the vector, or, by applying the union bound, with an overall probability of at most $t \cdot m \cdot 2^{-\alpha}$ for each message.

- When $\mathbf{y}_{ext}$ comes from a real LWR sample, the probability of **Abort** for each message is upper bounded by $tm2^{-\alpha} + \epsilon_{\text{LWR}}$, since otherwise detecting whether the event happens or not would already be an attack against LWR with advantage more than ϵ_{LWR} .

If the event **Abort** does not happen, it is then sufficient to know $\mathbf{y}_{ext}$ and all the $\mathbf{k}_i$ for $i \neq i^*$ to compute $\mathbf{y}_{i^*}$ without using $\mathbf{k}_{i^*}$.

4. Now that every other dependency to the unopened key share $\mathbf{k}_{i^*}$ has been removed, we can replace the pseudorandom output $G_{\mathcal{K}}(\mathbf{k}_{i^*})$ in the computation of **Share** with a uniform, unrelated random value. By reduction to the PRG $G_{\mathcal{K}}$, this incurs in the additional distinguishing advantage $N\epsilon_{\text{PRG}}$.
5. The only remaining dependency on the key $\mathbf{k}$ is the computation of $\mathbf{y}_{ext}$, which contains the additional α for each component. From now on, instead of also getting the key $\mathbf{k}$ as input, the experiment can just get the LWR sample $(\mathbf{X}, \mathbf{y}_{ext})$. For this, we define a (efficiently computable) function **Hyb** that takes as input the LWR sample and outputs the corresponding transcript.
6. Additionally, we consider a simulator Sim_1 that only takes as input an LWR sample $(\mathbf{X}, \mathbf{y})$, and randomly samples the additional missing α bits. It is easy to see that, when the sample comes from a random instance, the output of Sim_1 and **Hyb** is identically distributed:

$$\text{Hyb}(\text{LWR}'.\text{Rand}()) \equiv \text{Sim}_1(\text{LWR}.\text{Rand}()).$$

Both **Hyb** and Sim_1 can be seen as reductions to the respective LWR problems. Therefore, moving from $\text{LWR}'.\text{Real}$ to $\text{LWR}'.\text{Rand}$ first and from $\text{LWR}.\text{Rand}$ to $\text{LWR}.\text{Real}$ later incurs in an additional distinguishing advantage $2\epsilon_{\text{LWR}}$.

Notice that Steps 4 to 6 require that **Abort** does not happen. Let **Success** be the complementary event, and let $\epsilon_{\text{Success}}$ be the distinguishing advantage whenever **Success** happens. A generic reduction to the distinguishing problem between the initial hybrid experiment of Step 4 and the final simulator of Step 6 shows that conditioning on **Success** can increase the advantage by a factor of $2/p_{\text{Success}}$, where p_{Success} is the probability that **Success** happens. Indeed, by carefully crafting a reduction that cancels the advantage when **Abort** happens, and applying the total probability law, we get

$$\begin{aligned} \epsilon &= |p_{\text{Success}_0} \Pr[A(\mathcal{P}_0) = 1 | \text{Success}_0] - p_{\text{Success}_1} \Pr[A(\mathcal{P}_1) = 1 | \text{Success}_1]| \\ &\geq p_{\text{Success}} \epsilon_{\text{Success}} - \epsilon, \end{aligned}$$

where ϵ is the overall distinguishing advantage before conditioning on **Success**, Success_b for $b \in \{0, 1\}$ is the event that **Success** happens in experiment $\mathcal{P}_b$, $\epsilon_{\text{Success}} = |\Pr[A(\mathcal{P}_0) = 1 | \text{Success}_0] - \Pr[A(\mathcal{P}_1) = 1 | \text{Success}_1]|$, and the inequality comes from the application of the same reasoning as in Step 3: if the difference between p_{Success_0} and p_{Success_1} was more than ϵ , simply detecting the event happening would allow for a distinguishing advantage of more than ϵ , therefore we can upper bound this difference with ϵ . Since we are interested in $\epsilon_{\text{Success}}$ and, in our case, the overall advantage after considering the event

Abort is $\epsilon = Nt\epsilon_{\text{PRG}} + 2\epsilon_{\text{LWR}}$, we get that

$$\epsilon_{\text{Success}} = \frac{2Nt\epsilon_{\text{PRG}} + 4\epsilon_{\text{LWR}}}{p_{\text{Success}}},$$

and we are free to choose any event that is not more than ϵ far from any between **Success**₀ and **Success**₁; for simplicity, we choose the event caused by the LWR.Rand experiment, which, as argued in Step 3, happens with probability at most $1 - N \cdot t \cdot m \cdot 2^{-\alpha}$. Finally, in order to get rid of the event **Abort** in the simulator, we construct an additional simulator Sim_ω that runs Sim_1 until a non-aborting transcript is generated, or aborts if the number of attempts exceed ω . $\square$

Our scheme satisfies (2, 0)-special soundness, meaning that two accepting transcripts with the same first message and distinct challenges always enable the witness' reconstruction. Intuitively, this happens because two proofs with different challenges provide a complete sharing, and their validity with respect to the same first message guarantees that the reconstructed key is a valid witness. We state this observation in the next theorem and give its proof in [Section E.3](#).

Theorem 6 (Special soundness). *Let Σ be the sigma protocol described in [Fig. 3](#), and consider the relation $\mathcal{R} = \{((X, y), k) : |y - \lfloor Xk \rfloor_{2^{p-\alpha}}| \leq 1\}$. Then Σ is a sigma protocol with $(s, \epsilon_{\Sigma, \text{SS}})$ -special soundness (cf. [Definition 2](#)) for $\mathcal{R}$, where $s = 2$ and $\epsilon_{\Sigma, \text{SS}} = 0$.*

Proof (sketch). We observe that having $s = 2$ accepting proofs for the same commitment and different challenges provides the extractor with a complete sharing of a candidate witness k . Therefore, extraction can be finalized by reconstructing k from its sharing. The validity of each transcript with respect to the same statement and first message, together with the almost key homomorphism of the rounding function, imply that this extraction strategy always provides a valid witness for the given statement. $\square$

A fundamental property of our sigma protocol is having ϵ_{QCUR} -quantum computationally unique responses, ensuring hardness in finding two accepting transcripts with the same first message, same challenge, but distinct responses. Looking ahead, this relates to a powerful attack to the unforgeability of the signature scheme: the attacker could observe an honestly generated signature (which is a proof), and forge a valid one by just changing the response.

This attack is hard if the responses and the rest of the transcripts are related in a binding way. This is the reason why many MPCith-based schemes, including ours, hash the response with a collapsing hash function, and include the result in the first message. We state the corresponding theorem in the following with a sketch of proof and defer the complete proof to [Section E.4](#).

Theorem 7 (Quantum Computationally Unique Responses). *Let Σ be the Sigma protocol defined in [Fig. 3](#). Assume that the hash functions used are ϵ_{Coll} -collapsing (cf. [Definition 13](#)). Then Σ has ϵ_{QCUR} -quantum computationally unique responses, with*

$$\epsilon_{\text{QCUR}} \leq t(d-1) \cdot \epsilon_{\text{Coll}}.$$

Proof (sketch). This proof is very similar to the one provided for Picnic1 in Corollary 26 of [47]. To prove quantum computationally unique responses, we must argue about the event where an adversary provides two different responses for the same first message and challenge, and about the detection of a measurement in the computational basis. Notably, the response of our sigma protocol only contains preimages of the hash function. Therefore, the QCUR property reduces to the collapsingness of H on each of the $(d-1)t$ opened values. Using the bound on the concurrent composition of collapsing functions (Lemma 4, [51]), the theorem follows. $\square$

Finally, we prove that our scheme has unpredictable commitments. Looking ahead to the signature scheme, this property ensures that the input to the Random Oracle (RO) is sufficiently random, thereby guaranteeing that the RO-generated challenge is also random. This allows to relate the correctness of the signature to the completeness of the sigma protocol and to prevent the forger from crafting “comfortable” challenges. Similarly to other MPCith-based scheme, ours has highly unpredictable commitments. This is because each iteration involves the sampling of a new sharing, and each first message contains the evaluation of such shares with a collapsing hash function. We make this intuition formal in the following theorem, and provide the full proof in Section E.5.

Theorem 8 (Unpredictable Commitments). *Let Σ be the Sigma protocol of Fig. 3. Assume that the hash function used to instantiate it is ϵ_{Coll} -collapsing (cf. Definition 13). Then Σ has $\epsilon_{\text{ComGuess}}$ -unpredictable commitments, with*

$$\epsilon_{\text{ComGuess}} \leq \left(2\epsilon_{\text{Coll}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|} \right) + \frac{1}{|\mathcal{P}_{\mathcal{K}}|} \right)^{(d-1)t}.$$

In other words, for every couple of statement and witness generated with $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$, the min entropy ν of the first message is at least

$$- \log \left[\left(2\epsilon_{\text{Coll}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|} \right) + \frac{1}{|\mathcal{P}_{\mathcal{K}}|} \right)^{(d-1)t} \right].$$

Proof (sketch). To prove commitment unpredictability, we upper bound the probability that an honest prover outputs a fixed first message firstmes^* . This happens only if all hash values match those in firstmes^* . In turn, this requires that either all the PRG seeds equal those underlying firstmes^* , or that some of the PRG seeds differ but still yield identical hash outputs. Producing the same hash from two different seeds corresponds to finding a collision, which happens with probability at most $2\epsilon_{\text{Coll}}$ for ϵ_{Coll} -collapsing hash functions (cf. Lemma 24 of [82]). Combining the independent contributions of all seeds and applying the binomial theorem, we obtain the theorem bound. $\square$

6 Signatures from Sigma Protocols and HIGs

In this section, we provide the formal security analysis of our signature scheme. Similarly to other well-known signatures, our scheme is constructed from a sigma

protocol and a hard instance via the Fiat-Shamir transform with aborts (see [Section 3.6](#)). Specifically, we build on the sigma protocol defined in [Section 5](#) and the hard-instance generator of [Section 4.1](#). This construction is proved correct and post-quantum secure under various combinations of assumptions on the underlying sigma protocol. We rely on the results of Devevey *et al.* [43], Fehr *et al.* [47], and Barbosa *et al.* [14] (reported in [Section 3.6](#)). Plugging our parameters from [Section 4.1](#) and [5](#) into their theorems, we get the following bounds.

Theorem 9 (Correctness of Bittersweet Signatures). *Let Σ be the sigma protocol defined in [Section 5](#), and $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be the hard-instance generator of [Section 4.1](#). Let Π_{Sign} be the signature scheme constructed from $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and Σ using the Fiat-Shamir transform with Abort (cf. [Fig. 2](#)). Then Π_{Sign} is ϵ_{Corr} -correct, where ϵ_{Corr} is at most*

$$\frac{\left(2\epsilon_{\text{Coll}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right) + \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^{(d-1)t}}{(1 - tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}})^3}.$$

Theorem 10 (Unforgeability of Bittersweet Signatures). *Let Σ be the sigma protocol defined in [Section 5](#), and $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be the hard-instance generator of [Section 4.1](#). Let Π_{Sign} be the signature scheme constructed from $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and Σ using the Fiat-Shamir transform with Abort (cf. [Fig. 2](#)). Then Π_{Sign} is $(\epsilon_{\text{Sign}}, N_{\text{Sign}}, N_H, N_{\text{Sign}})$ -unforgeable, with*

$$\begin{aligned} \epsilon_{\text{Sign}, N_{\text{Sign}}} \leq & \left(\left(\epsilon_{\text{LWR}}^{(p-\alpha)} \left(1 + 3^m \frac{2^{qn}}{2^{(p-\alpha)m}} \right) + 3^m \frac{2^{qn}}{2^{2qnN}} \right) \cdot N_H^6 \right. \\ & + \frac{1}{d^t} + 2t(d-1) \cdot \epsilon_{\text{Coll}} + \frac{1}{O(N_H^5) \cdot C} + \frac{1}{8N_H^3 C^3} \Bigg)^{\frac{1}{3}} \\ & + \frac{2N_{\text{Sign}} \sqrt{\left(2\epsilon_{\text{Coll}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right) + \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^{(d-1)t}}}{1 - (tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}})} \\ & \cdot \sqrt{N_H + 1 + \frac{N_{\text{Sign}}}{1 - (tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}})}} \\ & + 2(N_H + 1) \sqrt{\frac{N_{\text{Sign}} \left(2\epsilon_{\text{Coll}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right) + \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^{(d-1)t}}{1 - (tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}})}} \\ & + N_{\text{Sign}} t \epsilon_{\text{Hide}} + \frac{2N_{\text{Sign}} t \epsilon_{\text{PRG}} + 4\epsilon_{\text{LWR}}}{1 - N_{\text{Sign}} \cdot t \cdot m \cdot 2^{-\alpha}} + 2\gamma^\omega, \end{aligned}$$

where $\gamma \leq N \cdot t \cdot m \cdot 2^{-\alpha}$, and ω is any value in $\mathbb{N}$ related to the (polynomial) runtime of the simulator.

The proofs of [Theorem 9](#) and [Theorem 10](#) are in [Section C.7](#) and [Section C.8](#).

7 Instantiation and Performance

We complete the paper by examining the instantiation of Bittersweet signatures. We start with a more operational description, follow with a discussion of parameters' choices and conclude with performance evaluation.

7.1 Operational Specifications

Let us first define a function that outputs m LWR samples:

$$\text{LWR}_{\mathbf{k}}(x) : \lambda \times \mathbb{Z}_{2^q}^n \rightarrow \mathbb{Z}_{2^{p-\alpha}}^n : x \mapsto \mathbf{y} = \lfloor \mathbf{G}(x) \cdot \mathbf{k} \rfloor_{2^{p-\alpha}},$$

where $\mathbf{G}(x)$ is a function that expands the λ -bit input x into a $m \times n$ matrix $\mathbf{X} \in \mathbb{Z}_{2^q}^{m \times n}$, typically instantiable with an eXtendable Output Function (XOF) [75].

The Bittersweet signatures' blueprint is given in Figure 5, which illustrates the parallel nature of their design. It is based on the following steps, which essentially match the descriptions in the previous sections, with a few additional details (separation bits, salt, ...) mimicking what can be found in Picnic [36, 35].

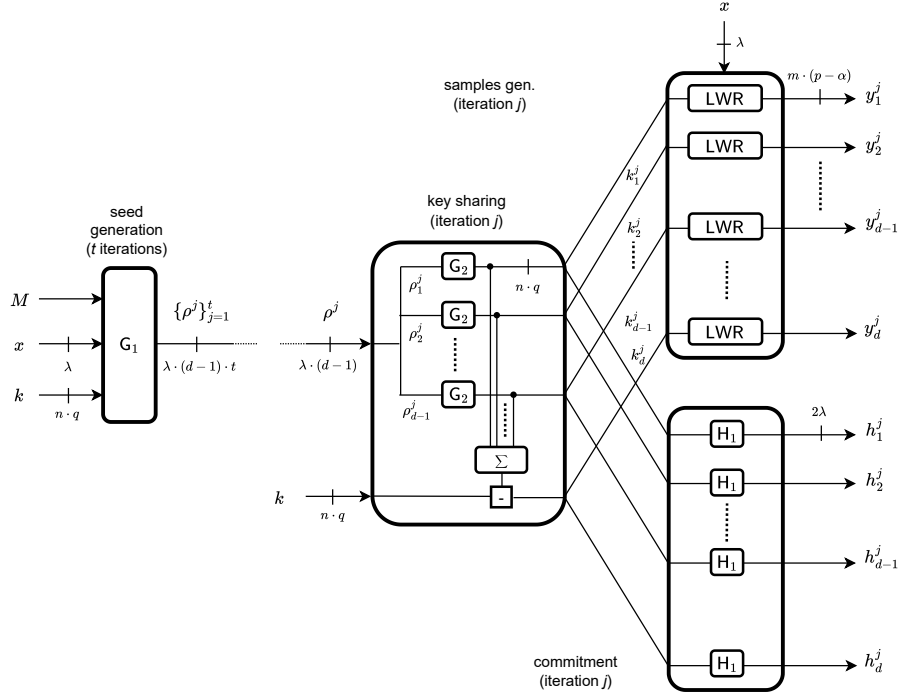


Fig. 5. Bittersweet signatures' blueprint.

Key generation. Pick up a plaintext $x \xleftarrow{\$} \{0,1\}^\lambda$ and a secret key $\mathbf{k} \xleftarrow{\$} \mathbb{Z}_{2^q}^n$ uniformly at random. Compute the public key $pk := (x, \mathbf{y})$ with $\mathbf{y} = \text{LWR}_{\mathbf{k}}(x)$.

Seeds generation. Compute seeds $\{\rho^j\}_{j=1}^t = G_1(M, x, \mathbf{k}, salt)$, where M is the message to sign, x the plaintext part of the public key, $\mathbf{k}$ the secret (signature) key, the λ -bit $salt$ is used to prevent multi-target attacks [45] and t is the number of iterations of the signature process. The resulting seeds have bitsize $\lambda \cdot (d-1) \cdot t$, with d being the number of parties. For each iteration j , the seed ρ^j is split into $d-1$ equal parts denoted as ρ_i^j , $1 \leq i \leq d-1$, each of them having bitsize λ .

Key sharing. For each iteration j , $1 \leq j \leq t$ and for each party i , $1 \leq i \leq d-1$, compute the expanded key shares $\mathbf{k}_i^j = G_2(\rho_i^j, i, j, salt)$ and $\mathbf{k}_d^j = \mathbf{k} - \sum_{i=1}^{d-1} \mathbf{k}_i^j \mod 2^q$, so that every key share $\mathbf{k}_i^j$, $1 \leq i \leq d$, $1 \leq j \leq t$, has bitsize $n \cdot q$.

Commitment. For each iteration j , $1 \leq j \leq t$, commit on the key share by computing $h_i^j = H_1(\mathbf{k}_i^j, i, j, salt)$, $1 \leq i \leq d$, so that each h_i^j has bitsize 2λ .

LWR samples generation. For each iteration j , $1 \leq j \leq t$, compute the output shares $\mathbf{y}_i^j = \text{LWR}_{\mathbf{k}_i}(x)$, $1 \leq i \leq d$, so that each $\mathbf{y}_i^j$ has bitsize $m \cdot (p - \alpha)$.

Challenge. Compute the hash of the message to sign M , the commitments $h_{1:d}^{1:t}$, the output shares $\mathbf{y}_{1:d}^{1:t}$, the signer's public key $(x, \mathbf{y})$ and the salt as:

$$c = H_2(\mathbf{y}_{1:d}^{1:t}, h_{1:d}^{1:t}, M, x, \mathbf{y}, salt).$$

Signature assembly. Assume $d = 2^\delta$ and $|h| = \delta t$ for simplicity. For each iteration j , denote the δ -bit index of the key share that is not opened as $nop(j)$, and the indexes of the $d-1$ key shares that are opened as $op(j)$. The signature σ is obtained by concatenating the $salt$ with all the output shares $\mathbf{y}_{1:d}^{1:t}$ and commitments $h_{1:d}^{1:t}$ as well as the opened key shares $\{\mathbf{k}_{op(j)}^j\}_{j=1}^t$.

$$\sigma = (salt \parallel \mathbf{y}_{1:d}^{1:t} \parallel h_{1:d}^{1:t} \parallel \{\mathbf{k}_{op(j)}^j\}_{j=1}^t)$$

Aborts. For each iteration j , $1 \leq j \leq t$, compute the first addend of the right-hand-side of Eq. (4), this time rounded to p (rather than $p - \alpha$) bits, i.e., $\left\lfloor \sum_{op(j)} G(x) \cdot \mathbf{k}_{op(j)}^j \mod 2^q \right\rfloor_{2^p}$. Compare it with $\mathbf{y}_{ext}$, which is the extended representation of $\mathbf{y}$ where $G(x) \cdot \mathbf{k}$ is rounded to p (rather than $p - \alpha$) bits. Abort if the α LSBs of these values match in any of the m vector components.

Verification. Derive the challenge c from the signature. For each iteration j , $1 \leq j \leq t$, recompute the commitments and output shares of the opened parties using $\{\mathbf{k}_{op(j)}^j\}_{j=1}^t$. Then check that they correspond to the ones present in the signature using the indexing associated to the challenge. If this holds, compute the noisy outputs for each iteration:

$$\tilde{\mathbf{y}}^j = \left\lfloor \sum_{op(j)} G(x) \cdot \mathbf{k}_{op(j)}^j \mod 2^q \right\rfloor_{2^{p-\alpha}} + \mathbf{y}_{nop(j)}^j \mod 2^{p-\alpha}. \quad (4)$$

If the inequality $\mathbf{y}^j - \tilde{\mathbf{y}}^j \leq 1$ holds component-wise, accept the signature.

Optimizations. We use the next optimizations To reduce the signature size. First, the output share of the opened parties $\{\mathbf{y}_{op(j)}^j\}_{j=1}^t$ can be removed from the

signature since they can be recomputed using the opened key shares $\{\mathbf{k}_{op(j)}^j\}_{j=1}^t$ and the public key x . Second, the commitments of the opened parties $\{h_{op(j)}^j\}_{j=1}^t$ can also be removed from the signature as they can be recomputed from the opened key shares $\{\mathbf{k}_{op(j)}^j\}_{j=1}^t$. To guarantee that these opened key shares were correctly committed, we include the challenge c in the signature. This challenge acts as a “commitment to the commitments” [36], since the verifier checks that the recomputed commitments are consistent by verifying that the recalculated challenge matches the one present in the signature. Third, the size of the first $d - 1$ key shares can be reduced by transmitting the corresponding seeds. Using a GGM-tree, only the root seed of the tree must be sent if the last party remains unopened. Otherwise, the last key share is sent with the sibling-path of the GGM tree associated to the unopened party, which consists of $\log_2(d)$ seeds.

By contrast, while most MPCitH schemes can get rid of the unopened output share by deriving it from the public statement and the opened output shares, this cannot be applied to Bittersweet due to presence of a small reconstruction error. Yet, since the unopened key shares $\mathbf{y}_{nop(j)}^j$ can be computed from the error vector $\mathbf{e}^j = \mathbf{y}^j - \tilde{\mathbf{y}}^j$, we can replace each $\mathbf{y}_{nop(j)}^j$ by $\mathbf{e}^j$ to reduce its size.

Signature size. Using the different aforementioned optimizations, the averaged size (in bits) of Bittersweet signatures is:

$$\begin{aligned}
|\sigma| &= t \cdot \left(\frac{d-1}{d} \cdot (n \cdot q + \lceil \log_2(d) \rceil \cdot \lambda) + \frac{1}{d} \cdot \lambda \right) && \rightarrow \text{seeds and key shares} \\
&+ t \cdot m && \rightarrow \text{error vectors} \\
&+ t \cdot 2\lambda && \rightarrow \text{commitments} \\
&+ \lambda + t \cdot \lceil \log_2(d) \rceil && \rightarrow \text{salt and challenge}
\end{aligned}$$

7.2 Parameters’ Choices

The functions and parameters of Bittersweet signatures depend on the security target that can be bounded using the theorems in the previous sections. We next show how they can be selected concretely by identifying the dominating terms of our main bounds, making clear when heuristic choices are operated.

First, the hiding guarantees of the commitments and the quantum computationally unique responses requirement drive the choice of the hash function. We use SHA3-256 (*resp.*, SHA3-384 and SHA3-512) for NIST’s security level 1 (*resp.*, 3, 5). Similarly, the PRG security drives the selection of the XOF. We use SHAKE-128 (*resp.*, SHAKE-256) for NIST’s security level 1 (*resp.*, 3, 5).

The next step is to select the LWR instance. The most demanding requirement in this respect comes from the ZK analysis of Theorem 5, where security is required for an instance of LWR rounding to p bits, despite the specifications of the previous subsection all round to $p' = p - \alpha$ bits (which is sufficient to make it hard to recover $\mathbf{k}$ from $\mathbf{y}$). The reason of this requirement is intuitive. First rewrite Eq. (4) as $\tilde{\mathbf{y}}^j = \tilde{\mathbf{y}}_{\text{rec}}^j + \tilde{\mathbf{y}}_{nop(j)}^j \pmod{2^{p'}}$. Then, write

$$\begin{array}{l} \tilde{\mathbf{y}}_{\text{rec}}^j = \overbrace{\text{MSBs}(\tilde{\mathbf{y}}_{\text{rec}}^j)}^{p' \text{ bits}} \parallel \overbrace{\text{buffer}(\tilde{\mathbf{y}}_{\text{rec}}^j)}^{\alpha \text{ bits}} \parallel \overbrace{\text{LSBs}(\tilde{\mathbf{y}}_{\text{rec}}^j)}^{q-p \text{ bits}}, \\ \mathbf{y} = \text{MSBs}(\mathbf{y}) \parallel \text{buffer}(\mathbf{y}) \parallel \text{LSBs}(\mathbf{y}). \end{array}$$

It allows expressing the abort condition as checking that the α -bit buffer differs for all components of $\tilde{\mathbf{y}}_{\text{rec}}^j$ and $\mathbf{y}$. Such a condition is necessary to ensure that information on the (secret) LSBs of $\mathbf{y}$ cannot be forwarded to its (public) MSBs due to carry propagation. This is because each time an adversary observes a signature (that did not abort), it can reject one value of the buffer. After observing a number of signatures N_{Sign} which is a small multiple of $2^{\alpha - \log_2(t)}$, one can get the buffer in full, justifying the requirement of Theorem 5.

It is nevertheless also possible to limit the carry leakage due to the observation of N_{Sign} signatures by increasing the buffer size. Say for example that $q = 96$, $\alpha = 90$, $p' = 3$ and N_{Sign} is bounded by the application to 2^{32} (more conservative) or 2^{64} (more aggressive). Then, one can consider Bittersweet signatures relying on a LWR instance that is rounding to p' bits with a small leakage on the buffer. Namely, the statistical distance between the uniform buffer distribution and the leaky one is $\frac{N_{\text{Sign}}}{2^\alpha}$ for each component, and roughly m times more for the full vectors, leading to an abort probability of $P_{\text{abort}} \approx m \cdot t \cdot 2^{-\alpha}$. We posit that this leakage does not significantly affect the security of the corresponding LWR instance as long as N_{Sign} is (very) small in front of 2^α (e.g., 2^{32} or 2^{64} in front of 2^{90} in the conservative and aggressive parameters that we propose above).

Albrecht et al.’s estimator. We now detail how we used the estimator from Albrecht *et al.* [5] to select LWR parameters, as it provides security estimates against a range of known attacks on LWE instances. We first show how to convert the parameters of an LWR problem into an equivalent LWE instance. Then we apply the estimator to find parameter sets (q, p', n) for each security level.

Converting LWR to LWE. As shown in [4], an LWR instance of the form $\mathbf{y} = \left\lfloor \mathbf{X} \mathbf{k} \frac{P'}{Q} \right\rfloor$ can be transformed into an LWE instance $\mathbf{y} = \mathbf{X} \mathbf{k} + \mathbf{e}$, where the error vector is sampled from $\mathbf{e} \stackrel{\$}{\leftarrow} \mathcal{U}_{Q, P'}^m$, with $\mathcal{U}_{Q, P'}$ the discrete uniform distribution over the set $[-\frac{Q}{2 \cdot P'} + 1; \frac{Q}{2 \cdot P'}]$. Therefore, a distribution LWR(Q, P', n, m) with a secret $\mathbf{k} \in \mathbb{Z}_Q^n$ can be converted, without loss of security, into a LWE distribution LWE(Q, n, m) with same secret distribution and where $\mathbf{e} \stackrel{\$}{\leftarrow} \mathcal{U}_{Q, P'}^m$. The estimator from [5] evaluates the security of an LWE instance based on its parameters (Q, n, m) and the standard deviation of the secret and noise distribution (denoted by σ_k and σ_e). Table 1 summarizes the relationship between those parameters and the corresponding LWR instance with rounding from q to p' .

Table 1. LWE parameters derived from an LWR instance

Q	σ_k	σ_e
2^q	$\sqrt{\frac{(2^q)^2 - 1}{12}}$	$\sqrt{\frac{(2^{q-p'})^2 - 1}{12}}$

Proposed instances. To choose the parameters (q, p', n) , we first impose some restrictions on q . In order to support signing up to 2^{64} messages while limiting carry leakage, we require $q \gg 64$. Moreover, we restrict q to multiples of 16 to leverage efficient SIMD multiplication instructions on 16 and 32 bits words in software execution. We additionally cap q at 128 and therefore consider the candidate set $q \in \{80, 96, 112, 128\}$. For p' , we limit the choices to $p' \in \{2, 3, 4\}$ as it provides small abort probability and small signature sizes.

Using the LWE estimator [5], we determine the smallest n for each parameter pair (q, p') that achieves the three NIST security levels. The resulting security estimates and signature sizes are summarized in Section F, Table 9 where the parameter set $(q, p') = (96, 3)$ yields the smallest signature size for security level L1 and L3 and relatively small signature size for L5, making it a natural choice for our Bittersweet instances. From these parameters, we can compute the number of samples m needed to ensure a hard instance thanks to Lemma 1 or 3: $m = \left\lceil \frac{n \cdot q}{p' - \log_2(3)} \right\rceil$, meaning $m = 747$ (*resp.*, 1222, 1697) for security level L1 (*resp.*, L3, L5).¹² The number of iteration t is finally chosen such that the soundness error, achieved with the multiple iterations, gets smaller than the security level $t = \left\lceil \frac{\lambda}{\log_2(d)} \right\rceil$. These parameters are in Table 2 for $d = \{32, 256\}$.

Table 2. Bittersweet parameters set for NIST security levels L1, L3 and L5.

Name	NIST Security (bits) λ	Parameters						Sizes (Bytes)		
		LWR			MPCitH					
		q	p'	α	n	m	d	t	P_{abort}	$ sk $ $ pk $ $ \sigma $
Bittersweet L1	128	96	3	90	11	747	32	26	$2^{-75.7}$	132 297 8 645
							256	16	$2^{-76.4}$	132 297 6 183
Bittersweet L3	192	96	3	90	18	1222	32	39	$2^{-74.4}$	216 483 20 602
							256	24	$2^{-75.2}$	216 483 14 622
Bittersweet L5	256	96	3	90	25	1697	32	52	$2^{-73.6}$	300 669 37 648
							256	32	$2^{-74.3}$	300 669 26 627

7.3 Performances Evaluation

We now study the size and computation time of Bittersweet signatures in function of the number of parties d for the NIST's three security levels.

Performances. The signing time is dominated, in decreasing order, by:

¹² As in the case of Picnic 2, this choice of parameters ignores the N_H^6 factor and the cubic root loss found in Theorem 10, which would both result in overly large parameters and are conjectured to be due to a non-tight security proof.

- $d \cdot m \cdot t$ inner products between (nq) -bit vectors.
- $(d - 1) \cdot t$ extensions/XOFs from λ -bit seeds to (nq) -bit key shares.
- $d \cdot t$ commitments (i.e., hashes from $n \cdot q$ bits to 2λ bits).
- The generation of the (nqm) -bit public matrices (with a XOF).

In order to highlight the interest of Bittersweet signatures’ simple design for hardware implementations, we implemented the three versions on a Xilinx Artix-7 FPGA (XC7A200T-3) and synthesized them with Vivado v2022.1. The hardware comparison between Bittersweet and other signature schemes, for which FPGA implementations are available, is given in [Table 4](#) for performances and [Table 5](#) for resources’ utilization. For completeness, software performance comparisons between Bittersweet and other signature schemes are given in [Tables 6](#) to [8](#) for all security levels. Cycle counts for Bittersweet are obtained using an Intel Core i7-1355U CPU with 16 GB of RAM, running Ubuntu 22.04.5. The code is compiled using GCC 11.4.0 with optimization flags `-O3 -march=native -std=gnu11`, on a system supporting SSE2 and AVX2 intrinsics.

In general, increasingly d leads to a tradeoff between signature size and computation time. Increasing the security levels leads to increased signature size due to the independent increase of the vector size n (to have a secure LWR instance) and the number of iterations t (to decrease the soundness error). The signing and verification processes are dominated by the inner product computations with a large modulus $Q = 2^{96}$ and the rejection probability is always below 2^{-70} .

As for the comparison with natural competitors, our main conclusion is that while Bittersweet signatures lead to mild overheads in the software case, they gain interest in the hardware case. In particular, Bittersweet instances consistently lead to better signature sizes compared with MPCitH signatures like Picnic [63, 68] and SDitH [42], while maintaining comparable performance and hardware footprint. Hardware performances could even be further improved by increasing the level of parallelism, for example by instantiating additional multipliers. By contrast, Bittersweet admittedly lags behind Mirath, which is based on TCitH framework [55], even in the hardware implementation context [71].

Finally, Dilithium remains out of reach from the signature size and performances’ viewpoint (at the cost of larger private and public keys). Yet, on the one hand, Bittersweet signatures do not rely on structured lattices, which explains most of the performance gap. Existing examples of signature schemes relying on unstructured lattices are indeed known to be significantly more expensive as highlighted in [Table 3](#). So the fact that Bittersweet signatures are competitive without such structured assumptions is actually interesting from the “not putting all eggs in the same basket” viewpoint. On the other hand, their potential for leakage-resilient implementations, which are known to be expensive for Dilithium [11], should reduce this gap. Not only because the key-homomorphic nature of its computations should make masking significantly simpler and more efficient than for MPC-in-the-head signatures based on symmetric cryptographic primitives [78, 10]. Also because the large modulus $Q = 2^{96}$ should make the

recombination of the information leaked by different shares significantly more difficult, especially with inner product masking [50]. The combination of these features with the optimizations and generalizations mentioned in the introduction could make Bittersweet signatures a solution of choice for hardware and leakage-resilient implementations with improved (physical) security vs. cost tradeoff, which is therefore an interesting direction for further research.

Table 3. Comparison of key and signature sizes for unstructured lattice-based signature schemes targeting 128-bit security (NIST security Level 1).

Scheme	Assumption	Size (Bytes)		
		$ sk $	$ pk $	$ \sigma $
Bittersweet L1	LWR	132	297	6 183
TESLA-1 [7]	LWE	4 332 000	11 559 900	2 444
Bai and Galbraith [12]	LWE/SIS	1 272 917	2 119 519	1 553
Lyubashevsky [70]	LWE/SIS	561 917	92 682	1 813
Squirrels-I [49]	LWE/SIS	2 593	681 780	1 019

Acknowledgments. We are grateful to Martin Albrecht, Cecilia Boschini, Yu-Hsuan Huang, and Andreas Hülsing for our very insightful discussions. Carmit Hazay is supported by ISF grant No. 607/25. François-Xavier Standaert is a research director of the F.R.S.-FNRS. This work has been funded by the German Research Foundation (DFG) CRC 1119 CROSSING (project S7), by the German Federal Ministry of Education and Research and the Hessen State Ministry for Higher Education, Research and the Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE, and by the European Research Council (ERC) Consolidator Grant 101044770 (CRYPTOLAYER) and Advanced Grant 101096871 (BRIDGE). Views and opinions expressed in the paper are those of the authors and do not necessarily reflect those of the European Union nor the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

Table 4. Hardware performance comparison of Bittersweet signatures with other MPCitH-, TCitH- and lattice-based signature schemes evaluated on Xilinx Artix-7 (XC7A200T-3) platform. Schemes marked with [†] have been evaluated on a Xilinx Kintex-7 (XC7K480T-3). The latencies are expressed in Mcycles and the times in ms.

Framework	Assumption	Scheme	d	KeyGen		Sign		Verify		Size (Bytes)			
				Latency	Time	Latency	Time	Latency	Time	$ sk $	$ pk $	$ \sigma $	
MPCitH & Fiat-Shamir with aborts	LWR	Bittersweet	L1	32	0.015	0.12	1.06	8.48	0.91	7.28	132	297	8 645
			L3		0.049	0.39	2.43	19.44	2.01	16.08	216	483	20 602
			L5		0.095	0.76	4.54	36.32	3.78	30.24	300	669	37 648
			L1	256	0.015	0.12	5.19	41.25	4.39	35.12	132	297	6 183
			L3		0.049	0.39	11.79	94.32	10.03	80.24	216	483	14 622
			L5		0.095	0.76	22.02	176.16	18.50	148.00	300	669	26 627
MPCitH & Fiat-Shamir	LowMC	Picnic1 [63]	L1-FS	3	-	-	0.03	0.25	0.03	0.24	32	16	32 862
			L5-FS		-	-	0.15	1.24	0.15	1.17	64	32	128 201
		Picnic3 [†] [68]	L1	4	-	-	0.02	0.08	-	-	17	34	19 573
			L5		-	-	0.03	0.18	-	-	32	64	75 721
	Syndrome Decoding	SDitH [42] hypercube	L1-GF256	256	0.055	0.34	6.73	41.04	8.69	52.98	404	120	8 260
			L1-GF251		0.057	0.35	21.14	130.77	23.40	142.71			
			L3-GF256	256	0.046	0.28	7.74	47.17	10.96	66.85	616	183	19 206
			L3-GF251		0.047	0.29	24.6	149.99	24.60	149.99			
			L5-GF256	256	0.083	0.51	17.48	106.60	24.94	152.10	812	234	33 448
			L5-GF251		0.086	0.53	45.28	276.07	30.11	183.57			
TCitH & Fiat-Shamir	MinRank	Mirath [71]	1b-fast	256	0.016	0.063	0.55	2.18	0.50	1.97	32	57	3 456
			5b-fast		0.061	0.278	1.37	6.29	1.21	6.56	64	112	14 262
Fiat-Shamir with aborts	MLWE MSIS	Dilithium [83]	L2		0.004	0.04	0.03	0.29	0.004	0.05	2 528	1 312	2 420
			L3		0.006	0.06	0.04	0.46	0.006	0.06	4 000	1 952	3 293
			L5		0.009	0.09	0.05	0.51	0.009	0.09	4 864	2 592	4 595

Bittersweet Signatures and Hardware-Friendly MPC-in-the-Head

35

Table 6. Software performance comparison of Bittersweet signatures with other MPCitH-, TCitH-, VOLEitH- and Lattice-based signature schemes for L1 security level. The reported runtimes are from the AVX optimized implementation. Values marked with * have been converted from milliseconds to cycles, as the cycle count was not provided.

Framework	Assumption	Scheme		d	Runtime (Mcycles)			Size (Bytes)		
		Name	Parameter set		Keygen	Sign	Verify	$ sk $	$ pk $	$ \sigma $
MPCitH & Fiat-Shamir with aborts	LWR	Bittersweet	L1	32	0.54	13.83	13.04	132	297	8 645
				256	0.54	63.53	65.52	132	297	6 183
				2048	0.54	377.94	373.83	132	297	5 232
MPCitH & Fiat-Shamir	LowMC	Picnic1 [35]	L1-FS	3	0.01	5.19	4.15	32	16	32 862
		Picnic3 [64]	L1	16	-	*18.61	*14.26	32	16	12 595
	Permuted Kernel	PERK [1]	I-fast-5	32	0.09	7.00	4.90	16	240	8 030
			I-short-5	256	0.09	35.00	25.00	16	240	5 780
TCitH & Fiat-Shamir	MinRank	Mirath [2]	1b-fast	256	0.60	15.10	12.20	32	57	3 456
			1b-short	4096	0.60	152.00	101.00	32	57	2 902
	Multivariate Quadratic	MQOM2 [20]	L1-gf2-fast-3r	256	1.90	14.12	11.69	52	72	3 212
			L1-gf2-short-3r	2048	1.74	26.52	25.11	52	72	2 868
	Rank Syndrome Decoding	RYDE [9]	1-Fast (gf2)	256	0.04	11.90	11.50	32	69	3 597
			1-Short (gf2)	4096	0.04	123.70	117.00	32	69	2 988
VOLEitH & Fiat-Shamir	AES	FAEST2 [15]	128-f		0.01	1.72	1.41	32	32	5 924
			128-s		0.01	12.79	9.78	32	32	4 506
	Syndrome Decoding	SDitH2 [73]	L1-gf2-fast		*1.45	*7.25	*6.54	163	70	4 484
			L1-gf2-short		*1.49	*22.86	*20.04	163	70	3 705
Fiat-Shamir with aborts	MLWE MSIS	Dilithium [79]	L2		0.12	0.33	0.12	2 528	1 312	2 420
Hash & Sign	NTRU	Falcon [57]	512		19.87	-	-	1 281	897	666
	MLIP	HAWK2 [32]	512		8.43	0.08	0.15	184	1024	555

Table 7. Software performance comparison of Bittersweet signatures with other MPCitH-, TCitH-, VOLEitH- and Lattice-based signature schemes for L3 security level. The reported runtimes are from the AVX optimized implementation. Values marked with * have been converted from milliseconds to cycles, as the cycle count was not provided.

Framework	Assumption	Scheme		d	Runtime (Mcycles)			Size (Bytes)		
		Name	Parameter set		Keygen	Sign	Verify	$ sk $	$ pk $	$ \sigma $
MPCitH & Fiat-Shamir with aborts	LWR	Bittersweet	L3	32	1.74	50.11	47.26	216	483	20 602
				256	1.74	237.78	234.76	216	483	14 622
				2048	1.74	1 464.41	1 456.87	216	483	12 299
MPCitH & Fiat-Shamir	LowMC	Picnic1 [35]	L3-FS	3	0.01	12.14	10.05	48	24	74 134
		Picnic3 [64]	L3	16	-	*37.56	*29.20	48	24	27 551
	Permuted Kernel	PERK [1]	III-fast-5	32	0.19	15.00	11.00	24	370	18 000
			III-short-5	256	0.20	75.00	59.00	24	370	13 200
	MinRank	Mirath [2]	3b-fast	256	1.20	55.00	51.60	48	84	7 936
			3b-short	4096	1.20	520.00	327.00	48	84	6 514
TCitH & Fiat-Shamir	Multivariate Quadratic	MQOM2 [20]	L3-gf2-fast-3r	256	7.85	57.37	51.19	78	108	7 576
			L3-gf2-short-3r	2048	8.23	114.90	103.60	78	108	6 388
	Rank Syndrome Decoding	RYDE [9]	3-Fast	256	0.06	32.10	31.40	48	101	8 264
			3-Short	4096	0.06	368.00	331.10	48	101	6 728
VOLEitH & Fiat-Shamir	AES	FAEST2 [15]	192f		0.01	7.04	6.08	40	48	14 948
			192s		0.01	54.69	42.29	40	48	11 260
	Syndrome Decoding	SDitH2 [73]	L3-gf2-fast		*3.94	*19.26	*17.42	232	98	9 916
			L3-gf2-short		*4.19	*109.12	*101.38	232	98	7 964
Fiat-Shamir with aborts	MLWE MSIS	Dilithium [79]	L3		0.26	0.53	0.18	4 000	1 952	3 293

Table 8. Software performance comparison of Bittersweet signatures with other MPCitH-, TCitH-, VOLEitH- and Lattice-based signature schemes for L5 security level. The reported runtimes are from the AVX optimized implementation. Values marked with * have been converted from milliseconds to cycles, as the cycle count was not provided.

Framework	Assumption	Scheme		d	Runtime (Mcycles)			Size (Bytes)		
		Name	Parameter set		Keygen	Sign	Verify	$ sk $	$ pk $	$ \sigma $
MPCitH & Fiat-Shamir with aborts	LWR	Bittersweet	L5	32	3.41	121.19	114.34	300	669	37 648
				256	3.41	570.06	565.34	300	669	26 627
				2048	3.41	3 549.47	3 521.74	300	669	22 333
MPCitH & Fiat-Shamir	LowMC	Picnic1 [35]	L5-FS	3	0.02	21.13	17.71	64	32	128 176
		Picnic3 [64]	L5	16	-	*65.34	*46.87	64	32	48 716
	Permuted Kernel	PERK [1]	V-fast-5	32	0.32	34.00	26.00	32	510	31 700
			V-short-5	256	0.33	168.00	131.00	32	510	23 000
TCitH & Fiat-Shamir	MinRank	Mirath [2]	5b-fast	256	2.00	121.00	88.00	64	112	14 262
			5b-short	4096	2.00	1 421.00	630.00	64	112	11 620
	Multivariate Quadratic	MQOM2 [20]	L5-gf2-fast-3r	256	13.71	145.51	129.97	104	144	13 412
			L5-gf2-short-3r	2048	13.81	278.92	266.51	104	144	11 764
	Rank Syndrome Decoding	RYDE [9]	5-Fast	256	0.08	65.90	61.60	64	133	14 609
			5-Short	4096	0.08	799.30	623.00	64	133	11 819
VOLEitH & Fiat-Shamir	AES	FAEST2 [15]	256f		0.01	11.07	10.24	48	48	26 548
			256s		0.01	76.33	74.55	48	48	20 696
	Syndrome Decoding	SDitH2 [73]	L5-gf2-fast		*4.70	*27.37	*25.06	307	132	17 540
			L5-gf2-short		*4.75	*152.34	*141.02	307	132	14 121
Fiat-Shamir with aborts	MLWE MSIS	Dilithium [79]	L5		0.30	0.64	0.28	4 864	2 592	4 595
Hash & Sign	NTRU	Falcon [57]	512		63.13	-	-	2 305	1 793	1 280
	MLIP	HAWK2 [32]	512		43.66	0.18	0.30	360	2 440	1 221

References

1. Najwa Aaraj, Slim Bettaiieb, Loïc Bidoux, Alessandro Budronand, Victor Dyseryn, Andre Esser, Thibault Feneuil, Philippe Gaborit, Mukul Kulkarni, Victor Mateu, Marco Palumbi, Lucas Perin, Matthieu Rivain, Jean-Pierre Tillich, and Keita Xawaga. PERK. Round 2 : Submission to the NIST Post-Quantum Cryptography : Additional Digital Signature Schemes, 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/perk-spec-round2-web.pdf>.
2. Gora Adj, Nicolas Aragon, Stefano Barbero, Magali Bardet, Emanuele Bellini, Loïc Bidoux, Jesús-Javier Chi-Domínguez, Victor Dyseryn, Andre Esser, Thibault Feneuil, Philippe Gaborit, Romaric Neveu, Matthieu Rivain, Luis Rivera-Zamarripa, Carlo Sanna, Jean-Pierre Tillich, Javier Verbel, and Floyd Zveydinger. Mirath Signature Scheme, Algorithm Specifications and Supporting Documentation. Round 2 : Submission to the NIST Post-Quantum Cryptography : Additional Digital Signature Schemes, 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/mirath-spec-round2-web.pdf>.
3. Gora Adj, Stefano Barbero, Emanuele Bellini, Andre Esser, Luis Rivera-Zamarripa, Carlo Sanna, Javier A. Verbel, and Floyd Zveydinger. MiRitH: Efficient Post-Quantum Signatures from MinRank in the Head. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(2):304–328, 2024.
4. Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the {LWE, NTRU} schemes! In Dario Catalano and Roberto De Prisco, editors, *Security and Cryptography for Networks - 11th International Conference, SCN 2018, Amalfi, Italy, September 5-7, 2018, Proceedings*, volume 11035 of *Lecture Notes in Computer Science*, pages 351–367. Springer, 2018.
5. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015.
6. Martin R. Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for MPC and FHE. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 430–454. Springer, Heidelberg, April 2015.
7. Erdem Alkim, Nina Bindel, Johannes Buchmann, Özgür Dagdelen, Edward Eaton, Gus Gutoski, Juliane Krämer, and Filip Pawlega. Revisiting TESLA in the quantum random oracle model. In *PQCrypto*, volume 10346 of *Lecture Notes in Computer Science*, pages 143–162. Springer, 2017.
8. Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligerio: Lightweight sublinear arguments without a trusted setup. In *CCS*, pages 2087–2104. ACM, 2017.
9. Nicolas Aragon, Magali Bardet, Loïc Bidoux, Jesús-Javier Chi-Domínguez, Victor Dyseryn, Thibault Feneuil, Philippe Gaborit, Antoine Joux, Romaric Neveu, Matthieu Rivain, Jean-Pierre Tillich, and Adrien Vinçotte. RYDE Signature Scheme, Algorithm Specifications and Supporting Documentation. Round 2 : Submission to the NIST Post-Quantum Cryptography : Additional Digital Signature Schemes, 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/ryde-spec-round2-web.pdf>.
10. Diego F. Aranha, Sebastian Berndt, Thomas Eisenbarth, Okan Seker, Akira Takahashi, Luca Wilke, and Greg Zaverucha. Side-channel protections for picnic signatures. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(4):239–282, 2021.

11. Melissa Azouaoui, Olivier Bronchain, Gaëtan Cassiers, Clément Hoffmann, Yulia Kuzovkova, Joost Renes, Tobias Schneider, Markus Schönauer, François-Xavier Standaert, and Christine van Vredendaal. Protecting Dilithium against Leakage Revisited Sensitivity Analysis and Improved Implementations. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(4):58–79, 2023.
12. Shi Bai and Steven D. Galbraith. An improved compression technique for signatures based on learning with errors. In *CT-RSA*, volume 8366 of *Lecture Notes in Computer Science*, pages 28–47. Springer, 2014.
13. Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In *EUROCRYPT*, pages 719–737. Springer, 2012.
14. Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of fiat-shamir with aborts and dilithium. In *CRYPTO*, pages 358–389. Springer, 2023.
15. Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. FAEST v2: Algorithm Specifications, Version 2.0. Round 2 : Submission to the NIST Post-Quantum Cryptography : Additional Digital Signature Schemes, 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/faest-spec-round2-web.pdf>.
16. Carsten Baum, Ward Beullens, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. One tree to rule them all: Optimizing GGM trees and owfs for post-quantum signatures. In *ASIACRYPT*, pages 463–493. Springer, 2024.
17. Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from vole-in-the-head. In *CRYPTO*, pages 581–615. Springer, 2023.
18. Carsten Baum, Cyprien Delpech de Saint Guilhem, Daniel Kales, Emmanuela Orsini, Peter Scholl, and Greg Zaverucha. Banquet: Short and Fast Signatures from AES. In *PKC*, pages 266–297. Springer, 2021.
19. Carsten Baum and Ariel Nof. Concretely-efficient zero-knowledge arguments for arithmetic circuits and their application to lattice-based cryptography. In *PKC*, pages 495–526. Springer, 2020.
20. Ryad Benadjila, Charles Bouillaguet, Thibauld Feneuil, and Matthieu Rivain. MQOM: MQ on my Mind, Algorithm Specifications and Supporting Documentation (Version 2.0). Round 2 : Submission to the NIST Post-Quantum Cryptography : Additional Digital Signature Schemes, 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/mqom-spec-round2-web.pdf>.
21. Ryad Benadjila, Thibauld Feneuil, and Matthieu Rivain. MQ on my mind: Post-quantum signatures from the non-structured multivariate quadratic problem. In *EuroS&P*, pages 468–485. IEEE, 2024.
22. Rikke Bendlin and Ivan Damgård. Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems. In *TCC*, pages 201–218. Springer, 2010.
23. Itay Berman, Akshay Degwekar, Ron D. Rothblum, and Prashant Nalini Vasudevan. From laconic zero-knowledge to public-key cryptography - extended abstract. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 674–697. Springer, Heidelberg, August 2018.

24. Slim Bettaieb, Loïc Bidoux, Victor Deryn, Andre Esser, Philippe Gaborit, Mukul Kulkarni, and Marco Palumbi. PERK: compact signature scheme based on a new variant of the permuted kernel problem. *Des. Codes Cryptogr.*, 92(8):2131–2157, 2024.
25. Ward Beullens. Sigma protocols for mq, PKP and sis, and fishy signature schemes. In *EUROCRYPT*, pages 183–211. Springer, 2020.
26. Ward Beullens and Cyprien Delpech de Saint Guilhem. LegRoast: Efficient Post-quantum Signatures from the Legendre PRF. In *PQCrypto*, pages 130–150. Springer, 2020.
27. Rishabh Bhadauria, Zhiyong Fang, Carmit Hazay, Muthuramakrishnan Venkatasubramanian, Tiancheng Xie, and Yupeng Zhang. Liger++: A new optimized sublinear IOP. In *CCS*, pages 2025–2038. ACM, 2020.
28. Loïc Bidoux, Philippe Gaborit, Mukul Kulkarni, and Víctor Mateu. Code-based Signatures from New Proofs of Knowledge for the Syndrome Decoding Problem. *Des. Codes Cryptogr.*, 91(2):497–544, 2023.
29. Gerrit Bleumer. Threshold Signatures. In *Encyclopedia of Cryptography and Security (2nd Ed.)*, pages 1294–1296. Springer, 2011.
30. Dan Boneh, Yuval Ishai, Alain Passelègue, Amit Sahai, and David J. Wu. Exploring crypto dark matter: - new simple PRF candidates and their applications. In *TCC*, pages 699–729. Springer, 2018.
31. Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic prfs and their applications. In *CRYPTO*, pages 410–428. Springer, 2013.
32. Joppe W. Bos, Olivier Bronchain, Léo Ducas, Serge Fehr, Yu-Hsuan Huang, Thomas Pornin, Eamonn W. Postlethwaite, Thomas Prest, Ludo N. Pulles, and Wessel van Woerden. Hawk, version 1.1. Round 2 : Submission to the NIST Post-Quantum Cryptography : Additional Digital Signature Schemes, 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/hawk-spec-round2-web.pdf>.
33. Dung Bui, Eliana Carozza, Geoffroy Couteau, Dahmun Goudarzi, and Antoine Joux. Faster signatures from mpc-in-the-head. In *ASIACRYPT*, pages 396–428. Springer, 2024.
34. Eliana Carozza, Geoffroy Couteau, and Antoine Joux. Short signatures from regular syndrome decoding in the head. In *EUROCRYPT*, pages 532–563. Springer, 2023.
35. Melissa Chase, David Derler, Steven Goldfeder, Jonathan Katz, Vladimir Kolesnikov, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, Xiao Wang, et al. The Picnic Signature Scheme. *The Picnic Signature Scheme - Design Document. Version 2.2, 14 April 2020*.
36. Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-Quantum Zero-Knowledge and Signatures from Symmetric-Key Primitives. In *CCS*, pages 1825–1842. ACM, 2017.
37. David Chaum and Eugène van Heyst. Group Signatures. In *EUROCRYPT*, pages 257–265. Springer, 1991.
38. Keng-Yu Chen and Jiun-Peng Chen. Masking Floating-Point Number Multiplication and Addition of Falcon First- and Higher-order Implementations and Evaluations. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(2):276–303, 2024.
39. Hongrui Cui, Hanlin Liu, Di Yan, Kang Yang, Yu Yu, and Kaiyi Zhang. sfresolved: Shorter signatures from regular syndrome decoding and vole-in-the-head. In *PKC*, pages 229–258. Springer, 2024.

40. Cyprien Delpech de Saint Guilhem, Lauren De Meyer, Emmanuela Orsini, and Nigel P. Smart. BBQ: Using AES in Picnic Signatures. In *SAC*, pages 669–692. Springer, 2019.
41. Cyprien Delpech de Saint Guilhem, Emmanuela Orsini, and Titouan Tanguy. Limbo: Efficient Zero-Knowledge MPCitH-based Arguments. In *CCS*, pages 3022–3036. ACM, 2021.
42. Sanjay Deshpande, James Howe, Jakub Szefer, and Dongze Yue. Sdith in hardware. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(2):215–251, 2024.
43. Julien Devevey, Pouria Fallahpour, Alain Passelègue, and Damien Stehlé. A detailed analysis of fiat-shamir with aborts. In *CRYPTO*, pages 327–357. Springer, 2023.
44. Itai Dinur, Steven Goldfeder, Tzipora Halevi, Yuval Ishai, Mahimna Kelkar, Vivek Sharma, and Greg Zaverucha. Mpc-friendly symmetric cryptography from alternating moduli: Candidates, protocols, and applications. In *CRYPTO*, pages 517–547. Springer, 2021.
45. Itai Dinur and Niv Nadler. Multi-target attacks on the picnic signature scheme and related protocols. In *EUROCRYPT*, pages 699–727. Springer, 2019.
46. Christoph Dobraunig, Daniel Kales, Christian Rechberger, Markus Schafneger, and Greg Zaverucha. Shorter Signatures Based on Tailor-Made Minimalist Symmetric-Key Crypto. In *CCS*, pages 843–857. ACM, 2022.
47. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 356–383. Springer, Heidelberg, August 2019.
48. Stefan Dziembowski, Sebastian Faust, Gottfried Herold, Anthony Journault, Daniel Masny, and François-Xavier Standaert. Towards sound fresh re-keying with hard (physical) learning problems. In *CRYPTO (2)*, pages 272–301. Springer, 2016.
49. Thomas Espitau, Guilhem Niot, Chao Sun, and Mehdi Tibouchi. Squirrels, Square Unstructured Integer Euclidean Lattice Signature. Round 1 : Submission to the NIST Post-Quantum Cryptography : Additional Digital Signature Schemes, 2023. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-1/spec-files/Squirrels-spec-web.pdf>.
50. Sebastian Faust, Loïc Masure, Elena Michel, Hai Hoang Nguyen, Maximilian Ortl, and François-Xavier Standaert. IP masking with generic security guarantees under minimum assumptions, and applications. In *ASIACRYPT (2)*, volume 16246 of *Lecture Notes in Computer Science*, pages 544–575. Springer, 2025.
51. Serge Fehr. Classical proofs for the quantum collapsing property of classical hash functions. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part II*, volume 11240 of *LNCS*, pages 315–338. Springer, Heidelberg, November 2018.
52. Thibault Feneuil, Antoine Joux, and Matthieu Rivain. Syndrome Decoding in the Head: Shorter Signatures from Zero-Knowledge Proofs. In *CRYPTO*, pages 541–572. Springer, 2022.
53. Thibault Feneuil, Jules Maire, Matthieu Rivain, and Damien Vergnaud. Zero-Knowledge Protocols for the Subset Sum Problem from MPC-in-the-Head with Rejection. In *ASIACRYPT*, pages 371–402. Springer, 2022.
54. Thibault Feneuil and Matthieu Rivain. Threshold linear secret sharing to the rescue of mpc-in-the-head. In *ASIACRYPT (1)*, pages 441–473. Springer, 2023.
55. Thibault Feneuil and Matthieu Rivain. Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. *J. Cryptol.*, 38(3):28, 2025.

56. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194. Springer, Heidelberg, August 1987.
57. Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon: Fast-fourier lattice-based compact signatures over ntru (specification v1.2). Round-3 : Submission to the NIST Post-Quantum Cryptography Standardization Project, 2020. <https://falcon-sign.info/falcon.pdf>.
58. Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. ZKBoo: Faster Zero-Knowledge for Boolean Circuits. In *USENIX Security Symposium*, pages 1069–1083. USENIX Association, 2016.
59. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part I*, pages 637–667. Springer, 2021.
60. Shay Gueron, Edoardo Persichetti, and Paolo Santini. Designing a Practical Code-Based Signature Scheme from Zero-Knowledge Proofs with Trusted Setup. *Cryptogr.*, 6(1):5, 2022.
61. Carmit Hazay, Muthuramakrishnan Venkitasubramaniam, and Mor Weiss. Beyond mpc-in-the-head: Black-box constructions of short zero-knowledge proofs. In *TCC*, pages 3–33. Springer, 2023.
62. Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-Knowledge from Secure Multiparty Computation. In *STOC*, pages 21–30. ACM, 2007.
63. Daniel Kales, Sebastian Ramacher, Christian Rechberger, Roman Walch, and Mario Werner. Efficient FPGA implementations of lowmc and picnic. In Stanislaw Jarecki, editor, *Topics in Cryptology - CT-RSA 2020 - The Cryptographers' Track at the RSA Conference 2020, San Francisco, CA, USA, February 24-28, 2020, Proceedings*, volume 12006 of *Lecture Notes in Computer Science*, pages 417–441. Springer, 2020.
64. Daniel Kales and Greg Zaverucha. Improving the performance of the picnic signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(4):154–188, 2020.
65. Matthias Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-Quantum Crypto Library for the ARM Cortex-M4. <https://github.com/mupq/pqm4>, 2019.
66. Matthias J. Kannwischer, Aymeric Génêt, Denis Butin, Juliane Krämer, and Johannes Buchmann. Differential power analysis of XMSS and SPHINCS. In *COSADE*, pages 168–188. Springer, 2018.
67. Jonathan Katz, Vladimir Kolesnikov, and Xiao Wang. Improved non-interactive zero knowledge with applications to post-quantum signatures. In *CCS*, pages 525–537. ACM, 2018.
68. Guoxiao Liu, Keting Jia, Puwen Wei, and Lei Ju. High-performance hardware implementation of mpcith and picnic3. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(2):190–214, 2024.
69. Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In *ASIACRYPT*, pages 598–616. Springer, 2009.
70. Vadim Lyubashevsky. Lattice signatures without trapdoors. In *EUROCRYPT*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755. Springer, 2012.

71. Stelios Manasidis, Quinten Norga, Suparna Kundu, and Ingrid Verbauwhede. Mpspeed: Implementing and optimizing mpc-in-the-head digital signatures in hardware. Cryptology ePrint Archive, Paper 2026/206, 2026. <https://eprint.iacr.org/2026/206>.
72. Loïc Masure, Pierrick Méaux, Thorben Moos, and François-Xavier Standaert. Effective and efficient masking with low noise using small-mersenne-prime ciphers. In *EUROCRYPT (4)*, pages 596–627. Springer, 2023.
73. Carlos Aguilar Melchor, Slim Bettaieb, Loïc Bidoux, Thibault Feneuil, Nicolas Gama, Shay Gueron, James Howe, Andreas Hülsing, David Joseph, Antoine Joux, Mukul Kulkarni, Edoardo Persichetti, Tovahery Hajatiana Randrianarisoa, Matthieu Rivain, and Dongze Yue. The Syndrome Decoding in the Head (SD-in-the-Head) Signature Scheme, Algorithm Specifications and Supporting Documentation – Version 2.0. Round 2 : Submission to the NIST Post-Quantum Cryptography : Additional Digital Signature Schemes, 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/sdith-spec-round2-web.pdf>.
74. Carlos Aguilar Melchor, Nicolas Gama, James Howe, Andreas Hülsing, David Joseph, and Dongze Yue. The return of the sdith. In *EUROCRYPT (5)*, volume 14008 of *Lecture Notes in Computer Science*, pages 564–596. Springer, 2023.
75. Arno Mittelbach and Marc Fischlin. *The Theory of Hash Functions and Random Oracles - An Approach to Modern Cryptography*. Information Security and Cryptography. Springer, 2021.
76. Ying Ouyang, Deng Tang, and Yanhong Xu. Code-based zero-knowledge from vole-in-the-head and their applications: Simpler, faster, and smaller. In *ASIACRYPT*, pages 436–470. Springer, 2024.
77. Ronald L. Rivest, Adi Shamir, and Yael Tauman. How to Leak a Secret. In *ASIACRYPT*, pages 552–565. Springer, 2001.
78. Okan Seker, Sebastian Berndt, Luca Wilke, and Thomas Eisenbarth. Sni-in-the-head: Protecting mpc-in-the-head protocols against side-channel analysis. In *CCS*, pages 1033–1049. ACM, 2020.
79. Léo Ducas Shi Bai, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS–dilithium: Algorithm specification and supporting documentation (version 3.1). Round-3 : Submission to the NIST Post-Quantum Cryptography Standardization Project, 2020. <https://cryptojedi.org/papers/#dilithiumnistr3>.
80. Dominique Unruh. Quantum proofs of knowledge. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 135–152. Springer, Heidelberg, April 2012.
81. Dominique Unruh. Non-interactive zero-knowledge proofs in the quantum random oracle model. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 755–784. Springer, Heidelberg, April 2015.
82. Dominique Unruh. Computationally binding quantum commitments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 497–527. Springer, Heidelberg, May 2016.
83. Cankun Zhao, Neng Zhang, Hanning Wang, Bohan Yang, Wenping Zhu, Zhengdong Li, Min Zhu, Shouyi Yin, Shaojun Wei, and Leibo Liu. A compact and high-performance hardware architecture for crystals-dilithium. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):270–295, 2022.

A New Variants of the LWR Problem

The standard LWR assumption is defined for a uniform matrix $\mathbf{X}$ and uniform vectors $\mathbf{k}, \mathbf{U}$, as

$$(\mathbf{X}, \lfloor \mathbf{X}\mathbf{k} \rfloor) \approx (\mathbf{X}, \mathbf{U}).$$

In the Sigma protocol introduced in Section 5, however, some extra information never makes it to the adversary's view thanks to the abort event. Simplifying, the key $\mathbf{k} \in \mathbb{Z}_Q^n$ is shared into shares $\mathbf{k}_1, \mathbf{k}_2 \in \mathbb{Z}_Q^n$,¹³ then the LWR on the single keys is computed by $\mathbf{y}_i \leftarrow \lfloor \mathbf{X}\mathbf{k}_i \rfloor$ for $i \in \{1, 2\}$, and finally $\mathbf{y}, \mathbf{y}_1, \mathbf{y}_2$ are output in the first message and one of the key shares is revealed to the adversary in the response message. Assuming that we reveal the first share, the view of the adversary is given by the random variable.

$$(\mathbf{X}, \lfloor \mathbf{X}\mathbf{k} \rfloor, \lfloor \mathbf{X}\mathbf{k}_2 \rfloor, \mathbf{k}_1).$$

Notice that, since both $\mathbf{X}$ and $\mathbf{k}_1$ are given to the adversary, we can just simplify and omit $\lfloor \mathbf{X}\mathbf{k}_1 \rfloor$ from the view. This process is repeated several times, and the end result is that the adversary's view is

$$\left(\mathbf{X}, \lfloor \mathbf{X}\mathbf{k} \rfloor, \left(\lfloor \mathbf{X}\mathbf{k}_2^{(j)} \rfloor, \mathbf{k}_1^{(j)} \right)_{j \in [N]} \right), \quad (5)$$

for some N that depends on the number of iterations in the Sigma protocol and the number of signatures. In principle, this is bad because now the adversary gets many samples of the error between $\lfloor \mathbf{X}\mathbf{k} \rfloor$ and $\lfloor \mathbf{X}\mathbf{k}_1 \rfloor + \lfloor \mathbf{X}\mathbf{k}_2 \rfloor$, along with the share $\mathbf{k}_1$ in full, which allows for averaging attacks. We define this new problem in Section A.1, and we define a useful abstraction in Section A.2 that allows us to deal with the problem more easily.

A.1 Learning with Rounding with Misuse

Here, we formally introduce the problem, which we call *Learning with Rounding with Misuse*, or *LWRwM* for short. The name comes from reusing the same matrix $\mathbf{X}$ to compute different instances of LWR, thus allowing averaging attacks on the secret. We describe the LWRwM distributions in Fig. 6, and we define the LWRwM assumption as

$$\text{LWRwM.Real}(\mathbf{k}) \approx \text{LWRwM.Rand}().$$

Notice that the definition in Fig. 6 is slightly different from what we stated at the beginning of the section; namely, instead of giving $\mathbf{y}_2$ to the adversary, we give $\mathbf{u} - \mathbf{r}_2$, where $\mathbf{r}_2 = (\mathbf{v} \bmod P)$. However, by writing

$$\mathbf{u} = \frac{Q}{P}\mathbf{y} + \mathbf{r} \quad (= \mathbf{X}\mathbf{k} \text{ in the Real case}),$$

¹³ In our protocol, we sample d shares instead of just two, but it is easy to see that this can be derived by sharing $\mathbf{k}$ into $\mathbf{k}_1, \mathbf{k}_2$ and then sharing again $\mathbf{k}_1$ into $\mathbf{k}_{1,1}, \dots, \mathbf{k}_{1,d-1}$; additionally, we generate most of the shares using a PRG, which we omit in this section for simplicity.

$\text{LWRwM}(Q, P, n, m, N).\text{Real}(\mathbf{k})$	$\text{LWRwM}(Q, P, n, m, N).\text{Rand}()$
$\mathbf{X} \leftarrow \mathbb{Z}_Q^{m \times n}, \mathbf{u} \leftarrow \mathbf{X}\mathbf{k}$	$\mathbf{X} \leftarrow \mathbb{Z}_Q^{m \times n}, \mathbf{u} \leftarrow \mathbb{Z}_Q^m$
$\mathbf{y} \leftarrow \lfloor \mathbf{u} \rfloor_P$	$\mathbf{y} \leftarrow \lfloor \mathbf{u} \rfloor_P$
for $i \leftarrow [N]$ do	for $i \leftarrow [N]$ do
$\mathbf{k}_1 \leftarrow \mathbb{Z}_Q^n$	$\mathbf{k}_1 \leftarrow \mathbb{Z}_Q^n$
$\mathbf{v} \leftarrow \mathbf{u} - \mathbf{X}\mathbf{k}_1$	$\mathbf{v} \leftarrow \mathbf{u} - \mathbf{X}\mathbf{k}_1$
$\mathbf{r}_2 \leftarrow \mathbf{v} - \frac{Q}{P} \lfloor \mathbf{v} \rfloor_P$	$\mathbf{r}_2 \leftarrow \mathbf{v} - \frac{Q}{P} \lfloor \mathbf{v} \rfloor_P$
$s^{(i)} \leftarrow (\mathbf{u} - \mathbf{r}_2, \mathbf{k}_1)$	$s^{(i)} \leftarrow (\mathbf{u} - \mathbf{r}_2, \mathbf{k}_1)$
return $\left(\mathbf{X}, \mathbf{y}, \left(s^{(i)} \right)_{i \in [N]} \right)$	return $\left(\mathbf{X}, \mathbf{y}, \left(s^{(i)} \right)_{i \in [N]} \right)$

Fig. 6. The real and random distributions of LWRwM, parametrized by the usual LWR parameters and, additionally, the number N of samples.

$$\begin{aligned} \mathbf{X}\mathbf{k}_1 &= \frac{Q}{P}\mathbf{y}_1 + \mathbf{r}_1, \\ \mathbf{v} = \mathbf{u} - \mathbf{X}\mathbf{k}_1 &= \frac{Q}{P}\mathbf{y}_2 + \mathbf{r}_2 \quad (= \mathbf{X}\mathbf{k}_2 \text{ in the Real case}), \end{aligned}$$

with $\mathbf{r}, \mathbf{r}_1, \mathbf{r}_2 \in \mathbb{Z}_{Q/P}^m$, we can write the output $\mathbf{u} - \mathbf{r}_2$ in the LWRwM distribution as

$$\begin{aligned} \mathbf{u} - \mathbf{r}_2 &= \mathbf{u} - \mathbf{v} + \frac{Q}{P} \lfloor \mathbf{v} \rfloor_P \\ &= \mathbf{u} - \mathbf{u} + \mathbf{X}\mathbf{k}_1 + \frac{Q}{P} \lfloor \mathbf{u} - \mathbf{X}\mathbf{k}_1 \rfloor_P \\ &= \mathbf{X}\mathbf{k}_1 + \frac{Q}{P} \lfloor \mathbf{u} - \mathbf{X}\mathbf{k}_1 \rfloor_P. \end{aligned}$$

When the sample comes from the real distribution, $\mathbf{u} = \mathbf{X}\mathbf{k}$, and

$$\lfloor \mathbf{X}\mathbf{k} - \mathbf{X}\mathbf{k}_1 \rfloor_P = \lfloor \mathbf{X}\mathbf{k}_2 \rfloor_P,$$

which is exactly the value appearing in Eq. (5). On the other side, if $\mathbf{u}$ is random, there is no $\mathbf{k}_2$, and the value $\lfloor \mathbf{X}\mathbf{k}_2 \rfloor_P$ must be simulated by $\lfloor \mathbf{u} - \mathbf{X}\mathbf{k}_1 \rfloor_P$, that is, again, what we get in the above derivation.

The LWRwM problem is useful to understand why the adversary gets more information. In particular, by writing the extra information as $\mathbf{u} - \mathbf{r}_2$, we can see that the adversary gets the unrounded LWR value $\mathbf{u}$, which can be either $\mathbf{X}\mathbf{k}$ or uniform, with many samples of the error $\mathbf{r}_2$, which can be used to average the distribution and leak information from $\mathbf{u}$.

A.2 Learning with Rounding and Carry Leakage

LWRCL(Q, P, n, m, N).Real($\mathbf{k}$)	LWRCL(Q, P, n, m, N).Rand()
$\mathbf{X} \leftarrow \mathbb{Z}_Q^{m \times n}, \mathbf{u} \leftarrow \mathbf{X}\mathbf{k}$	$\mathbf{X} \leftarrow \mathbb{Z}_Q^{m \times n}, \mathbf{u} \leftarrow \mathbb{Z}_Q^m$
for $i \leftarrow [N]$ do	for $i \leftarrow [N]$ do
$\mathbf{k}_1 \leftarrow \mathbb{Z}_Q^n$	$\mathbf{k}_1 \leftarrow \mathbb{Z}_Q^n$
$\mathbf{y}_1 \leftarrow \lfloor \mathbf{X}\mathbf{k}_1 \rfloor_P$	$\mathbf{y}_1 \leftarrow \lfloor \mathbf{X}\mathbf{k}_1 \rfloor_P$
$\mathbf{y}_2 \leftarrow \lfloor \mathbf{u} - \mathbf{X}\mathbf{k}_1 \rfloor_P$	$\mathbf{y}_2 \leftarrow \lfloor \mathbf{u} - \mathbf{X}\mathbf{k}_1 \rfloor_P$
$\mathbf{c} \leftarrow \lfloor \mathbf{u} \rfloor_P - \mathbf{y}_1 - \mathbf{y}_2$	$\mathbf{c} \leftarrow \lfloor \mathbf{u} \rfloor_P - \mathbf{y}_1 - \mathbf{y}_2$
$s^{(i)} \leftarrow (\mathbf{k}_1, \mathbf{c})$	$s^{(i)} \leftarrow (\mathbf{k}_1, \mathbf{c})$
return $\left(\mathbf{X}, \lfloor \mathbf{u} \rfloor_P, (s^{(i)})_{i \in [N]} \right)$	return $\left(\mathbf{X}, \lfloor \mathbf{u} \rfloor_P, (s^{(i)})_{i \in [N]} \right)$

Fig. 7. The real and random distributions of LWRCL, parametrized by the usual LWR parameters and, additionally, the number N of samples.

In Fig. 7 introduce another variant of LWR, which we name *learning with rounding and carry leakage* (LWRCL). In this setting, instead of getting the $\lfloor \mathbf{X}\mathbf{k}_2 \rfloor_P$, the adversary learns the error between the original LWR value and the sum of the rounded shares:

$$\mathbf{c} = \lfloor \mathbf{X}\mathbf{k} \rfloor_P - \lfloor \mathbf{X}\mathbf{k}_1 \rfloor_P - \lfloor \mathbf{X}\mathbf{k}_2 \rfloor_P.$$

Transformed for the general case, the carry can be defined as

$$\mathbf{c} = \lfloor \mathbf{u} \rfloor_P - \lfloor \mathbf{X}\mathbf{k}_1 \rfloor_P - \lfloor \mathbf{v} \rfloor_P.$$

The name *carry leakage* comes from the fact that, for $Q = 2^q$ and $P = 2^p$, the value $\mathbf{c}$ is the carry of the sum $\mathbf{r}_1 + \mathbf{r}_2$.

As before, we define the LWRCL assumption as

$$\text{LWRCL.Real}(\mathbf{k}) \approx \text{LWR.Rand}().$$

We also show that the two problems are equivalent by constructing an LWRCL sample from an LWRwM sample and vice versa. Recall that, from the previous section,

$$\mathbf{r}_2 = (\mathbf{v} \bmod P) = \mathbf{v} - \frac{Q}{P} \lfloor \mathbf{v} \rfloor_P = \mathbf{v} - \frac{Q}{P} \mathbf{y}_2,$$

and, therefore,

$$\begin{aligned} \lfloor \mathbf{u} \rfloor_P &= \lfloor \mathbf{X}\mathbf{k}_1 + \mathbf{v} \rfloor_P \\ &= \left\lfloor \frac{Q}{P} \mathbf{y}_1 + \mathbf{r}_1 + \frac{Q}{P} \mathbf{y}_2 + \mathbf{r}_2 \right\rfloor_P \end{aligned}$$

$$= \mathbf{y}_1 + \mathbf{y}_2 + \lfloor \mathbf{r}_1 + \mathbf{r}_2 \rfloor_P,$$

therefore, by plugging in $\mathbf{y}_1 = \lfloor \mathbf{X}\mathbf{k}_1 \rfloor_P$ and $\mathbf{y}_2 = \lfloor \mathbf{X}\mathbf{k}_2 \rfloor_P$, we get that

$$\mathbf{c} = \lfloor \mathbf{r}_1 + \mathbf{r}_2 \rfloor_P.$$

Getting carry leakage from misuse sampling. Given an instance of LWRwM, the adversary obtains $\mathbf{k}_1$ and $\mathbf{u} - \mathbf{r}_2$, and can compute

$$\begin{aligned} (\mathbf{u} - \mathbf{r}_2) - \mathbf{r}_1 &= (\mathbf{X}\mathbf{k}_1 + \mathbf{v} - \mathbf{r}_2) - \mathbf{r}_1 \\ &= \frac{Q}{P}(\mathbf{y}_1 + \mathbf{y}_2), \end{aligned}$$

hence the carry can be obtained by

$$\begin{aligned} \mathbf{c} &= \lfloor \mathbf{u} \rfloor_P - \lfloor \mathbf{X}\mathbf{k}_1 \rfloor_P - \lfloor \mathbf{v} \rfloor_P \\ &= \lfloor \mathbf{u} \rfloor_P - (\mathbf{y}_1 + \mathbf{y}_2) \\ &= \lfloor \mathbf{u} \rfloor_P - \lfloor (\mathbf{u} - \mathbf{r}_2) - \mathbf{r}_1 \rfloor_P. \end{aligned}$$

Getting misuse sampling from carry leakage. Given an instance of LWRCL, the adversary obtains $\mathbf{k}_1$ and the carry $\mathbf{c}$, and can compute

$$\begin{aligned} (\mathbf{u} - \mathbf{r}_2) &= (\mathbf{X}\mathbf{k}_1 + \mathbf{v} - \mathbf{r}_2) \\ &= \frac{Q}{P}\mathbf{y}_1 + \mathbf{r}_1 + \frac{Q}{P}\mathbf{y}_2 + \mathbf{r}_2 - \mathbf{r}_2 \\ &= \frac{Q}{P}(\mathbf{y}_1 + \mathbf{y}_2 + \mathbf{c} - \mathbf{c}) + \mathbf{r}_1 \\ &= \frac{Q}{P}(\lfloor \mathbf{u} \rfloor_P - \mathbf{c}) + \mathbf{r}_1, \end{aligned}$$

thus getting back the misuse sampling.

A.3 Reducing LWRCL to LWR

In this subsection, we show that, for $Q = 2^q$ and $P = 2^p$, we can easily reduce the LWRCL problem to the LWR problem, provided that we have α extra bits to use for the reduction. The theorem below proves exactly this fact by adding a distinguishing error of $2^{1-\alpha}Nm + \epsilon_{\text{LWR}}$ caused by the fact that, intuitively, one of the shares $\mathbf{k}_1$ given to the adversary is “too close” to the original key $\mathbf{k}$ after the matrix multiplication.

Theorem 11. *Let $q, p, n, m, N, \alpha \in \mathbb{N}$ such that $q > p$ and $2^\alpha > Nm$. Assume that the decisional assumption*

$$\text{LWR}(2^q, 2^{p+\alpha}, n, m)$$

holds with advantage ϵ_{LWR} , for some $\epsilon_{\text{LWR}} \in [0, 1]$. Then, the decisional assumption

$$\text{LWRCL}(2^q, 2^p, n, m, N)$$

holds with advantage

$$\epsilon_{\text{LWRCL}} := 2 \cdot (2^{-\alpha} N m + \epsilon_{\text{LWR}}).$$

Looking ahead, the check of being “too close” is performed by the reduction using the α additional bits given in the sample. For this reason, the original LWR instance must be secure even with parameters $(2^q, 2^{p+\alpha}, n, m)$. Unfortunately, this means that α should be quite large in order to achieve a negligible advantage. Interestingly, if we limit the adversary to only see LWRCL samples that are not “too close” to the original LWR sample (i.e., in which the α bits are different), α does not need to be large anymore. This strategy is what we use in our Sigma protocol in [Section 5](#), where the protocol aborts if the α bits of the shares match the ones in the original LWR sample.

$\mathcal{R}(\mathbf{X}, \mathbf{y})$ where $\mathbf{X} \in \mathbb{Z}_{2^q}^{m \times n}, \mathbf{y} \in \mathbb{Z}_{2^{p+\alpha}}^m$
<pre> 1 : $r_S \leftarrow q - p - \alpha, r_E \leftarrow r_S + \alpha$ 2 : for $i \leftarrow 1..N$ do 3 : $\mathbf{k}_1 \leftarrow \mathbb{Z}_{2^q}^n$ 4 : for $j \leftarrow 1..m$ do // Get the j-th component of the vector $\mathbf{X}\mathbf{k}_1 \in \mathbb{Z}_{2^q}^m$. 5 : $z \leftarrow (\mathbf{X}\mathbf{k}_1)_j$ // Compare z with the challenge $\mathbf{y}$ and set the carry accordingly. 6 : if $z[r_S..r_E] < y_j[0..\alpha]$ then $c_j \leftarrow 0$ 7 : elseif $z[r_S..r_E] > y_j[0..\alpha]$ then $c_j \leftarrow 1$ // If a strict ordering is not possible, the carry is determined by one of the // least significant bits of $\mathbf{X}\mathbf{k}$, hence $\mathcal{R}$ has not enough information. 8 : else abort 9 : $\mathbf{c} := (c_1, \dots, c_m)$ 10 : $s^{(i)} \leftarrow (\mathbf{k}_1, \mathbf{c})$ 11 : return $\left(\mathbf{X}, [\mathbf{y}]_{2^p}, \left(s^{(i)} \right)_{i \in [N]} \right)$ </pre>

Fig. 8. The reduction from LWRCL to LWR

The rest of this section is dedicated to the proof of the above theorem. We proceed by reduction. Namely, consider the reduction $\mathcal{R}$ depicted in [Fig. 8](#).

Let $\text{BAD}_j^{(i)}$ be the event where $z[r_S..r_E] = y_j[0..\alpha]$ in iteration i , and let

$$\text{BAD} := \bigcup_{i \in [N]} \bigcup_{j \in n} \text{BAD}_j^{(i)}.$$

In particular, BAD happens in the reduction if and only if $\mathcal{R}$ aborts.

Lemma 4. *Let $q, p, n, m, N, \alpha \in \mathbb{N}$ such that $q > p$ and $2^\alpha > Nm$. Assume that BAD does not happen. Then, the reduction $\mathcal{R}$ is perfect.*

Proof. The only difference between the output of LWRCL and the output of the reduction $\mathcal{R}$ lies in how the carry vector is computed. Assuming that the two ways to compute the carry lead to the same result, it is easy to see that the reduction perfectly simulates LWRCL.Real when receiving a sample from LWR.Real as input and perfectly simulates LWRCL.Rand when receiving a sample from LWR.Rand as input. Therefore, the rest of the proof will focus on the carry computation.

In the original LWRCL experiments, the carry is computed as

$$\mathbf{c} \leftarrow \lfloor \mathbf{u} \rfloor_{2^p} - \lfloor \mathbf{X}\mathbf{k}_1 \rfloor_{2^p} - \lfloor \mathbf{u} - \mathbf{X}\mathbf{k}_1 \rfloor_{2^p},$$

where $\mathbf{u}$ is either the real computation $\mathbf{X}\mathbf{k}$ or a uniformly sampled vector in $\mathbb{Z}_{2^q}^m$. By letting $\mathbf{z} := \mathbf{X}\mathbf{k}_1$, this means that, for every component $j \in [m]$ of the vectors, the carry is computed as

$$c_j \leftarrow \lfloor u_j \rfloor_{2^p} - \lfloor z_j \rfloor_{2^p} - \lfloor u_j - z_j \rfloor_{2^p}.$$

We can now multiply by 2^{q-p} and apply [Proposition 2](#) to get

$$\begin{aligned} 2^{q-p}c_j &= u_j - u_j[0..(q-p)] \\ &\quad - z_j + z_j[0..(q-p)] \\ &\quad - (u_j - z_j) + (u_j - z_j)[0..(q-p)] \\ &= (u_j - z_j)[0..(q-p)] + z_j[0..(q-p)] - u_j[0..(q-p)]. \end{aligned}$$

Notice that all the values on the right-hand side of the equality are $(q-p)$ -bits long, hence they are all strictly less than 2^{q-p} . This means that

$$-2^{q-p} < 2^{q-p}c_j < 2 \cdot 2^{q-p}.$$

Since c_j is an integer, the only two possible values for it are either $c_j = 0$ or $c_j = 1$. We have:

$$\begin{aligned} &- \text{ if } z_j[0..(q-p)] \leq u_j[0..(q-p)], \\ &\quad 2^{q-p}c_j \leq (u_j - z_j)[0..(q-p)] < 2^{q-p}, \end{aligned}$$

which means that c_j cannot be 1, hence $c_j = 0$;

$$\begin{aligned} &- \text{ if } z_j[0..(q-p)] > u_j[0..(q-p)], \\ &\quad 2^{q-p}c_j > (u_j - z_j)[0..(q-p)] \geq 0, \end{aligned}$$

which means that c_j cannot be 0, hence $c_j = 1$.

To conclude, it suffices to check that the reduction does the same. First of all, notice that $y_j = \lfloor u_j \rfloor_{2^{p+\alpha}}$ or, by applying [Proposition 2](#) and recalling the definition of r_S and r_E ,

$$y_j = u_j[(q-p-\alpha)..q] = u_j[r_S..q].$$

Then, by applying again [Proposition 2](#) and $r_E = r_S + \alpha$, we get that

$$y_j[0..\alpha] = u_j[r_S..q][0..\alpha] = u_j[r_S..(r_S + \alpha)] = u_j[r_S..r_E].$$

Now, assuming that BAD does not happen, there are only two possibilities for the comparison between $z[r_S..r_E]$ and $y_j[0..\alpha] = u_j[r_S..r_E]$

- If $z[r_S..r_E] < u_j[r_S..r_E]$, by [Proposition 2](#), $z[0..r_E] < u_j[0..r_E]$, and both $\mathcal{R}$ and the original LWRCL experiments set $c_j = 0$.
- If $z[r_S..r_E] > u_j[r_S..r_E]$, by [Proposition 2](#), $z[0..r_E] > u_j[0..r_E]$, and both $\mathcal{R}$ and the original LWRCL experiments set $c_j = 1$.

This concludes the proof.

Lemma 5. *Let $q, p, n, m, N, \alpha \in \mathbb{N}$ such that $q > p$ and $2^\alpha > Nm$. Then,*

$$\mathbb{P}[\text{BAD happens} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Rand}()] \leq 2^{-\alpha} Nm.$$

Proof. We start by bounding the probability of the event $\text{BAD}_j^{(i)}$; then, the proof will follow by the union bound. The event $\text{BAD}_j^{(i)}$ happens if and only if, when sampling z in line 5 of the reduction, it happens that $z[r_S..r_E] = y_j[0..\alpha]$. However, when the reduction receives $(\mathbf{X}, \mathbf{y})$ from LWR.Rand , the value y_j is uniformly sampled, and the value z is sampled independently of it. This means that

$$\begin{aligned} & \mathbb{P}[\text{BAD}_j^{(i)} \text{ happens} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Rand}()] \\ &= \mathbb{P}[z[r_S..r_E] = y_j[0..\alpha] \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Rand}()] \\ &= \mathbb{P}[z[r_S..r_E] = \mathbf{U}_{\mathbb{Z}_{2^\alpha}}] \\ &= 2^{-\alpha}. \end{aligned}$$

Since

$$\text{BAD} = \bigcup_{i \in [N]} \bigcup_{j \in n} \text{BAD}_j^{(i)},$$

the proof follows by the union bound. $\square$

Lemma 6. *Let $q, p, n, m, N, \alpha \in \mathbb{N}$ such that $q > p$ and $2^\alpha > Nm$. Then, for a uniformly sampled $\mathbf{k}$,*

$$\mathbb{P}[\text{BAD happens} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Real}(\mathbf{k})] \leq 2^{-\alpha} Nm + \epsilon_{\text{LWR}}.$$

Proof. By reduction to the $\text{LWR}(2^q, 2^{p+\alpha}, n, m)$ problem. Assume that BAD happens with probability more than $2^{-\alpha} Nm + \epsilon_{\text{LWR}}$ when the sample comes from the real distribution, and consider the following distinguisher.

$\mathcal{D}(\mathbf{X}, \mathbf{y})$ where $\mathbf{X} \in \mathbb{Z}_{2^q}^{m \times n}$, $\mathbf{y} \in \mathbb{Z}_{2^{p+\alpha}}^m$

```

1:  run  $\mathcal{R}(\mathbf{X}, \mathbf{y})$ 
2:  if  $\mathcal{R}$  aborts then
3:    return 1
4:  else
5:    return 0
```

Recall that, by hypothesis,

$$\mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Real}(\mathbf{k})] > 2^{-\alpha} Nm + \epsilon_{\text{LWR}},$$

and, by [Lemma 5](#),

$$2^{-\alpha} Nm \geq \mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Rand}()].$$

Therefore, the difference

$$\mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Real}(\mathbf{k})] - \mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Rand}()]$$

is positive and we can drop the absolute value. Furthermore, by definition of $\mathcal{D}$,

$$\mathbb{P}[\mathcal{D}(\mathbf{X}, \mathbf{y}) = 1 \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{Real}(\mathbf{k})] = \mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Real}(\mathbf{k})]$$

and

$$\mathbb{P}[\mathcal{D}(\mathbf{X}, \mathbf{y}) = 1 \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{Rand}()] = \mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{LWR.Rand}()].$$

With these considerations, we can compute the distinguishing advantage of $\mathcal{D}$ as

$$\begin{aligned} & |\mathbb{P}[\mathcal{D}(\mathbf{X}, \mathbf{y}) = 1 \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{Real}(\mathbf{k})] \\ & \quad - \mathbb{P}[\mathcal{D}(\mathbf{X}, \mathbf{y}) = 1 \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{Rand}()]| \\ &= |\mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{Real}(\mathbf{k})] \quad \quad \quad (\text{by definition of } \mathcal{D}) \\ & \quad - \mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{Rand}()]| \\ &= \mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{Real}(\mathbf{k})] \quad \quad \quad (\text{by positivity of the difference}) \\ & \quad - \mathbb{P}[\text{BAD} \mid (\mathbf{X}, \mathbf{y}) \leftarrow^s \text{Rand}()] \\ &> 2^{-\alpha} Nm + \epsilon_{\text{LWR}} - 2^{-\alpha} Nm \quad \quad \quad (\text{by hypothesis and Lemma 5}) \\ &= \epsilon_{\text{LWR}}. \end{aligned}$$

This means that $\mathcal{D}$ is a distinguisher for the $\text{LWR}(2^q, 2^{p+\alpha}, n, m)$ with advantage more than ϵ_{LWR} , which is against the assumption of hardness of the decisional LWR problem. $\square$

We are now ready to prove [Theorem 11](#).

Proof. The proof proceeds by a reduction $\mathcal{R}$ to the $\text{LWR}(2^q, 2^{p+\alpha}, n, m)$ problem. Assume that there exists a distinguisher $\mathcal{D}$ such that

$$|\mathbb{P}[\mathcal{D}(\text{LWRCL.Real}(\mathbf{k})) = 1] - \mathbb{P}[\mathcal{D}(\text{LWRCL.Rand}()) = 1]| > \epsilon_{\text{LWRCL}}.$$

We show that $\mathcal{D} \circ \mathcal{R}$ is a distinguisher for the LWR problem with an advantage of more than ϵ_{LWR} . We start by observing that, since the only difference between the computation of $\mathcal{D}(\mathcal{R}(\text{LWR.Real}(\mathbf{k})))$ and $\mathcal{D}(\text{LWRCL.Real}(\mathbf{k}))$ is that $\mathcal{D} \circ \mathcal{R}$ aborts when BAD happens,

$$|\mathbb{P}[\mathcal{D}(\mathcal{R}(\text{LWR.Real}(\mathbf{k}))) = 1] - \mathbb{P}[\mathcal{D}(\text{LWRCL.Real}(\mathbf{k})) = 1]|$$

$$\begin{aligned}
&\leq \mathbb{P}[\text{BAD}] \\
&\leq 2^{-\alpha}Nm + \epsilon_{\text{LWR}},
\end{aligned}$$

where the last inequality is simply the result of [Lemma 6](#). Similarly, the event $\mathcal{R}$ is the only difference between $\mathcal{D}(\mathcal{R}(\text{LWR}.\text{Rand}()))$ and $\mathcal{D}(\text{LWRCL}.\text{Rand}())$ as well, therefore

$$\begin{aligned}
&|\mathbb{P}[\mathcal{D}(\mathcal{R}(\text{LWR}.\text{Rand}())) = 1] - \mathbb{P}[\mathcal{D}(\text{LWRCL}.\text{Rand}()) = 1]| \\
&\leq \mathbb{P}[\text{BAD}] \\
&\leq 2^{-\alpha}Nm,
\end{aligned}$$

where the last inequality is simply the result of [Lemma 5](#). From the above observation, and the assumption that $\mathcal{D}$ is a distinguisher for the $\text{LWRCL}(2^q, 2^{p+\alpha}, n, m, N)$ problem,

$$\begin{aligned}
&|\mathbb{P}[\mathcal{D}(\mathcal{R}(\text{LWR}.\text{Real}(\mathbf{k}))) = 1] - \mathbb{P}[\mathcal{D}(\mathcal{R}(\text{LWR}.\text{Rand})) = 1]| \\
&\geq |\mathbb{P}[\mathcal{D}(\text{LWRCL}.\text{Real}(\mathbf{k})) = 1] - \mathbb{P}[\mathcal{D}(\text{LWRCL}.\text{Rand}()) = 1]| \\
&\quad - |\mathbb{P}[\mathcal{D}(\mathcal{R}(\text{LWR}.\text{Real}(\mathbf{k}))) = 1] - \mathbb{P}[\mathcal{D}(\text{LWRCL}.\text{Real}(\mathbf{k})) = 1]| \\
&\quad - |\mathbb{P}[\mathcal{D}(\mathcal{R}(\text{LWR}.\text{Rand}())) = 1] - \mathbb{P}[\mathcal{D}(\text{LWRCL}.\text{Rand}()) = 1]| \\
&> \epsilon_{\text{LWRCL}} - 2 \cdot 2^{-\alpha}Nm - \epsilon_{\text{LWR}} \\
&= \epsilon_{\text{LWR}},
\end{aligned}$$

which concludes the proof. $\square$

B Standard Cryptographic Primitives

B.1 Signature Schemes

Let ROdist be a distribution on functions, modeling the distribution of the random oracle H . A *signature scheme* is a triple of classical polynomial-time algorithms $\Pi_{\text{Sign}} = (\text{S.Gen}, \text{Sign}, \text{S.Verify})$ specified as follows.

Key Generation: The probabilistic key generation algorithm S.Gen takes as input the security parameter $\lambda \in \mathbb{N}$ (in unary) and returns a pair of keys $(vk, sk) \in \mathcal{V} \times \mathcal{S}$. We call vk the *verification key* and sk the *signing key*.

Signature Generation: The probabilistic signing algorithm Sign takes as input a message $msg \in \mathcal{M}$ and a signing key sk , and outputs a signature $\sigma \leftarrow^* \text{Sign}^H(sk, msg)$ with $\sigma \in \Sigma$.

Signature Verification: The deterministic verification algorithm S.Verify takes as input a pair (msg, σ) and a verification key vk , and outputs a bit. A signature σ is called *valid* with respect to msg and vk iff $\text{S.Verify}^H(vk, msg, \sigma) = 1$.

In what follows, we define *correctness* and *unforgeability* and we refer to the experiments in [Fig. 9](#). The scheme satisfies ϵ_{Corr} -*correctness* if for every quantum polynomial-time adversary A ,

$$\mathbb{P}[\text{Corr}_{\Pi_{\text{Sign}}, H}(A) = 0] \leq \epsilon_{\text{Corr}}.$$

$\text{Corr}_{\Pi_{\text{Sign}}, H}(\mathcal{A})$	$\text{UF} - \text{CMA}_{\Pi_{\text{Sign}}, H}(\mathcal{A})$
$H \leftarrow \$ \text{ROdist}$	$H \leftarrow \$ \text{ROdist}$
$(vk, sk) \leftarrow \$ \text{S.Gen}(1^\lambda)$	$(vk, sk) \leftarrow \$ \text{S.Gen}(1^\lambda)$
$msg \leftarrow \$ \mathcal{A}^H(vk)$	$(msg^*, \sigma^*) \leftarrow \$ \mathcal{A}^{H(\cdot), \text{Sign}(sk, \cdot)}(vk)$
$\sigma \leftarrow \$ \text{Sign}^H(sk, msg)$	if $msg^* \in \mathcal{Q}_{\text{Sign}}$
$b = \text{S.Verify}^H(sk, msg, \sigma)$	return 0
return b	else
	$b = \text{S.Verify}^H(vk, msg^*, \sigma^*)$
	return b

Fig. 9. The experiments Corr and $\text{UF} - \text{CMA}$ for correctness and unforgeability. Here, $\mathcal{Q}_{\text{Sign}}$ is the list of all queries m to $\text{Sign}^H(sk, \cdot)$. The oracle access to Sign is assumed to be classical, while the one to H is quantum.

As for security, we consider *unforgeability*. That is, no efficient attacker can forge a valid message-signature pair (msg^*, σ^*) for a fresh message msg^* without knowledge of the signing key.

Definition 9 (Unforgeable Signature). *We call a signature scheme Π_{Sign} $(\epsilon_{\text{Sign}}, N_H, N_{\text{Sign}})$ -unforgeable if for all quantum polynomial-time adversaries $\mathcal{A}$ asking at most N_H queries to H and N_{Sign} queries to Sign , then:*

$$\mathbb{P}[\text{UF} - \text{CMA}_{\Pi_{\text{Sign}}, H}(\mathcal{A}) = 1] \leq \epsilon_{\text{Sign}}(\lambda).$$

We say that Π_{Sign} is $(\epsilon_{\text{Sign}}, N_H)$ -unforgeable against no-message attacks if it is $(\epsilon_{\text{Sign}}, N_H, 0)$ -unforgeable.

B.2 Pseudorandom Generators

A pseudorandom generator (PRG) is a deterministic polynomial-time algorithm G that, upon input a seed of true randomness, expands it into a much longer pseudorandom string.

Definition 10 (Pseudorandom generator). *We call $G : \mathcal{P} \rightarrow \mathcal{K}$ a ϵ_{PRG} -secure PRG if it holds that $|\mathcal{K}| > |\mathcal{P}|$, and additionally that for every quantum polynomial-time adversary $\mathcal{A}$,*

$$|\mathbb{P}[\mathcal{A}(G(\mathbf{U}_{\mathcal{P}})) = 1] - \mathbb{P}[\mathcal{A}(\mathbf{U}_{\mathcal{K}}) = 1]| \leq \epsilon_{\text{PRG}}$$

where $\mathbf{U}_S$ denotes the uniform distribution over the set S .

B.3 Weak Pseudo-Random Functions

Let $\lambda \in \mathbb{N}$, let $\mathcal{K}(\lambda)$, $\mathcal{X}(\lambda)$ and $\mathcal{Y}(\lambda)$ be sets. Let $F(\lambda) : \mathcal{K}(\lambda) \times \mathcal{X}(\lambda) \rightarrow \mathcal{Y}(\lambda)$ be an efficient function. For succinctness, we omit λ from now on. The property we

consider is that of *weak-pseudorandomness*. Intuitively, this means that N evaluations of F over a random key and a random input look random as long as the key stays secret. In other words, we work with a relaxed version of conventional *pseudorandomness*, where the inputs to the function are randomly sampled. We make weak pseudorandomness formal by considering the experiments in Fig. 10.

$\text{wPRF.Real}_F(\mathbf{A})$	$\text{wPRF.Rand}_F(\mathbf{A})$
$k \leftarrow \$ \mathcal{K}$	$k \leftarrow \$ \mathcal{K}$
$(x_i)_{i \in [N]} \leftarrow \$ \mathcal{X}^N$	$(x_i)_{i \in [N]} \leftarrow \$ \mathcal{X}^N$
$(y_i)_{i \in [N]} \leftarrow (F_k(x_i))_{i \in [N]}$	$(y_i)_{i \in [N]} \leftarrow \$ \mathcal{Y}^N$
$b \leftarrow \$ \mathbf{A}((x_i, y_i)_{i \in [N]})$	$b \leftarrow \$ \mathbf{A}((x_i, y_i)_{i \in [N]})$
return b	return b

Fig. 10. The experiments defining the weak pseudorandomness of F .

Definition 11 (Weak Pseudorandom Function). F is a $(N, \epsilon_{\text{wPRF}})$ -weak pseudorandom function (wPRF) if for every quantum polynomial-time adversary $\mathbf{A}$,

$$|\mathbb{P}[\text{wPRF.Real}_F(\mathbf{A}) = 1] - \mathbb{P}[\text{wPRF.Rand}_F(\mathbf{A}) = 1]| \leq \epsilon_{\text{wPRF}}.$$

We additionally consider the conventional notion of *weak security against key recovery*, where the adversary gets an input-output pair and wins when it finds the key used to generate it. As for the previous definition, the input is not adversarially chosen, but uniformly random.

Definition 12 (Weak Security against Key Recovery). F is ϵ_{wKR} -weakly secure against key recovery if for every quantum polynomial time adversary $\mathbf{A}$,

$$\mathbb{P}[\text{KR}_F(\mathbf{A}) = 1] \leq \epsilon_{\text{wKR}},$$

where $\text{KR}_F(\mathbf{A})$ is the experiment defined as follows:

$\text{KR}_F(\mathbf{A})$
$k \leftarrow \$ \mathcal{K}$
$x \leftarrow \$ \mathcal{X},$
$k' \leftarrow \$ \mathbf{A}(x, F_k(x))$
if $k' = k$
return 1
return 0

It is well known that any pseudorandom function is also secure against key recovery. The same observation stays true for uniformly random inputs. We make it formal in the following lemma, and for completeness, provide a proof in Section C.4.

Lemma 7. *Let F be a $(N, \epsilon_{\text{wPRF}})$ -weak pseudorandom function with output space $\mathcal{Y}$, with $N \geq 2$. Then, F is also ϵ_{wKR} -secure against key recovery, with $\epsilon_{\text{wKR}} \leq \epsilon_{\text{wPRF}} + \frac{1}{|\mathcal{Y}|^{N-1}}$.*

B.4 Hash Functions

We call *hash function* a non-injective function H that maps arbitrary-length inputs into fixed-length outputs.

The notion of *collapsing* hash function was introduced by Unruh [82] to extend the notion of collision-resistance to the quantum setting. Intuitively, it states that a quantum adversary able to find collisions to the hash functions cannot tell whether they are measured or not. This notion is used to prove that our sigma protocol achieves quantum computationally unique responses (cf. Section 3.4).

Definition 13 (Collapsing Hash Function [82]). *Let H be a hash function, and $A = (A_1, A_2)$ be a two-stage adversary. Consider the following two games*

Game ₁ (A)	Game ₂ (A)
$(state, K, h) \leftarrow A_1$	$(state, K, h) \leftarrow A_1$
$k \leftarrow \mathcal{M}(K)$	
$b \leftarrow A_2(state, K)$	$b \leftarrow A_2(state, K)$
return b	return b

Here, $\mathcal{M}$ denotes a measurement in the computational basis. We call an adversary A valid if $\Pr[H(k) = h] = 1$ when we run $(state, K, h) \leftarrow A$ and measure K in the computational basis as k . A function H is collapsing iff for any quantum-polynomial-time valid adversary A , then

$$|\Pr[\text{Game}_1(A) = 1] - \Pr[\text{Game}_2(A) = 1]| \leq \epsilon_{\text{Coll}}.$$

For a hash function H , we call *canonical commitment scheme* the couple of algorithms (Commit, Open) defined as follows. Commit takes as input one value x , concatenates it with some fresh randomness ρ , and outputs $H(x \parallel \rho)$. Open outputs x and ρ .

The canonical commitment scheme is implicitly used in our construction, as we hash the key shares with uniformly random coins. Proving its zero-knowledge requires relying on its *hiding property*, which we report below for completeness.

Definition 14 (Computationally Hiding). *We call a (hash-based) commitment scheme H ϵ_{Hide} -computationally hiding if for every pair of inputs x_0, x_1 , random coins ρ_0, ρ_1 and every quantum polynomial-time adversary A ,*

$$|\mathbb{P}[A(H(x_0 \parallel \rho_0)) = 1] - \mathbb{P}[A(H(x_1 \parallel \rho_1)) = 1]| \leq \epsilon_{\text{Hide}}.$$

The hiding property of the canonical commitment scheme is proven in the random oracle model in Lemma 17 of [82].

C Other results and omitted proofs

C.1 Proof of Lemma 1

First, we observe that f always produces pairs $((x, y), k) \in \mathcal{R}$. This proves the first part of the definition of HIG with $\epsilon_{\text{Rel}} = 0$. For the analysis of ϵ_{HIG} , we introduce the following variants of $f.\text{HIG}(\mathbf{A})$:

$\text{Hyb}^{(1)}(\mathbf{A})$	$\text{Hyb}^{(2)}(\mathbf{A})$
$x \leftarrow \mathcal{X},$	$x \leftarrow \mathcal{X},$
$y \leftarrow \mathbb{Z}_P^m,$	$y \leftarrow \mathbb{Z}_P^m,$
$k' \leftarrow \mathbf{A}(x, y)$	$k' \leftarrow \mathbf{A}(x, y)$
if $ F_{k'}(x) - y \leq \delta$	if $ F_{k'}(x) - y = 0$
return 1	return 1
return 0	return 0

Lemma 1 follows from the claims below.

Claim 1 $\left| \Pr[f.\text{HIG}(\mathbf{A}) = 1] - \Pr[\text{Hyb}^{(1)}(\mathbf{A}) = 1] \right| \leq \epsilon_{\text{wPRF}}.$

Claim 2 For every $\mathbf{A}$ there exists $\tilde{\mathbf{A}}$ such that

$$\Pr[\text{Hyb}^{(1)}(\mathbf{A}) = 1] = (2\delta + 1)^m \cdot \Pr[\text{Hyb}^{(2)}(\tilde{\mathbf{A}}) = 1].$$

Claim 3 $\Pr[\text{Hyb}^{(2)}(\mathbf{A}) = 1] \leq \frac{|\mathcal{K}|}{|\mathbb{Z}_P^m|} \cdot \epsilon_{\text{wKR}}.$

First, we show how to derive Lemma 1 from the three claims above.

Proof (Lemma 1). We can rewrite $\Pr[f.\text{HIG}(\mathbf{A}) = 1]$ as

$$\left(\Pr[f.\text{HIG}(\mathbf{A}) = 1] - \Pr[\text{Hyb}^{(1)}(\mathbf{A}) = 1] \right) + \Pr[\text{Hyb}^{(1)}(\mathbf{A}) = 1].$$

Claim 1 and Claim 2 allow to upper bound the above equation with $\epsilon_{\text{wPRF}} + (2\delta + 1)^m \Pr[\text{Hyb}^{(2)}(\mathbf{A}) = 1]$, and thus the lemma follows by using Claim 3. $\square$

We now proceed with the proofs of each of the above claims.

Proof (Claim 1). This is a standard reduction to the PRF. Let $\mathbf{A}$ be an adversary in the experiments $f.\text{HIG}$ and $\text{Hyb}^{(1)}(\mathbf{A})$, i.e. an algorithm that, upon receiving $(x, y) \in \mathcal{X} \times \mathbb{Z}_P^m$, outputs a candidate key k' . Let $\tilde{\mathbf{A}}$ be an adversary for ϵ_{wPRF} that runs $\mathbf{A}$ on input (x_1, y_1) , and returns 1 whenever $\mathbf{A}$ outputs k' satisfying $|F_{k'}(x_1) - y_1| \leq \delta$, and 0 otherwise. Then

$$\left| \mathbb{P} \left[b = 1 \left| \begin{array}{l} k \leftarrow \mathcal{K}, \\ \{x_i\}_{i \in [N]} \leftarrow \mathcal{X}^N, \\ b \leftarrow \tilde{\mathbf{A}}(\{x_i, F_k(x_i)\}_{i \in [N]}) \end{array} \right. \right] - \mathbb{P} \left[b = 1 \left| \begin{array}{l} \{x_i\}_{i \in [N]} \leftarrow \mathcal{X}^N, \\ \{y_i\}_{i \in [N]} \leftarrow (\mathbb{Z}_P^m)^N, \\ b \leftarrow \tilde{\mathbf{A}}(\{x_i, y_i\}_{i \in [N]}) \end{array} \right. \right] \right|$$

$$= \left| \mathbb{P} \left[\left| F_{k'}(x_1) - y_1 \right| \leq \delta \left| \begin{array}{l} k \leftarrow \mathcal{K} \\ x_1 \leftarrow \mathcal{X} \\ k' \leftarrow \mathbf{A}(x_1, F_k(x_1)) \end{array} \right. \right] - \mathbb{P} \left[\left| F_{k'}(x_1) - y_1 \right| \leq \delta \left| \begin{array}{l} x_1 \leftarrow \mathcal{X} \\ y_1 \leftarrow \mathbb{Z}_P^m \\ k' \leftarrow \mathbf{A}(x_1, y_1) \end{array} \right. \right] \right|.$$

Since the first term represents a particular instance of ϵ_{wPRF} with $N = 1$, while the second yields an arbitrary instance of $|\Pr[f.\text{HIG}(\mathbf{A}) = 1] - \Pr[\text{Hyb}^{(1)}(\mathbf{A}) = 1]|$, the claim follows.

Proof (Claim 2). This claim formalizes the intuition that finding k' satisfying $|F_{k'}(x) - y| \leq \delta$ is $(2\delta + 1)^m$ times easier than finding k' satisfying $F_{k'}(x) = y$. We now prove this observation formally. Let $\mathbf{A}$ be an adversary for $\text{Hyb}^{(1)}$. We construct an adversary $\tilde{\mathbf{A}}$ for $\text{Hyb}^{(2)}$. Upon input two random values (x, y) , $\tilde{\mathbf{A}}$ samples a random e in the interval $[-\delta, +\delta]^m$ and computes $y' = y + e$. It runs $\mathbf{A}$ upon input (x, y') , and outputs the same key k' as $\mathbf{A}$. Note that whenever $\mathbf{A}$ produces a key k' for (x, y') , then k' satisfies $F_{k'}(x) = y$ with probability $\frac{1}{(2\delta+1)^m}$, as

$$\begin{aligned} \mathbb{P} \left[F_{k'}(x) = y \left| \begin{array}{l} x \leftarrow \mathcal{X}, \\ y \leftarrow \mathbb{Z}_P^m, \\ k' \leftarrow \tilde{\mathbf{A}}(x, y) \end{array} \right. \right] &= \mathbb{P} \left[F_{k'}(x) = y \left| \begin{array}{l} x \leftarrow \mathcal{X}, \\ y \leftarrow \mathbb{Z}_P^m, \\ e \leftarrow [-\delta, +\delta]^m, \\ y' = y + e, \\ k' \leftarrow \mathbf{A}(x, y') \end{array} \right. \right] \\ &= \frac{1}{(2\delta + 1)^m} \cdot \sum_{e^* \in [-\delta, +\delta]^m} \mathbb{P} \left[F_{k'}(x) = y' - e^* \left| \begin{array}{l} x \leftarrow \mathcal{X}, \\ y' \leftarrow \mathbb{Z}_P^m, \\ k' \leftarrow \mathbf{A}(x, y') \end{array} \right. \right] \\ &= \frac{1}{(2\delta + 1)^m} \cdot \mathbb{P} \left[|F_{k'}(x) - y'| \leq \delta \left| \begin{array}{l} x \leftarrow \mathcal{X}, \\ y' \leftarrow \mathbb{Z}_P^m, \\ k' \leftarrow \mathbf{A}(x, y') \end{array} \right. \right]. \end{aligned}$$

This proves that for every $\mathbf{A}$ there exists in $\text{Hyb}^{(2)}$ such that $\Pr[\text{Hyb}^{(1)}(\mathbf{A}) = 1] = (2\delta + 1)^m \cdot \Pr[\text{Hyb}^{(2)}(\tilde{\mathbf{A}}) = 1]$.

Proof (Claim 3). This claim formalizes the observation that finding a key k' satisfying $F_{k'}(x) = y$ is easier than finding the exact key k used to compute y from x . To intuitively quantify how easier it is, we use the fact that there are in average $\frac{|\mathcal{K}|}{|\mathbb{Z}_P^m|}$ keys k' for every possible (x, y) . Therefore, finding a key such that $F_{k'}(x) = y$ is $\frac{|\mathcal{K}|}{|\mathbb{Z}_P^m|}$ times easier than finding the exact key used for generating (x, y) . Below, we prove this observation formally. By employing the law of total probability and Bayes' Theorem, we can rewrite $\Pr[\text{Hyb}^{(2)}(\mathbf{A}) = 1]$ as

$$\mathbb{P} \left[F_{k'}(x) = y \left| \begin{array}{l} x \leftarrow \mathcal{X}, \\ y \leftarrow \mathbb{Z}_P^m, \\ k' \leftarrow \mathbf{A}(x, y) \end{array} \right. \right]$$

$$\begin{aligned}
&= \sum_{(x^*, y^*) \in \mathcal{X} \times \mathbb{Z}_P^m} \mathbb{P} \left[\begin{array}{l} F_{k'}(x) = y, \\ x = x^*, \\ y = y^* \end{array} \middle| \begin{array}{l} x \leftarrow \mathcal{X}, \\ y \leftarrow \mathbb{Z}_P^m, \\ k' \leftarrow \mathbf{A}(x, y) \end{array} \right] \\
&= \sum_{(x^*, y^*) \in \mathcal{X} \times \mathbb{Z}_P^m} \mathbb{P} [F_{k'}(x^*) = y^* \mid k' \leftarrow \mathbf{A}(x^*, y^*)] \cdot \mathbb{P} \left[\begin{array}{l} x = x^*, \\ y = y^* \end{array} \middle| \begin{array}{l} x \leftarrow \mathcal{X}, \\ y \leftarrow \mathbb{Z}_P^m \end{array} \right] \\
&= \frac{1}{|\mathcal{X}| \cdot |\mathbb{Z}_P^m|} \cdot \sum_{(x^*, y^*) \in \mathcal{X} \times \mathbb{Z}_P^m} \mathbb{P} [F_{k'}(x^*) = y^* \mid k' \leftarrow \mathbf{A}(x^*, y^*)]. \tag{6}
\end{aligned}$$

In what follows, we show that $\frac{|\mathcal{K}|}{|\mathbb{Z}_P^m|} \Pr[\text{KR}_F(\mathbf{A}) = 1]$ equals [Eq. \(6\)](#) as well. First, we use the fact that $k' = k$ implies that $F_{k'}(x) = F_k(x)$

$$\begin{aligned}
\Pr[\text{KR}_F(\mathbf{A}) = 1] &= \mathbb{P} \left[\begin{array}{l} k' = k \\ k' \leftarrow \mathbf{A}(x, F_k(x)) \end{array} \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ x \leftarrow \mathcal{X}, \end{array} \right] \\
&= \mathbb{P} \left[\begin{array}{l} k' = k, \\ F_{k'}(x) = F_k(x) \end{array} \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ x \leftarrow \mathcal{X}, \\ k' \leftarrow \mathbf{A}(x, F_k(x)) \end{array} \right].
\end{aligned}$$

Next, we use both the law of total probabilities and Bayes' theorem, yielding

$$\begin{aligned}
\Pr[\text{KR}_F(\mathbf{A}) = 1] &= \frac{1}{|\mathcal{X}|} \sum_{x^* \in \mathcal{X}} \mathbb{P} \left[\begin{array}{l} k' = k, \\ F_{k'}(x^*) = F_k(x^*) \end{array} \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \\
&= \frac{1}{|\mathcal{X}|} \sum_{x^* \in \mathcal{X}} \mathbb{P} \left[\begin{array}{l} k' = k \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)) \end{array} \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ F_{k'}(x^*) = F_k(x^*) \end{array} \right] \\
&\quad \cdot \mathbb{P} \left[F_{k'}(x^*) = F_k(x^*) \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \\
&= \frac{1}{|\mathcal{X}|} \sum_{x^* \in \mathcal{X}} \mathbb{P} \left[F_{k'}(x^*) = F_k(x^*) \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \\
&\quad \cdot \sum_{y^* \in \mathbb{Z}_P^m} \mathbb{P} \left[\begin{array}{l} k' = k, \\ F_k(x^*) = y^* \end{array} \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)), \\ F_{k'}(x^*) = F_k(x^*) \end{array} \right] \\
&= \frac{1}{|\mathcal{X}|} \sum_{x^* \in \mathcal{X}} \mathbb{P} \left[F_{k'}(x^*) = F_k(x^*) \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \\
&\quad \cdot \sum_{y^* \in \mathbb{Z}_P^m} \mathbb{P} \left[\begin{array}{l} F_k(x^*) = y^* \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)) \end{array} \middle| \begin{array}{l} k \leftarrow \mathcal{K}, \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)), \\ F_{k'}(x^*) = F_k(x^*) \end{array} \right] \\
&\quad \cdot \mathbb{P} \left[\begin{array}{l} k' = k \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)) \\ F_{k'}(x^*) = F_k(x^*) \end{array} \middle| \begin{array}{l} k \leftarrow \{k : F_k(x^*) = y^*\}, \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)), \\ F_{k'}(x^*) = F_k(x^*) \end{array} \right]. \tag{7}
\end{aligned}$$

To conclude the proof, we analyze the probabilities appearing in [Eq. \(7\)](#). In the second probability, we can apply Bayes twice, yielding

$$\begin{aligned}
& \mathbb{P} \left[F_k(x^*) = y^* \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)), \\ F_{k'}(x^*) = F_k(x^*) \end{array} \right] \\
&= \mathbb{P} \left[\begin{array}{l} F_k(x^*) = y^*, \\ F_{k'}(x^*) = F_k(x^*) \end{array} \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \\
&\quad \cdot \left(\mathbb{P} \left[F_{k'}(x^*) = F_k(x^*) \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \right)^{-1} \\
&= \mathbb{P} \left[\begin{array}{l} F_{k'}(x^*) = F_k(x^*) \\ F_k(x^*) = y^* \end{array} \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \\
&\quad \cdot \mathbb{P} \left[F_k(x^*) = y^* \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \\
&\quad \cdot \left(\mathbb{P} \left[F_{k'}(x^*) = F_k(x^*) \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \right)^{-1}. \tag{8}
\end{aligned}$$

The first term of [Eq. \(8\)](#) can be restated as

$$\begin{aligned}
& \mathbb{P} \left[F_{k'}(x^*) = F_k(x^*) \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)), \\ F_k(x^*) = y^* \end{array} \right] \\
&= \mathbb{P} [F_{k'}(x^*) = y^* \mid k' \leftarrow_s \mathbf{A}(x^*, y^*)].
\end{aligned}$$

For the second term of [Eq. \(8\)](#), we observe that the probability depends uniquely on the structure of the weak PRF

$$\begin{aligned}
& \mathbb{P} \left[F_k(x^*) = y^* \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \\
&= \mathbb{P} [F_k(x^*) = y^* \mid k \leftarrow_s \mathcal{K}] \\
&= \frac{|\{k : F_k(x^*) = y^*\}|}{|\mathcal{K}|}.
\end{aligned}$$

For the last term of [Eq. \(8\)](#), we remark that it is independent of y^* and it's the inverse of the first term of [Eq. \(7\)](#). Summing up, the second probability in [Eq. \(7\)](#) can be rewritten as [Eq. \(8\)](#), which in turn equals

$$\begin{aligned}
& \mathbb{P} [F_{k'}(x^*) = y^* \mid k' \leftarrow_s \mathbf{A}(x^*, y^*)] \\
&\quad \cdot \frac{|\{k : F_k(x^*) = y^*\}|}{|\mathcal{K}|} \\
&\quad \cdot \left(\mathbb{P} \left[F_{k'}(x^*) = F_k(x^*) \mid \begin{array}{l} k \leftarrow_s \mathcal{K}, \\ k' \leftarrow_s \mathbf{A}(x^*, F_k(x^*)) \end{array} \right] \right)^{-1}.
\end{aligned}$$

In the last term of Eq. (7), we condition on the event where $F_{k'}(x^*) = F_k(x^*)$ and k' is chosen by $\mathbf{A}$ by only observing $(x^*, F_k(x^*))$. In this setting, $\mathbf{A}$ can only guess the exact key with uniform probability among the keys yielding the same output, *i.e.*

$$\mathbb{P} \left[k' = k, \begin{array}{l} k \leftarrow \mathcal{K} \\ k' \leftarrow \mathbf{A}(x^*, F_k(x^*)) \\ F_{k'}(x^*) = F_k(x^*) \end{array} \right] = \frac{1}{|\{k : F_k(x^*) = y^*\}|}.$$

Going back to Eq. (7), we get

$$\begin{aligned} \Pr[\text{KR}_F(\mathbf{A}) = 1] &= \frac{1}{|\mathcal{X}|} \sum_{(x^*, y^*) \in \mathcal{X} \times \mathbb{Z}_P^m} \mathbb{P}[F_{k'}(x^*) = y^* \mid k' \leftarrow \mathbf{A}(x^*, y^*)] \\ &\quad \cdot \frac{|\{k : F_k(x^*) = y^*\}|}{|\mathcal{K}|} \cdot \frac{1}{|\{k : F_k(x^*) = y^*\}|} \\ &= \frac{1}{|\mathcal{X}| \cdot |\mathcal{K}|} \sum_{(x^*, y^*) \in \mathcal{X} \times \mathbb{Z}_P^m} \mathbb{P}[F_{k'}(x^*) = y^* \mid k' \leftarrow \mathbf{A}(x^*, y^*)]. \end{aligned}$$

This proves the claim.

C.2 Proof of Lemma 2

To prove that F is a weak PRF, we must upper bound the following quantity for every quantum polynomial adversary $\mathbf{A}$:

$$|\mathbb{P}[\text{wPRF.Real}_F(\mathbf{A}) = 1] - \mathbb{P}[\text{wPRF.Rand}_F(\mathbf{A}) = 1]|,$$

where $\text{wPRF.Real}_F(\mathbf{A})$ and $\text{wPRF.Rand}_F(\mathbf{A})$ are the experiments defining the security of F as a weak PRF (*cf.* Section B.3). In what follows, we prove that

$$\text{wPRF.Real}_F(\mathbf{A}) \equiv \text{LWR}(Q, P, n, Nm). \text{Real}(\mathbf{k}) \quad (9)$$

and

$$\text{wPRF.Rand}_F(\mathbf{A}) \equiv \text{LWR}(Q, P, n, Nm). \text{Rand}(), \quad (10)$$

when $\mathbf{k} \leftarrow \mathbb{Z}_Q^n$. Note that Eq. (9) and Eq. (10) imply Lemma 2, as then

$$\begin{aligned} &|\mathbb{P}[\text{wPRF.Real}_F(\mathbf{A}) = 1] - \mathbb{P}[\text{wPRF.Rand}_F(\mathbf{A}) = 1]| \\ &= |\mathbb{P}[\mathbf{A}(\text{LWR}(\text{Real}(\mathbf{k})) = 1 \mid \mathbf{k} \leftarrow \mathbb{Z}_Q^n] - \mathbb{P}[\mathbf{A}(\text{LWR}(\text{Rand}()) = 1]| \\ &\leq \epsilon_{\text{LWR}}. \end{aligned}$$

Therefore, in the rest of the proof we show Eq. (9) and Eq. (10). For a better visualization, we report the experiments of interest in Fig. 11.

The only difference between the experiments wPRF and LWR (both in the Real and Rand case) is that the first one samples N small matrices $(\mathbf{X}_i)_{i \in [N]}$ in $\mathbb{Z}_Q^{m \times n}$, while the second directly samples a big matrix $\mathbf{X} \in \mathbb{Z}_Q^{Nm \times n}$. The key

$\text{wPRF.Real}_F(\mathbf{A})$	$\text{LWR}(Q, P, n, Nm).\text{Real}(\mathbf{k})$
$\mathbf{k} \leftarrow \mathbb{Z}_Q^n$ for $i \in [N]$ $\quad \mathbf{X}_i \leftarrow \mathbb{Z}_Q^{m \times n}$ $\quad \mathbf{y}_i \leftarrow \lfloor \mathbf{X}_i \mathbf{k} \rfloor_P$ $b \leftarrow \mathbf{A}((\mathbf{X}_i, \mathbf{y}_i)_{i \in [N]})$ return b	$\mathbf{X} \leftarrow \mathbb{Z}_Q^{Nm \times n}, \mathbf{y} \leftarrow \lfloor \mathbf{X} \mathbf{k} \rfloor_P$ return $(\mathbf{X}, \mathbf{y})$
$\text{wPRF.Rand}_F(\mathbf{A})$	$\text{LWR}(Q, P, n, Nm).\text{Rand}()$
$\mathbf{k} \leftarrow \mathbb{Z}_Q^n$ for $i \in [N]$ $\quad \mathbf{X}_i \leftarrow \mathbb{Z}_Q^{m \times n}$ $\quad \mathbf{y}_i \leftarrow \mathbb{Z}_P^m$ $b \leftarrow \mathbf{A}((\mathbf{X}_i, \mathbf{y}_i)_{i \in [N]})$ return b	$\mathbf{X} \leftarrow \mathbb{Z}_Q^{Nm \times n}, \mathbf{y} \leftarrow \mathbb{Z}_P^m$ return $(\mathbf{X}, \mathbf{y})$

Fig. 11. The experiments involved in the proof of [Lemma 2](#).

insight here is to think of $\mathbf{X}$ as the matrix where the small matrices $(\mathbf{X}_i)_{i \in [N]}$ are stacked one on top of the other, *i.e.*

$$\begin{pmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_N \end{pmatrix}.$$

Therefore, sampling $\mathbf{X}$ at random in $\mathbb{Z}_Q^{Nm \times n}$ is equivalent to sampling $(\mathbf{X}_i)_{i \in [N]}$ at random in $\mathbb{Z}_Q^{m \times n}$. In the **Rand** experiments, the same reasoning applies to $\mathbf{y}$ and $(\mathbf{y}_i)_{i \in [N]}$, and therefore [Eq. \(10\)](#) follows. To prove [Eq. \(9\)](#), we further observe that multiplying $\mathbf{X}$ by $\mathbf{k}$ is equivalent to multiplying $(\mathbf{X}_i)_{i \in [N]}$ by $\mathbf{k}$ individually and then stacking the results on top of each other, *i.e.*

$$\begin{pmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_N \end{pmatrix} \cdot (\mathbf{k}) = \begin{pmatrix} \mathbf{X}_1 \mathbf{k} \\ \mathbf{X}_2 \mathbf{k} \\ \vdots \\ \mathbf{X}_N \mathbf{k} \end{pmatrix}.$$

Since $(\mathbf{y}_i)_{i \in [N]}$ are computed by rounding $(\mathbf{X}_i \mathbf{k})_{i \in [N]}$, and since the rounding acts component-wise, then $\mathbf{y}$ is also the same as stacking $(\mathbf{y}_i)_{i \in [N]}$ on top of each other. This proves the claim.

C.3 Proof of Lemma 3

Lemma 1 guarantees that the PRF-based distribution $\mathcal{D}_{\mathcal{X} \times \mathcal{Y}}$ is a hard-instance generator for $\mathcal{R}$, with $\epsilon_{\text{Rel}} = 0$ and

$$\epsilon_{\text{HIG}} = \epsilon_{\text{wPRF}} + 3^m \cdot \frac{Q^n}{P^m} \cdot \epsilon_{\text{wKR}}.$$

Here ϵ_{wPRF} and ϵ_{wKR} correspond, respectively, to the pseudorandomness and the security against key recovery of F . According to Lemma 7, a $(N, \epsilon_{\text{wPRF}})$ -weak pseudorandom function with output space $\mathbb{Z}_P^m$ is also ϵ_{wKR} -secure against key recovery with $N = 2$, with $\epsilon_{\text{wKR}} \leq \epsilon_{\text{wPRF}} + \frac{1}{P^m N}$. According to Lemma 2, $\epsilon_{\text{wPRF}} \leq \epsilon_{\text{LWR}}$. The theorem follows.

C.4 Proof of Lemma 7

This lemma describes the standard observation that recovering the key of a PRF is harder than distinguishing it from random. For completeness, we report its adaptation to the weaker PRF notion we consider. Let $\mathbf{A}$ be any adversary in the game underlying ϵ_{wKR} , *i.e.* an algorithm that, upon receiving a couple $(x, y) \in \mathcal{X} \times \mathcal{Y}$, returns a candidate key k' . We now use $\mathbf{A}$ to construct an adversary $\tilde{\mathbf{A}}$ for the experiment corresponding to ϵ_{wPRF} . Upon input $\{x, y\}_{i \in [N]}$, $\tilde{\mathbf{A}}$ runs $\mathbf{A}$ over the first pair (x_1, y_1) . If the key k' returned by $\mathbf{A}$ satisfies $F_{k'}(x_i) = y_i$ for every $i \in [N]$, $\tilde{\mathbf{A}}$ outputs 1. Otherwise, the output is 0. This implies that

$$\begin{aligned} & \left| \mathbb{P} \left[\text{wPRF.Real}_F(\tilde{\mathbf{A}}) = 1 \right] - \mathbb{P} \left[\text{wPRF.Rand}_F(\tilde{\mathbf{A}}) = 1 \right] \right| \\ &= \left| \mathbb{P} \left[b = 1 \left| \begin{array}{l} k \leftarrow \mathcal{K}, \\ \{x_i\}_{i \in [N]} \leftarrow \mathcal{X}^N, \\ b \leftarrow \tilde{\mathbf{A}} \left(\{x_i, F_k(x_i)\}_{i \in [N]} \right) \end{array} \right. \right] - \mathbb{P} \left[b = 1 \left| \begin{array}{l} \{x_i\}_{i \in [N]} \leftarrow \mathcal{X}^N, \\ \{y_i\}_{i \in [N]} \leftarrow \mathcal{Y}^N, \\ b \leftarrow \tilde{\mathbf{A}} \left(\{x_i, y_i\}_{i \in [N]} \right) \end{array} \right. \right] \right| \\ &\geq \mathbb{P} \left[b = 1 \left| \begin{array}{l} k \leftarrow \mathcal{K}, \\ \{x_i\}_{i \in [N]} \leftarrow \mathcal{X}^N, \\ k' \leftarrow \mathbf{A}(x_1, F_k(x_1)), \\ k' = k, \\ b \leftarrow \tilde{\mathbf{A}} \left(\{x_i, F_k(x_i)\}_{i \in [N]} \right) \end{array} \right. \right] \cdot \mathbb{P} \left[k' = k \left| \begin{array}{l} k \leftarrow \mathcal{K}, \\ x_1 \leftarrow \mathcal{X}, \\ k' \leftarrow \mathbf{A}(x_1, F_k(x_1)) \end{array} \right. \right] \\ &\quad + \mathbb{P} \left[b = 1, k' \neq k \left| \begin{array}{l} k \leftarrow \mathcal{K}, \\ \{x_i\}_{i \in [N]} \leftarrow \mathcal{X}^N, \\ k' \leftarrow \mathbf{A}(x_1, F_k(x_1)), \\ b \leftarrow \tilde{\mathbf{A}} \left(\{x_i, F_k(x_i)\}_{i \in [N]} \right) \end{array} \right. \right] \\ &\quad - \mathbb{P} \left[b = 1 \left| \begin{array}{l} \{x_i\}_{i \in [N]} \leftarrow \mathcal{X}^N, \\ \{y_i\}_{i \in [N]} \leftarrow \mathcal{Y}^N, \\ k' \leftarrow \mathbf{A}(x_1, F_k(x_1)), \\ b \leftarrow \tilde{\mathbf{A}} \left(\{x_i, y_i\}_{i \in [N]} \right) \end{array} \right. \right] \end{aligned}$$

where the inequality above follows by the definition of conditional probability and the law of total probabilities. In the right-hand side of the equation, the first probability equals 1 by definition of $\tilde{A}$, the second is an arbitrary instance for $\text{KR}_F(A)$, and the third can be lower-bounded with 0. As for the last probability, $b = 1$ happens only when the randomly sampled elements $y_2, \dots, y_N$ hit exactly the evaluations of $F_{k'}$ over $x_2, \dots, x_N$, which happens with probability $\frac{1}{|\mathcal{Y}|^{N-1}}$. The lemma follows.

C.5 Proof of Theorem 2

The main purpose of this section is to show how Theorem 2 follows from the work of [47]. In particular, we restate some of their results using our notation, and include relations between parameters that they only state implicitly. Theorem 2 is directly implied by the two results below.

Theorem 12 (Theorem 25 of [47], restated, for $s = 2$ and $\epsilon_{\Sigma, \text{SS}} = 0$). *Let Σ be a Sigma protocol with challenge space size C , $(s, \epsilon_{\Sigma, \text{SS}})$ -special soundness with $s = 2$ and $\epsilon_{\Sigma, \text{SS}} = 0$, and ϵ_{QCUR} -quantum computationally unique responses. Then Σ is a computational proof of knowledge for adaptive adversaries as in Definition 14 of [47], with parameters $p = 1, d = 3$ and $\kappa = \frac{1}{C} + 2\epsilon_{\text{QCUR}}$.*

Theorem 13 (Theorem 21 of [47], restated, for $d = 3$). *Let Σ be a Sigma protocol for a relation $\mathcal{R}$ with challenge space size C , satisfying the adaptive proof of knowledge property with parameters $p, d = 3$, and κ as in Definition 14 of [47]. Let $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be a ϵ_{HIG} -hard instance generator yielding a pair of statements and witnesses in $\mathcal{R}$. Let Π_{Sign} be the signature scheme constructed from $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and Σ via the Fiat-Shamir with Abort transform (cf. Fig. 2). Then Π_{Sign} is $(\epsilon_{\text{Sign}, 0}, N_H)$ -unforgeable against no-message attack, with*

$$\epsilon_{\text{Sign}, 0} \leq \left(\epsilon_{\text{HIG}} N_H^6 p(\eta) + \kappa(\eta) + \frac{1}{p(\eta)} \cdot \left(\frac{1}{8N_H^3 C^3} + \frac{1}{\mathcal{O}(N_H^5 C)} \right) \right)^{1/3}$$

In the rest of this section, we relate Theorem 12 and Theorem 13 with the corresponding results in [47]. To do it, we also restate two of their intermediate results. We present them below.

Lemma 8 (Lemma 15 of [47], with parameters). *If Σ is a computational/statistical proof of knowledge for static A with parameters p, d, κ , then it is also a computational/statistical proof of knowledge for adaptive A with the same parameters (cf. Definition 14 of [47]).*

Corollary 1 (Corollary 16 of [47], with parameters, for $d = 3$). *Let Σ be a Sigma protocol with super polynomial challenge space size C . Let $\text{FS}(\Sigma)$ be the Fiat-Shamir-based non-interactive proof system described in Section 3.2 of [47]. If Σ is a computational/statistical proof of knowledge for static adversaries with parameters p, d, κ , with $d = 3$, then $\text{FS}(\Sigma)$ is a computational/statistical proof of knowledge for adaptive adversaries (cf. Definition 14 of [47]) with parameters*

p, d as in Σ , and $e = 6, \mu(\eta) = \kappa(\eta) + \frac{1}{p(\eta)} \cdot \left(\frac{1}{8N_H^3 C^3} + \frac{1}{\mathcal{O}(N_H^5)C} \right)$, where N_H is the number of queries to the random oracles made by the attacker to $\text{FS}(\Sigma)$.

First, we prove [Theorem 12](#) and [Theorem 13](#) based on [Lemma 8](#), [Corollary 1](#), and the corresponding results in [\[47\]](#).

Proof (Theorem 12). The statement of [Theorem 12](#) differs from the one of [Theorem 25](#) in [\[47\]](#) in the following points:

1. They assume t -soundness, while we assume $(s, \epsilon_{\Sigma, \text{SS}})$ -special soundness with $s = 2$ and $\epsilon_{\Sigma, \text{SS}} = 0$.
2. They claim that Sigma is a computational proof of knowledge, while we claim that it is a computational proof of knowledge *for adaptive adversaries*.
3. We include the parameters for the proof of knowledge property.

We discuss each of these points, starting from [Item 1](#). In [\[47\]](#) (section 5), t -soundness is defined as follows: "for any first message a , for uniformly random chosen challenges $c_1, \dots, c_t$, and for any responses $z_1, \dots, z_t$ with $V(x, a_i, c_i, z_i)$ for all $i \in \{1, \dots, t\}$, a witness w for x can be efficiently computed except with negligible probability (over the choices of c_i)". In other words, [Theorem 25](#) [\[47\]](#) works with a weaker version of $(s, \epsilon_{\Sigma, \text{SS}})$ -special soundness, where the challenges are random instead of adversarially chosen, and $\epsilon_{\Sigma, \text{SS}}$ is negligible. Therefore, its statements still holds under the stronger assumption of $(s, \epsilon_{\Sigma, \text{SS}})$ -special soundness. Moreover, since their result holds for any s and every negligible $\epsilon_{\Sigma, \text{SS}}$, it also holds for $s = 2$ and $\epsilon_{\Sigma, \text{SS}} = 0$.

We now move to [Item 2](#). In their [Theorem 25](#) [\[47\]](#), the authors do not mention whether they prove the computational proof of knowledge for static or adaptive adversaries. Nonetheless, their [Lemma 15](#) ([Lemma 8](#) in our work) guarantees that the two notions are equivalent. For readability, we here claim computational proof of knowledge for adaptive adversaries, while only proving it for static ones.

It remains to address [Item 3](#), and thus motivate the parameters. For this part, we refer to both the proof of [Theorem 25](#) in [\[47\]](#) and of [Theorem 9](#) in Unruh's work [\[80\]](#), as the first only provides a proof sketch, while the second offers a comprehensive analysis of a similar statement. Both works consider the canonical extractor (see [Section 3](#) of [\[80\]](#) for a detailed description), which works as follows. It runs the malicious prover A to the point where it outputs the first message firstmes . Then, it chooses a random challenge ch_1 and sends it to A , obtaining a response resp_1 by measuring A 's corresponding register. The extractor E rewinds A on the measured state, and chooses and sends to A a fresh random challenge ch_2 , resulting in a response resp_2 . If $\Sigma.\text{Verify}(x, \text{firstmes}, ch_s, \text{resp}_s) = 1$ for all $s \in \{1, 2\}$, then E can compute the witness w using the extractor E_{SS} corresponding to the special soundness property. Both proofs follow a hybrid argument, and we refer to the hybrid experiments [Game 1](#), $\dots$, [Game 7](#) defined in the proof of [Theorem 9](#) in [\[80\]](#). Below, we only report the most relevant hybrids to our proof.

Game 1. [Game 1](#) is the original experiment defining the proof of knowledge property (*cf.* [Definition 14](#) of [\[47\]](#)). the success event in [Game 1](#), denoted by Succ:Game 1 , describes the event where E outputs a valid witness w for x .

Game 2. Game 2 is identical to Game 1, except for the definition of the success event. Success Succ in Game 2 corresponds to the event where the transcripts $(\text{firstmes}, ch_1, \text{resp}_1)$ and $(\text{firstmes}, ch_2, \text{resp}_2)$ are accepting for x and $ch_1 \neq ch_2$.

Game 5. Game 5 is a variant of the above experiments where, before measuring the responses $\text{resp}_1, \text{resp}_2$, the extractor E measures whether the transcripts $(\text{firstmes}, ch_1, \text{resp}_1)$ and $(\text{firstmes}, ch_2, \text{resp}_2)$ yield an accepting conversation. Moreover, the extractor E doesn't run E_{SS} anymore. Here, Succ denotes the event where $ch_1 \neq ch_2$ and both verification measurements succeed.

Game 6. Game 6 is identical to Game 5, except that the measurements of $\text{resp}_1, \text{resp}_2$ are removed.

The goal of the proof is to lower bound $\Pr_E = \Pr[\text{Succ} : \text{Game 1}]$. Since special soundness holds with $\epsilon_{\Sigma, SS} = 0$, success in Game 2 always implies success in Game 1. Therefore, $\Pr_E = \Pr[\text{Succ} : \text{Game 1}] \geq \Pr[\text{Succ} : \text{Game 2}]$. For the same reasons explained in the proof of Theorem 9 [80], $\Pr[\text{Succ} : \text{Game 2}] = \Pr[\text{Succ} : \text{Game 5}]$. Notably, the only difference between Game 5 and 6 is that the second doesn't measure the values $\text{resp}_1, \text{resp}_2$ involved in the verification measurements. Since Σ has quantum computationally unique responses (cf. Definition 6), by union bound it holds $|\Pr[\text{Succ} : \text{Game 5}] - \Pr[\text{Succ} : \text{Game 6}]| \leq 2\epsilon_{\text{QCUR}}$. The rest is identical to Unruh's proof of Theorem 9. Namely,

$$\Pr[\text{Succ} : \text{Game 6}] \geq \Pr_V \left(\Pr_V^2 - \frac{1}{C} \right) \geq \Pr_V^3 - \frac{1}{C},$$

where $\Pr_V$ denotes the probability that A generates an accepting proof in the static setting. Putting things together, we get

$$\begin{aligned} \Pr_E &= \Pr[\text{Succ} : \text{Game 1}] \\ &\geq \Pr[\text{Succ} : \text{Game 2}] \\ &= \Pr[\text{Succ} : \text{Game 5}] \\ &\geq \Pr[\text{Succ} : \text{Game 6}] - |\Pr[\text{Succ} : \text{Game 5}] - \Pr[\text{Succ} : \text{Game 6}]| \\ &\geq \Pr_V^3 - \frac{1}{C} - 2\epsilon_{\text{QCUR}}. \end{aligned}$$

This concludes the proof.

Proof (Theorem 13). The statement of Theorem 13 differs from the one of Theorem 21 of [47] in the following points.

1. We assume that $d = 3$.
2. Theorem 13 assumes that $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ is a hard-instance generator, while Theorem 21 assumes a hard relation $\mathcal{R}$.
3. Theorem 13 assumes adaptive proofs of knowledge, while Theorem 21 does not specify the static/adaptive variant.
4. Theorem 13 refers to the Fiat-Shamir *with abort* transform (cf. Fig. 2), while Theorem 21 proves unforgeability for standard Fiat-Shamir signatures (cf. Definition 19 in [47]).

5. **Theorem 13** provides an explicit upper bound based on the underlying parameters.

In what follows, we address each of these points, starting from **Item 1**. Theorem 21 is true for every d , therefore the same holds for a specific choice of d . We now move to **Item 2**. Since $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ is a distribution yielding a pair of statements and witnesses in $\mathcal{R}$, assuming that $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ is a hard-instance generator (as in **Definition 7**) is the same as assuming that $\mathcal{R}$ is a hard relation (with generator $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$, as in Definition 18 of [47]). To motivate **Item 3**, we remark that **Lemma 8** guarantees that the static and adaptive notions are equivalent for proofs of knowledge. Regarding **Item 4**, we compare the unforgeability against no message attack of standard Fiat-Shamir signatures with the same property for Fiat-Shamir-with-Abort signatures. We observe that the first implies the second in the natural scenario where proofs with $\text{resp} = \perp$ are not accepted. In other words, the possibility of abort only reduces the probability that an adversary can generate an accepting signature. To conclude with **Item 5**, we discuss how the parameters of **Theorem 13** follow from the proof of Theorem 21. As a first step, Theorem 21 leverages **Corollary 1** to argue that $\text{FS}(\Sigma)$ is a proof of knowledge for adaptive adversaries, with parameters $p, d = 3$ as in Σ , and $e = 6, \mu(\eta) = \kappa(\eta) + \frac{1}{p(\eta)} \cdot \left(\frac{1}{8N_H^3 C^3} + \frac{1}{\mathcal{O}(N_H^5)C} \right)$. This implies their Equation 6. Taking the expectation in Equation 6 and using Jensen's inequality, they get the following:

$$\begin{aligned} \mathbb{E}_{x \leftarrow G} & \left[\left(\Pr [x' \| m \in X \wedge (x', w) \in R : (x' \| m, w) \leftarrow \mathcal{E}^A] q^e p(\eta) + \mu(\eta) \right)^{1/d} \right] \\ & \leq \left(\mathbb{E}_{x \leftarrow G} \Pr [x' \| m \in X \wedge (x', w) \in R : (x' \| m, w) \leftarrow \mathcal{E}^A] q^e p(\eta) + \mu(\eta) \right)^{1/d}. \end{aligned}$$

Notably, the expected value in the right-hand side corresponds to ϵ_{HIG} . The theorem follows by converting the proof according to our notation, and by replacing the values p, d, e, μ and κ .

We now conclude the analysis with the proofs of **Lemma 8** and **Corollary 1**.

Proof (Lemma 8). The only difference between **Lemma 8** and Lemma 15 in [47] is that the first makes the parameters explicit. Thus, it suffices to focus on the parameters and, in particular, to show that they remain unchanged. This can be deduced by the proof of Lemma 15 (Appendix A of [47]). The parameters p, d, κ appearing there come directly from the assumption that Σ is a proof of knowledge for static adversaries (used in the first inequality), and remain the same throughout the proof. This happens because neither Jensen's inequality nor the compact representation of the probability impact the parameters.

Proof (Corollary 1). For the same reason discussed in the proof of **Lemma 8**, we only need to motivate the relations between parameters. Again, these relations naturally emerge from the proof of Corollary 16 in [47]. From the last equation of their proof, one can readily deduce that p and d are the same as in Σ , while $e = 2d$. On the other hand, the description of μ stays implicit. In the rest of the

proof, we provide a description for μ in the special case where $d = 3$. We can rewrite their second last equation by expanding the cube of a binomial, *i.e.*

$$\begin{aligned} & \frac{1}{p(\eta)} \left(\frac{1}{\mathcal{O}(q^2)} \sum_{x_0 \in X} \Pr_H [x = x_0 \wedge V_{FS}^H(x, \pi) : (x, \pi) \leftarrow \mathcal{A}^H] - \frac{1}{2q|C|} \right)^3 - \kappa(\eta) \\ &= \frac{1}{p(\eta)\mathcal{O}(q^6)} \left(\sum_{x_0 \in X} \Pr_H [x = x_0 \wedge V_{FS}^H(x, \pi) : (x, \pi) \leftarrow \mathcal{A}^H] \right)^3 \\ & \quad - \frac{1}{p(\eta)\mathcal{O}(q^5)|C|} \left(\sum_{x_0 \in X} \Pr_H [x = x_0 \wedge V_{FS}^H(x, \pi) : (x, \pi) \leftarrow \mathcal{A}^H] \right)^2 \end{aligned} \quad (11)$$

$$\begin{aligned} & + \frac{1}{p(\eta)\mathcal{O}(q^4)|C|^2} \left(\sum_{x_0 \in X} \Pr_H [x = x_0 \wedge V_{FS}^H(x, \pi) : (x, \pi) \leftarrow \mathcal{A}^H] \right) \quad (12) \\ & - \frac{1}{8p(\eta)q^3|C|^3} - \kappa(\eta) \\ & \geq \frac{1}{p(\eta)\mathcal{O}(q^6)} \left(\sum_{x_0 \in X} \Pr_H [x = x_0 \wedge V_{FS}^H(x, \pi) : (x, \pi) \leftarrow \mathcal{A}^H] \right)^3 \\ & \quad - \left(\frac{1}{p(\eta)\mathcal{O}(q^5)|C|} + \frac{1}{8p(\eta)q^3|C|^3} + \kappa(\eta) \right), \end{aligned}$$

where we get the last inequality by ignoring the positive factor of [Eq. \(12\)](#) and by upper-bounding the probability in [Eq. \(11\)](#) with 1. Moving back to our notation, where q is replaced with N_H and $|C|$ is replaced with C , the corollary follows.

C.6 Proof of [Theorem 3](#)

We consider the same hybrid experiments as in [\[14\]](#), depicted in [Fig. 12](#). In particular, we use the fact, already proved in [\[14\]](#), that, for every quantum polynomial-time distinguisher D querying the signing oracle at most N_{Sign} times and the random oracle at most N_H times,

$$|\mathbb{P}[D^{\text{Sign}, H} = 1] - \mathbb{P}[D^{\text{Prog}, H}]| \leq \frac{2N_{\text{Sign}}\sqrt{\epsilon_{\text{ComGuess}}}}{1 - \epsilon_{\text{HAbort}}} \sqrt{N_H + \frac{N_{\text{Sign}}}{1 - \epsilon_{\text{HAbort}}}} \quad (13)$$

and

$$|\mathbb{P}[D^{\text{Prog}, H} = 1] - \mathbb{P}[D^{\text{Trans}, H}]| \leq 2N_H \sqrt{\frac{N_{\text{Sign}}\epsilon_{\text{ComGuess}}}{1 - \epsilon_{\text{HAbort}}}}, \quad (14)$$

for every quantum polynomial-time adversary A querying the random oracle at most N_H times, since these hybrids do not depend on the zero-knowledge property.

Lemma 9. *For every quantum polynomial-time adversary A querying the random oracle at most N_H times,*

$$|\mathbb{P}[A^{\text{Trans}, H} = 1] - \mathbb{P}[A^{\text{Sim}, H} = 1]| \leq \epsilon_{\Sigma.\text{HVZK}}.$$

Sign (msg) do $(firstmes, state) \leftarrow \Sigma.Prove_1(sk)$ $ch := H(firstmes, msg)$ $resp := \Sigma.Prove_2(firstmes, ch, state)$ until $resp \neq \perp$ return $(firstmes, resp)$	Trans (msg) do $(firstmes, state) \leftarrow \Sigma.Prove_1(sk)$ $ch \leftarrow \mathcal{N}_{ch}$ $resp := \Sigma.Prove_2(firstmes, ch, state)$ until $resp \neq \perp$ $H(firstmes, msg) := ch$ return $(firstmes, resp)$
Prog (msg) do $(firstmes, state) \leftarrow \Sigma.Prove_1(sk)$ $H(firstmes, msg) := ch \leftarrow \mathcal{N}_{ch}$ $resp := \Sigma.Prove_2(firstmes, ch, state)$ until $resp \neq \perp$ return $(firstmes, resp)$	Sim (msg) $(firstmes, ch, resp) \leftarrow ZKSim(pk)$ $H(firstmes, msg) := ch$ return $(firstmes, resp)$

Fig. 12. The hybrid experiments used in the proof of [Theorem 3](#).

Proof. We replace the oracles **Trans** and **Sim** with the oracles **Multi** $\langle\langle\mathbf{T}\rangle\rangle$ and **Multi** $\langle\langle\mathbf{S}\rangle\rangle$ depicted in [Fig. 13](#). Notice that the only difference between **Trans** and **Multi** $\langle\langle\mathbf{T}\rangle\rangle$ is syntactic. Indeed, in both experiments the challenge ch is sampled independently of the message msg , and then the random oracle is reprogrammed using $firstmes, msg$. The only difference is that **Trans** lazily samples $firstmes, resp$ during every query, while **Multi** $\langle\langle\mathbf{T}\rangle\rangle$ samples everything at the beginning; hence, **Trans** $\equiv$ **Multi** $\langle\langle\mathbf{T}\rangle\rangle$. With the same reasoning, **Sim** $\equiv$ **Multi** $\langle\langle\mathbf{S}\rangle\rangle$. Finally, the only difference between **Multi** $\langle\langle\mathbf{T}\rangle\rangle$ and **Multi** $\langle\langle\mathbf{S}\rangle\rangle$ lies in how the transcripts are sampled. Namely, **Multi** $\langle\langle\mathbf{T}\rangle\rangle$ samples the transcripts as in the real setting, while **Multi** $\langle\langle\mathbf{S}\rangle\rangle$ invokes the zero-knowledge simulator. An easy reduction shows that, for every quantum polynomial-time adversary A querying the random oracle at most N_H times,

$$\begin{aligned}
& \left| \mathbb{P} [A^{\mathbf{Trans}, H} = 1] - \mathbb{P} [A^{\mathbf{Sim}, H} = 1] \right| \\
&= \left| \mathbb{P} [A^{\mathbf{Multi}\langle\langle\mathbf{T}\rangle\rangle, H} = 1] - \mathbb{P} [A^{\mathbf{Multi}\langle\langle\mathbf{S}\rangle\rangle, H} = 1] \right| \\
&\leq \epsilon_{\Sigma.HVZK},
\end{aligned}$$

thus concluding the proof. $\square$

The next lemma is a simplified version of [[14](#), Lemma 1], in which we consider Γ to be the event that is always true.

$\mathbf{T}()$	$\mathbf{S}()$
for $\beta \in [N_{\text{Sign}}]$: do $(\text{firstmes}_\beta, \text{state}_\beta) \leftarrow \Sigma.\text{Prove}_1(sk)$ $ch_\beta \leftarrow \mathcal{N}_{\text{ch}}$ $\text{query} \leftarrow (\text{firstmes}_\beta, ch_\beta, \text{resp}_\beta)$ $\text{resp}_\beta := \Sigma.\text{Prove}_2(\text{query})$ until $\text{resp}_\beta \neq \perp$ return $(\text{firstmes}_\beta, \text{resp}_\beta)_{\beta \in [N_{\text{Sign}}]}$	$(\text{firstmes}_\beta, ch_\beta,$ $\quad \text{resp}_\beta)_{\beta \in [N_{\text{Sign}}]} \leftarrow \text{ZKSim}(pk)$ return $(\text{firstmes}_\beta, \text{resp}_\beta)_{\beta \in [N_{\text{Sign}}]}$ $\text{Multi}\langle\langle\mathcal{O}\rangle\rangle(msg)$ if $\beta = \text{undefined}$: $\beta \leftarrow 0$ $(\text{firstmes}_\beta, \text{resp}_\beta)_{\beta \in [N_{\text{Sign}}]} \leftarrow \mathcal{O}()$ $\beta \leftarrow \beta + 1$ $H(\text{firstmes}_\beta, msg) := ch_\beta$ return $(\text{firstmes}_\beta, \text{resp}_\beta)$

Fig. 13. The oracles $\text{Multi}\langle\langle\mathbf{T}\rangle\rangle$ and $\text{Multi}\langle\langle\mathbf{S}\rangle\rangle$ that compute every transcript at the beginning instead of on request by the adversary.

Lemma 10 (Lemma 1 of [14], adapted to Multi-HVZK and $\Gamma = \text{true}$).

Let $\epsilon_{\text{ComGuess}} \geq 0$ and $\epsilon_{\text{HAbort}} < 1$. Let $\mathbf{A}_{N_{\text{Sign}}, N_H}^{\text{Sign}, H}$ be a quantum polynomial-time adversary CMA attacker against Σ that makes at most N_{Sign} queries to the signing oracle $\mathbf{Sign}$ and N_H queries to the random oracle H . Then, there exists a quantum polynomial-time adversary NMA attacker $\mathbf{B}^H$ against Σ and a pair of quantum polynomial-time distinguishers $\mathbf{D}^{\cdot, H}, \mathbf{D}'^{\cdot, H}$,

$$\begin{aligned}
\mathbb{P}[\mathbf{A}_{N_{\text{Sign}}, N_H}^{\text{Sign}, H} \text{ wins}] &\leq \mathbb{P}[\mathbf{B}^H \text{ wins}] \\
&\quad + \left| \mathbb{P}[\mathbf{D}_{N_{\text{Sign}}, N_H+1}^{\text{Sign}, H} = 1] - \mathbb{P}[\mathbf{D}_{N_{\text{Sign}}, N_H+1}^{\text{Prog}, H} = 1] \right| \\
&\quad + \left| \mathbb{P}[\mathbf{D}'_{N_{\text{Sign}}, N_H+1}^{\text{Prog}, H} = 1] - \mathbb{P}[\mathbf{D}'_{N_{\text{Sign}}, N_H+1}^{\text{Trans}, H} = 1] \right| \\
&\quad + \epsilon_{\Sigma, \text{HVZK}}.
\end{aligned}$$

Proof (sketch). As in the original proof, we consider an adversary $\tilde{\mathbf{A}}$ that, upon input the public key vk internally runs $\mathbf{A}$, verifies the correctness of the produced forgery and outputs 1 if and only if it is a valid forgery (and 0 otherwise). Since $\tilde{\mathbf{A}}$ requires an additional oracle query for the verification, the amount of oracle queries from $\tilde{\mathbf{A}}$ is at most $N_H + 1$ and, moreover,

$$\mathbb{P}[\mathbf{A}_{N_{\text{Sign}}, N_H}^{\text{Sign}, H} \text{ wins}] = \mathbb{E} \left[\mathbb{P}[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}^{\text{Sign}, H} = 1] \right].$$

We can upper bound the guessing probability of $\tilde{\mathbf{A}}$ with

$$\mathbb{P}[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}^{\text{Sign}, H} = 1] \leq \mathbb{P}[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}^{\text{Sim}, H} = 1]$$

$$\begin{aligned}
& + \left| \mathbb{P} \left[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}^{\text{Sign}, H} = 1 \right] - \mathbb{P} \left[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}^{\text{Prog}, H} = 1 \right] \right| \\
& + \left| \mathbb{P} \left[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}'^{\text{Prog}, H} = 1 \right] - \mathbb{P} \left[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}'^{\text{Trans}, H} = 1 \right] \right| \\
& + \left| \mathbb{P} \left[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}'^{\text{Trans}, H} = 1 \right] - \mathbb{P} \left[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}'^{\text{Sim}, H} = 1 \right] \right|
\end{aligned}$$

and, by applying [Lemma 9](#) and averaging over the choice of vk , there exists a pair of quantum polynomial-time distinguishers $\mathbf{D}^{\cdot, H}, \mathbf{D}'^{\cdot, H}$ such that

$$\begin{aligned}
\mathbb{P} \left[\mathbf{A}_{N_{\text{Sign}}, N_H}^{\text{Sign}, H} \text{ wins} \right] & \leq \mathbb{P} \left[\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}^{\text{Sim}, H} = 1 \right] \\
& + \left| \mathbb{P} \left[\mathbf{D}_{N_{\text{Sign}}, N_H+1}^{\text{Sign}, H} = 1 \right] - \mathbb{P} \left[\mathbf{D}_{N_{\text{Sign}}, N_H+1}^{\text{Prog}, H} = 1 \right] \right| \\
& + \left| \mathbb{P} \left[\mathbf{D}_{N_{\text{Sign}}, N_H+1}'^{\text{Prog}, H} = 1 \right] - \mathbb{P} \left[\mathbf{D}_{N_{\text{Sign}}, N_H+1}'^{\text{Trans}, H} = 1 \right] \right| \\
& + \epsilon_{\Sigma, \text{HVZK}}.
\end{aligned}$$

Recall that, for a random key pair (vk, sk) , $\tilde{\mathbf{A}}_{N_{\text{Sign}}, N_H+1}^{\text{Sim}, H}$ runs $\mathbf{A}$ against the CMA game in which all the signing queries are answered by **Sim** instead of **Sign**, which means that no signature queries are actually performed. This implies that there exists an adversary $\mathbf{B}_{N_{\text{Sign}}, N_H}^H$ with the same success probability against the NMA game. The lemma follows. $\square$

Finally, [Theorem 3](#) follows by combining [Eqs. \(13\) and \(14\)](#) and [Lemma 10](#).

C.7 Proof of [Theorem 9](#)

Our scheme meets all the assumptions of [Theorem 1](#), with

$$\begin{aligned}
\epsilon_{\Sigma, \text{Comp}} & = tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}}, \\
\epsilon_{\text{HAbort}} & = tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}}, \\
\nu & = -\log \left[\left(2\epsilon_{\text{Coll}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|} \right) + \frac{1}{|\mathcal{P}_{\mathcal{K}}|} \right)^{(d-1)t} \right],
\end{aligned}$$

according to, respectively, [Theorem 4](#), [Theorem 14](#), and [Theorem 8](#). By plugging these parameters into [Theorem 1](#), [Theorem 9](#) follows.

C.8 Proof of [Theorem 10](#)

Combining the results of Fehr *et al.* [47] and Barbosa *et al.* [14] (reported in [Section 3.6](#)), we get

$$\begin{aligned}
\epsilon_{\text{Sign}, N_{\text{Sign}}} & \leq \left(\epsilon_{\text{HIG}} \cdot N_H^6 + \frac{1}{C} + 2\epsilon_{\text{QCUR}} + \frac{1}{O(N_H^5) \cdot C} + \frac{1}{8N_H^3 C^3} \right)^{\frac{1}{3}} \\
& + \frac{2N_{\text{Sign}} \sqrt{\epsilon_{\text{ComGuess}}}}{1 - \epsilon_{\text{HAbort}}} \sqrt{N_H + 1 + \frac{N_{\text{Sign}}}{1 - \epsilon_{\text{HAbort}}}}
\end{aligned}$$

$$+ 2(N_H + 1) \sqrt{\frac{N_{\text{Sign}} \cdot \epsilon_{\text{ComGuess}}}{1 - \epsilon_{\text{HAbort}}}} + N_{\text{Sign}} \epsilon_{\Sigma.\text{HVZK}}.$$

From [Lemma 3](#), we know that

$$\epsilon_{\text{HIG}} \leq \epsilon_{\text{LWR}}^{(p-\alpha)} \left(1 + 3^m \frac{2^{qn}}{2^{(p-\alpha)m}} \right) + 3^m \frac{2^{qn}}{2^{2qnN}}.$$

Our sigma protocol has challenge size $C = d^t$, and from [Section 5](#) we derive the parameters

$$\begin{aligned} \epsilon_{\text{HAbort}} &\leq tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}}, \\ \epsilon_{\Sigma.\text{HVZK}} &= Nt\epsilon_{\text{Hide}} + \frac{2Nt\epsilon_{\text{PRG}} + 4\epsilon_{\text{LWR}}}{1 - N \cdot t \cdot m \cdot 2^{-\alpha}} + 2\gamma^\omega, \\ \epsilon_{\text{QCUR}} &\leq t(d-1) \cdot \epsilon_{\text{Coll}}, \\ \epsilon_{\text{ComGuess}} &\leq \left(2\epsilon_{\text{Coll}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|} \right) + \frac{1}{|\mathcal{P}_{\mathcal{K}}|} \right)^{(d-1)t}. \end{aligned}$$

The lemma follows.

D Properties of the Rounding Function

In this section, we introduce simple properties of the rounding function that we use throughout our proofs.

Proposition 1 (Generic properties of the rounding function). *Let Q, P be integers such that $Q > P$, and let $x \in \mathbb{Z}_Q$. Then the following properties hold*

1. *Let $P' \in \mathbb{N}$ such that $P' | P$. Moreover, assume that $P | Q$. Then*

$$\lfloor x \rfloor_{P'} = \lfloor \lfloor x \rfloor_P \rfloor_{P'}.$$

2. *If $P | Q$, then*

$$x = \frac{Q}{P} \lfloor x \rfloor_P + \left(x \bmod \frac{Q}{P} \right).$$

3. *Let $x' \in \mathbb{Z}_Q$, and assume that $P | Q$. Then*

$$\lfloor x + x' \rfloor_P - \lfloor x \rfloor_P - \lfloor x' \rfloor_P \in \{0, 1\}.$$

Proposition 2 (Properties of the rounding functions over bit strings). *Let q, p be integers such that $q > p$. Then, the following properties hold.*

1. (MSB notation) *For all $x \in \mathbb{Z}_{2^q}$,*

$$\lfloor x \rfloor_{2^p} = x[(q-p) \dots q].$$

2. (LSB notation) For all $x \in \mathbb{Z}_{2^q}$,

$$2^{q-p} \lfloor x \rfloor_{2^p} = x - x[0..(q-p)],$$

or, equivalently,

$$x = 2^{q-p} \lfloor x \rfloor_{2^p} + x[0..(q-p)].$$

3. (Slices of slices equivalence) Let $i, j, k, h \in \mathbb{N}$ be such that $k < h \leq q$ and $i < j \leq h - k$. Then, for all $x \in \mathbb{Z}_{2^q}$,

$$x[k..h][i] = x[k+i]$$

and

$$x[k..h][i..j] = x[(k+i)..(k+j)].$$

4. (Monotonicity of rounding) Let $i, j \in \mathbb{N}$ be such that $i < j \leq q$. Then, for all $u, v \in \mathbb{Z}_{2^q}$,

- If $u[i..j] < v[i..j]$, then $u[0..j] < v[0..j]$.
- If $u[i..j] > v[i..j]$, then $u[0..j] > v[0..j]$.

In what follows, we report the proofs of both propositions.

Proof (Proposition 1). We start with the proof of Item 1. We first show the inequality $\lfloor \lfloor x \rfloor_P \rfloor_{P'} \leq \lfloor x \rfloor_{P'}$, and then the same in the opposite direction, *i.e.* $\lfloor x \rfloor_{P'} \leq \lfloor \lfloor x \rfloor_P \rfloor_{P'}$, which implies that the equality holds. We start with the first inequality:

$$\lfloor \lfloor x \rfloor_P \rfloor_{P'} = \left\lfloor \left\lfloor x \cdot \frac{P}{Q} \right\rfloor \cdot \frac{P'}{P} \right\rfloor \leq \left\lfloor x \cdot \frac{P}{Q} \cdot \frac{P'}{P} \right\rfloor = \left\lfloor x \cdot \frac{P'}{Q} \right\rfloor = \lfloor x \rfloor_{P'}.$$

We now prove the second inequality. First, we remark that $P'|P$ and $P|Q$ imply that there exist c_1, c_2, c_3 such that $\frac{Q}{P'} = c_1$, $\frac{P}{P'} = c_2$, $\frac{Q}{P} = c_3$ and $c_1 = c_2 \cdot c_3$. Now, consider the Euclidean division of x by c_1 , and name the quotient M_1 and the remainder R_1 , *i.e.* $x = M_1 \cdot c_1 + R_1$ with $M_1, R_1 \in \mathbb{N}$ and $R_1 < c_1$. Then

$$\lfloor x \rfloor_{P'} = \left\lfloor \frac{M_1 \cdot c_1 + R_1}{c_1} \right\rfloor = M_1, \tag{15}$$

and

$$\begin{aligned} \lfloor \lfloor x \rfloor_P \rfloor_{P'} &= \left\lfloor \left\lfloor \frac{M_1 \cdot c_1 + R_1}{c_3} \right\rfloor \cdot \frac{1}{c_2} \right\rfloor \\ &\geq \left\lfloor \left\lfloor \frac{M_1 \cdot c_1}{c_3} \right\rfloor \cdot \frac{1}{c_2} \right\rfloor \\ &= \left\lfloor \lfloor M_1 \cdot c_2 \rfloor \cdot \frac{1}{c_2} \right\rfloor = \left\lfloor M_1 \cdot c_2 \cdot \frac{1}{c_2} \right\rfloor = M_1 \end{aligned}$$

$$= \lfloor x \rfloor_{P'} . \quad (16)$$

where Eq. (16) follows from Eq. (15). This proves Item 1.

We now prove Item 2. Consider the Euclidean division of x by $\frac{Q}{P}$, and name M the quotient and R the rest, *i.e.* $x = M \cdot \frac{Q}{P} + R$ with $M, R \in \mathbb{N}$ and $R < \frac{Q}{P}$. Then

$$\begin{aligned} x \bmod \frac{Q}{P} &= R, \\ \lfloor x \rfloor_P &= \left\lfloor M \cdot \frac{Q}{P} \cdot \frac{P}{Q} + R \cdot \frac{P}{Q} \right\rfloor = M + \left\lfloor R \cdot \frac{P}{Q} \right\rfloor = M. \end{aligned}$$

To conclude, we prove Item 3. According to Item 2, we can rewrite

$$x + x' = \frac{Q}{P} \lfloor x \rfloor_P + \frac{Q}{P} \lfloor x' \rfloor_P + \left(x \bmod \frac{Q}{P} \right) + \left(x' \bmod \frac{Q}{P} \right).$$

By applying the rounding on both sides, we get

$$\begin{aligned} \lfloor x + x' \rfloor_P &= \left\lfloor \frac{Q}{P} \lfloor x \rfloor_P + \frac{Q}{P} \lfloor x' \rfloor_P + \left(x \bmod \frac{Q}{P} \right) + \left(x' \bmod \frac{Q}{P} \right) \right\rfloor_P \\ &= \lfloor x \rfloor_P + \lfloor x' \rfloor_P + \left\lfloor \left(x \bmod \frac{Q}{P} \right) + \left(x' \bmod \frac{Q}{P} \right) \right\rfloor_P \end{aligned} \quad (17)$$

where Eq. (17) follows from the definition of rounding and from the fact that $\lfloor x \rfloor_P$ and $\lfloor x' \rfloor_P$ are integers. To conclude the proof, it suffices to show that

$$\left\lfloor \left(x \bmod \frac{Q}{P} \right) + \left(x' \bmod \frac{Q}{P} \right) \right\rfloor_P \in \{0, 1\}.$$

This is true because every $x, x' \in \mathbb{Z}_Q$ satisfies

$$0 \leq \left(x \bmod \frac{Q}{P} \right), \left(x' \bmod \frac{Q}{P} \right) \leq \frac{Q}{P} - 1,$$

and therefore

$$0 \leq \left(x \bmod \frac{Q}{P} \right) + \left(x' \bmod \frac{Q}{P} \right) \leq 2\frac{Q}{P} - 2. \quad (18)$$

Moving to the rounding, Eq. (18) yields

$$0 \leq \left\lfloor \left(x \bmod \frac{Q}{P} \right) + \left(x' \bmod \frac{Q}{P} \right) \right\rfloor_P \leq \left\lfloor 2\frac{Q}{P} - 2 \right\rfloor_P \leq 1. \quad (19)$$

This concludes the proof.

Proof (Proposition 2). We start with Item 1. We rewrite $\lfloor x \rfloor_{2^p}$ as

$$\lfloor x \rfloor_{2^p} = \left\lfloor \sum_{i=0}^{q-1} 2^i x[i] \right\rfloor_{2^p} \quad (\text{binary representation}),$$

$$\begin{aligned}
&= \left\lfloor \sum_{i=0}^{q-1} 2^i x[i] \cdot \frac{2^p}{2^q} \right\rfloor && \text{(by definition of rounding),} \\
&= \left\lfloor \sum_{i=0}^{q-1} 2^{i-(q-p)} x[i] \right\rfloor && \text{(simplifying the fraction),} \\
&= \sum_{i=q-p}^{q-1} 2^{i-(q-p)} x[i] && \text{(rounding in } \mathbb{Z} \text{),} \\
&= x[(q-p)..q] && \text{(binary representation),}
\end{aligned}$$

where the rounding in $\mathbb{Z}$ happens because

$$\sum_{i=q-p}^{q-1} 2^{i-(q-p)} x[i]$$

is an integer and

$$\sum_{i=0}^{q-p-1} 2^{i-(q-p)} x[i] < 1.$$

We now move to **Item 2**. We rewrite $x - x[0..(q-p)]$ as

$$\begin{aligned}
x - x[0..(q-p)] &= \sum_{i=0}^{q-1} 2^i x[i] - \sum_{i=0}^{q-p-1} 2^i x[i] && \text{(binary representation),} \\
&= \sum_{i=q-p}^{q-1} 2^i x[i] && \text{(simplifying the sum),} \\
&= 2^{q-p} \sum_{i=q-p}^{q-1} 2^{i-q+p} x[i] && \text{(moving } 2^{q-p} \text{ outside),} \\
&= 2^{q-p} x[(q-p)..q] && \text{(binary representation),} \\
&= 2^{q-p} \lfloor x \rfloor_{2^p} && \text{(by Item 1).}
\end{aligned}$$

Next, we prove **Item 3**. We can rewrite $x[k..h][i]$ as

$$x[k..h][i] = \left(\sum_{\eta=k}^{h-1} 2^{\eta-k} x[\eta] \right) [i] = x[\eta^*]$$

for η^* such that $\eta^* - k = i$, hence $x[\eta^*] = x[k+i]$, which proves the first equality. Now we can use this to prove the second equality:

$$x[k..h][i..j] = \sum_{\eta=i}^{j-1} 2^{\eta-i} x[k..h][\eta] \quad \text{(binary representation),}$$

$$\begin{aligned}
&= \sum_{\eta=i}^{j-1} 2^{\eta-i} x[k+\eta] && \text{(by the first equality),} \\
&= \sum_{\eta=k+i}^{k+j-1} 2^{\eta-k-i} x[\eta] && \text{(reparametrization of the sum index),} \\
&= x[(k+i)..(k+j)] && \text{(binary representation).}
\end{aligned}$$

We now conclude with the proof of [Item 4](#). We only prove the first statement, since the proof for the second one is identical. First, we rewrite $2^i u[i..j]$ as

$$\begin{aligned}
2^i u[i..j] &= 2^i u[0..j][i..j] && \text{(by [Item 3](#)),} \\
&= 2^i \lfloor u[0..j] \rfloor_{2^{j-i}} && \text{(by [Item 1](#)),} \\
&= u[0..j] - u[0..i] && \text{(by [Item 2](#)),}
\end{aligned}$$

and, similarly, $2^i v[i..j] = v[0..j] - v[0..i]$. Now, assume that $u[i..j] < v[i..j]$ or, since both are integers, $u[i..j] + 1 \leq 2^i v[i..j]$. Then,

$$\begin{aligned}
u[0..j] &= u[0..j] - u[0..i] + u[0..i] && \text{(by adding and subtracting } u[0..i]) \\
&= 2^i u[i..j] + u[0..i] && \text{(by applying the above)} \\
&< 2^i u[i..j] + 2^i && \text{(by } u[0..i] < 2^i) \\
&= 2^i (u[i..j] + 1) && \text{(distributive property)} \\
&\leq 2^i v[i..j] && \text{(by } u[i..j] + 1 \leq 2^i v[i..j]) \\
&= v[0..j] - v[0..i] && \text{(by applying the above again)} \\
&\leq v[0..j] && \text{(by } v[0..i] \geq 0).
\end{aligned}$$

E Security of the Sigma Protocol in [Section 5](#)

In this section, we formally prove all the properties of the sigma protocol. Before that, we need to introduce a preliminary result.

Theorem 14. *Let Σ be the Sigma protocol from [Fig. 3](#), and $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ be the distribution described in [Fig. 4](#). Assume that decisional LWR holds with parameters $(2^q, 2^p, n, m)$ and advantage ϵ_{LWR} , and that $G_{\mathcal{K}}$ is a $\epsilon_{\text{PRG}, \mathcal{K}}$ -secure PRG. Then, the probability ϵ_{HAbort} that **Abort** outputs 1 in an honest execution of Σ satisfies*

$$\epsilon_{\text{HAbort}} \leq tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}}.$$

We prove the above via a hybrid argument. We report the experiments of interest in [Fig. 14](#), and shortly summarize them below.

HonestAbort: This experiment describes the event where **Abort** outputs 1 during an honest execution of the protocol. Notably, this experiment ignores the response and verification, as we are only interested in the output of **Abort**.

HonestAbort $((\mathbf{X}, \mathbf{y}), \mathbf{k}) \leftarrow \mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ $(\text{firstmes}, \text{state}) \leftarrow \Sigma.\text{Prove}_1((\mathbf{X}, \mathbf{y}), \mathbf{k})$ $ch \leftarrow \mathcal{N}_{\text{ch}}$ if $\text{Abort}(\text{state}, ch)$ return 1 return 0	$\Sigma.\text{Prove}'_1((\mathbf{X}, \mathbf{y}), \mathbf{k}, ch)$ $(\mathcal{I}^{(j)})_{j \in [t]} \leftarrow ch$ for $j \in [t]$ $(\mathbf{k}_1^{(j)}, \dots, \mathbf{k}_d^{(j)}) \leftarrow \text{Share}(\mathbf{k})$ for $i \in [d]$ $\mathbf{z}_i^{(j)} \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i^{(j)}, i)$ $sum^{(j)} \leftarrow \sum_{i \in \mathcal{I}^{(j)}} \mathbf{z}_i^{(j)}$ $\mathbf{y}_{\text{ext}} \leftarrow \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^p}$ $\text{state}' \leftarrow (\mathbf{y}_{\text{ext}}, (sum^{(j)})_{j \in [t]})$ return state'
Hyb⁽¹⁾ $((\mathbf{X}, \mathbf{y}), \mathbf{k}) \leftarrow \mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ $ch \leftarrow \mathcal{N}_{\text{ch}}$ $\text{state}' \leftarrow \Sigma.\text{Prove}'_1((\mathbf{X}, \mathbf{y}), \mathbf{k}, ch)$ if $\text{Abort}'(\text{state}')$ return 1 return 0	$\Sigma.\text{Prove}''_{1,j^*}((\mathbf{X}, \mathbf{y}), \mathbf{k}, ch)$ $(\mathcal{I}^{(j)})_{j \in [t]} \leftarrow ch$ for $j \in [t]$ if $j \leq j^*$ $\tilde{\mathbf{k}}^{(j)} \leftarrow \mathbb{Z}_Q^n$ $sum^{(j)} \leftarrow \mathbf{X}\tilde{\mathbf{k}}^{(j)}$ else $(\mathbf{k}_1^{(j)}, \dots, \mathbf{k}_d^{(j)}) \leftarrow \text{Share}(\mathbf{k})$ for $i \in [d]$ $\mathbf{z}_i^{(j)} \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i^{(j)}, i)$ $sum^{(j)} \leftarrow \sum_{i \in \mathcal{I}^{(j)}} \mathbf{z}_i^{(j)}$ $\mathbf{y}_{\text{ext}} \leftarrow \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^p}$ $\text{state}' \leftarrow (\mathbf{y}_{\text{ext}}, (sum^{(j)})_{j \in [t]})$ return state'
Hyb⁽²⁾ $((\mathbf{X}, \mathbf{y}), \mathbf{k}) \leftarrow \mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ $ch \leftarrow \mathcal{N}_{\text{ch}}$ $\text{state}' \leftarrow \Sigma.\text{Prove}''_{1,j^*}((\mathbf{X}, \mathbf{y}), \mathbf{k}, ch)$ if $\text{Abort}'(\text{state}')$ return 1 return 0	$\text{Abort}'(\text{state}')$ $(\mathbf{y}_{\text{ext}}, (sum^{(j)})_{j \in [t]}) \leftarrow \text{state}'$ for $j \in [t]$ if $0 \in sum^{(j)}[r_S..r_E] - \mathbf{y}_{\text{ext}}[0..\alpha]$ return 1 return 0
Hyb⁽³⁾ $((\mathbf{X}, \mathbf{y}), \mathbf{k}) \leftarrow \mathcal{D}'_{\mathcal{X} \times \mathcal{W}}$ $ch \leftarrow \mathcal{N}_{\text{ch}}$ $\text{state}' \leftarrow \Sigma.\text{Prove}''_{1,t}((\mathbf{X}, \mathbf{y}), \mathbf{k}, ch)$ if $\text{Abort}'(\text{state}')$ return 1 return 0	$\mathcal{D}'_{\mathcal{X} \times \mathcal{W}}$ $\mathbf{k} \leftarrow \mathbb{Z}_{2^q}^n$ $\mathbf{X} \leftarrow \mathbb{Z}_Q^{m \times n}$ $\mathbf{y} \leftarrow \mathbb{Z}_{2^{p-\alpha}}^m$ return $((\mathbf{X}, \mathbf{y}), \mathbf{k})$

Fig. 14. The hybrid experiments for the proof of Theorem 14.

- Hyb⁽¹⁾**: This is a variant of **HonestAbort** where we tailor the prover's description to only compute the values determining the abort condition. Specifically, we consider an alternative prover $\Sigma.\text{Prove}'_1$ that, instead of passing to the abort algorithm a state $state$ including all the key shares $(\mathbf{k}_i^{(j)})_{i,j}$ and the matrix $\mathbf{X}$, only passes a state $state'$ containing the sums $(sum^{(j)})_{j \in [t]}$. Since this value depends on the challenge ch , $\Sigma.\text{Prove}'_1$ takes it as an extra input. Accordingly, we define an alternative algorithm, **Abort'**, which no longer computes the values $(sum^{(j)})_{j \in [t]}$ internally, but instead takes them directly as part of the input.
- (Hyb⁽²⁾)_{j*}**: These experiments are identical to **Hyb⁽¹⁾**, except that the first j^* partial sums are generated in a key-independent way.
- Hyb⁽³⁾**: This is a variant of **Hyb⁽²⁾** where statement-witness pair is generated with an alternative algorithm $\mathcal{D}'_{\mathcal{X} \times \mathcal{W}}$. The only difference between $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and $\mathcal{D}'_{\mathcal{X} \times \mathcal{W}}$ is that the latter samples $\mathbf{y}$ at random, independently of $\mathbf{X}$ and $\mathbf{k}$.

The proof proceeds via a sequence of hybrid arguments, and follows from the claims below.

Claim 4 $\text{HonestAbort} \equiv \text{Hyb}^{(1)}$.

Proof (Claim 4). The main observation of this proof is that the output of **Abort** only depends on the extended output $\mathbf{y}_{ext}$ and the partial sums $(sum^{(j)})_j$. Therefore, omitting the computation of the rest does not affect the output of **Abort**. Aside from this, the only difference between **HonestAbort** and **Hyb⁽¹⁾** is the order of computation. However, since the information used by **Abort** stays the same, so does the output. The claim follows.

Claim 5 $\text{Hyb}^{(1)} \equiv \text{Hyb}_0^{(2)}$.

Proof (Claim 5). This is true because $\Sigma.\text{Prove}'_1$ and $\Sigma.\text{Prove}''_{1,j^*}$ are the same algorithm when $j^* = 0$.

Claim 6 Assume that $G_{\mathcal{K}}$ is a $\epsilon_{\text{PRG}, \mathcal{K}}$ -secure pseudorandom generator. Then

$$\mathbb{P} \left[\text{Hyb}_{j^*-1}^{(2)} = 1 \right] - \mathbb{P} \left[\text{Hyb}_{j^*}^{(2)} = 1 \right] \leq \epsilon_{\text{PRG}, \mathcal{K}}.$$

Proof (Claim 6). This proof works via a reduction to the pseudorandomness of $G_{\mathcal{K}}$. The only difference between the two experiments concerns the partial sum $sum^{(j^*)}$. In the first case, it is computed as

$$sum^{(j^*)} = \sum_{i \in \mathcal{I}^{(j^*)}} z_i^{(j^*)} = \mathbf{X} \left(\sum_{i \in [d-1] \cap \mathcal{I}^{(j^*)}} G_{\mathcal{K}}(\mathbf{k}_i) + \sum_{i \in \{d\} \cap \mathcal{I}^{(j^*)}} \mathbf{k}_i \right), \quad (20)$$

whereas in the second, it's just the multiplication of $\mathbf{X}$ with a fresh random vector $\tilde{\mathbf{k}}^{(j^*)}$. The main remark here is that $sum^{(j^*)}$ in **Hyb⁽²⁾_{j*-1}** is distributed

either as the shifted multiplication of $\mathbf{X}$ with a pseudorandom value, or as the multiplication of $\mathbf{X}$ with a sum of pseudorandom values. Moreover, this can be determined based on $\mathcal{I}^{(j^*)}$. In the next few lines, we make this observation formal. First, assume that $d \in \mathcal{I}^{(j^*)}$. Then we can use the description of $\mathbf{k}_d^{(j)}$ in Eq. (20) and get

$$\begin{aligned} \text{sum}^{(j^*)} &= \mathbf{X} \left(\sum_{i \in [d-1] \cap \mathcal{I}^{(j^*)}} G_{\mathcal{K}}(\mathbf{k}_i) + \mathbf{k}_d \right) \\ &= \mathbf{X} \left(\sum_{i \in [d-1] \cap \mathcal{I}^{(j^*)}} G_{\mathcal{K}}(\mathbf{k}_i) + \mathbf{k} - \sum_{i \in [d-1]} G_{\mathcal{K}}(\mathbf{k}_i) \right) \\ &= -\mathbf{X} G_{\mathcal{K}}(\mathbf{k}_{i^*}) + \mathbf{X} \mathbf{k}, \end{aligned} \quad (21)$$

where i^* is defined as $[t] \setminus \mathcal{I}^{(j)}$. Now, consider the case where $d \notin \mathcal{I}^{(j)}$. Then

$$\text{sum}^{(j^*)} = \mathbf{X} \sum_{i \in [d-1]} G_{\mathcal{K}}(\mathbf{k}_i). \quad (22)$$

With this intuition in mind, we can construct a reduction to the pseudorandomness of $G_{\mathcal{K}}$. We consider a reduction $\mathbf{A}_{\text{PRG}}$ that takes as input a value $\tilde{\mathbf{k}} \in \mathbb{Z}_Q^n$, and behaves as in Fig. 15. In a nutshell, $\mathbf{A}_{\text{PRG}}$ does the same as both hybrids in every iteration $j \neq j^*$, and the key point is the way it samples $\text{sum}^{(j^*)}$. If $d \in \mathcal{I}^{(j^*)}$, then $\text{sum}^{(j)} = \mathbf{X} \tilde{\mathbf{k}}$, and otherwise $\text{sum}^{(j)} = \mathbf{X} \left(\tilde{\mathbf{k}} + \sum_{i \in [d-2]} G_{\mathcal{K}}(\mathbf{k}_i^{(j)}) \right)$. Notably, if $\tilde{\mathbf{k}}$ is pseudorandom, this yields a perfect simulation of $\text{Hyb}_{j^*-1}^{(2)}$ according to Eq. (21) and Eq. (22). On the other hand, if $\tilde{\mathbf{k}}$ is purely random, then both ways of computing $\text{sum}^{(j^*)}$ correspond to the multiplication of $\mathbf{X}$ with a random vector. This is because in both cases $\text{sum}^{(j)} = \mathbf{X} \tilde{\mathbf{k}}'$, where $\tilde{\mathbf{k}}'$ is the sum of a random vector with independent objects, and therefore is still random. This proves that $\mathbf{A}_{\text{PRG}}$ perfectly simulates both hybrids, and proves the claim.

Claim 7 *Assume that decisional LWR holds with parameters $(2^q, 2^p, n, m)$ and advantage ϵ_{LWR} . Then*

$$\mathbb{P} \left[\text{Hyb}_t^{(2)} = 1 \right] - \mathbb{P} \left[\text{Hyb}^{(3)} = 1 \right] \leq \epsilon_{\text{LWR}}.$$

Proof (Claim 7). This proof is a reduction to the one-time pseudorandomness of the wPRF F defined in Section 4.1. We can reduce to the wPRF F for three main reasons. First, because $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ and $\mathcal{D}'_{\mathcal{X} \times \mathcal{W}}$ generate the statement and witness, respectively, as the real and ideal experiments for F generate their input, output, and key. Second, because the Abort' condition does not depend on the key $\mathbf{k}$ directly, but only on the extended wPRF output $\mathbf{y}_{\text{ext}} \in \mathbb{Z}_{2^p}^m$ and the partial sums $(\text{sum}^{(j)})_j$. Finally, because $\Sigma.\text{Prove}_{1,t}''$ generates the partial sums independently of $\mathbf{k}$. In other words, the only input to Abort' which could be related to $\mathbf{k}$ is $\mathbf{y}_{\text{ext}}$, which either equals $\lfloor \mathbf{X} \mathbf{k} \rfloor_{2^p}$, or is an independent random

```

APRG( $\tilde{\mathbf{k}}$ )
(( $\mathbf{X}, \mathbf{y}$ ),  $\mathbf{k}$ )  $\leftarrow$   $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ ,
for  $j \in [t]$ 
   $i^* \leftarrow [d]$ 
   $\mathcal{I}^{(j)} \leftarrow [d] \setminus \{i^*\}$ 
  if  $j \leq j^* - 1$ 
     $\tilde{\mathbf{k}}^{(j)} \leftarrow \mathbb{Z}_Q^n$ 
     $sum^{(j)} \leftarrow \mathbf{X} \tilde{\mathbf{k}}^{(j)}$ 
  if  $j = j^*$ 
    if  $d \in \mathcal{I}^{(j)}$ 
       $sum^{(j)} \leftarrow \mathbf{X}(\mathbf{k} - \tilde{\mathbf{k}})$ 
    else
       $sum^{(j)} \leftarrow \mathbf{X} \left( \tilde{\mathbf{k}} + \sum_{i \in [d-2]} G_{\mathcal{K}}(\mathbf{k}_i^{(j)}) \right)$ 
  if  $j \geq j^* + 1$ 
     $(\mathbf{k}_1^{(j)}, \dots, \mathbf{k}_d^{(j)}) \leftarrow \text{Share}(\mathbf{k})$ 
    for  $i \in [d]$ 
       $\mathbf{z}_i^{(j)} \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i^{(j)}, i)$ 
     $sum^{(j)} \leftarrow \sum_{i \in \mathcal{I}^{(j)}} \mathbf{z}_i^{(j)}$ 
 $\mathbf{y}_{ext} \leftarrow \lfloor \mathbf{X} \mathbf{k} \rfloor_{2^p}$ 
 $state' \leftarrow (\mathbf{y}_{ext}, (sum^{(j)})_{j \in [t]})$ 
if Abort'( $state'$ )
  return 1
return 0

```

Fig. 15. The reduction to the PRG $G_{\mathcal{K}}$ of [Claim 6](#)

string. For these reasons, even though the statement includes a wPRF output $\mathbf{y}$ with $p - \alpha$ bits, we reduce to a wPRF with a p -bits output. Our reduction A_{wPRF} to the one-time security of F takes as input a matrix $\mathbf{X}$ (*i.e.* the input to the PRF) and a p -bits output $\mathbf{y}_{\text{ext}}$, and behaves as in Fig. 16. In short, A_{wPRF}

$A_{\text{wPRF}}(\mathbf{X}, \mathbf{y}_{\text{ext}})$ for $j \in [t]$ $i^* \leftarrow \$ [d]$ $\mathcal{I}^{(j)} \leftarrow [d] \setminus \{i^*\}$ $\tilde{\mathbf{k}}^{(j)} \leftarrow \$ \mathbb{Z}_Q^n$ $sum^{(j)} \leftarrow \mathbf{X} \tilde{\mathbf{k}}^{(j)}$ $ch \leftarrow (\mathcal{I}^{(j)})_{j \in [t]}$ $state' \leftarrow (\mathbf{y}_{\text{ext}}, (sum^{(j)})_{j \in [t]})$ if $\text{Abort}'(state')$ $\quad$ return 1 return 0

Fig. 16. The reduction to the wPRF F of Claim 7

samples $(sum^{(j)})_{j \in [t]}$ as in $\text{Hyb}_t^{(2)}$ and $\text{Hyb}^{(3)}$, and feeds its input $\mathbf{y}_{\text{ext}}$ as input to Abort' too. This guarantees that when $\mathbf{y}_{\text{ext}}$ equals $\lfloor \mathbf{X} \mathbf{k} \rfloor_{2^p} = F_{\mathbf{k}}(\mathbf{X})$, then $A_{\text{wPRF}}(\mathbf{X}, \mathbf{y}_{\text{ext}})$ perfectly simulates $\text{Hyb}_t^{(2)}$; whereas when $\mathbf{y}_{\text{ext}}$ is random, then $A_{\text{wPRF}}(\mathbf{X}, \mathbf{y}_{\text{ext}})$ perfectly simulates $\text{Hyb}^{(3)}$. More formally, this proves that

$$\begin{aligned} \mathbb{P}[\text{wPRF.Real}_F(A_{\text{wPRF}}) = 1] &= \mathbb{P}[\text{Hyb}_t^{(2)} = 1] \\ \mathbb{P}[\text{wPRF.Rand}_F(A_{\text{wPRF}}) = 1] &= \mathbb{P}[\text{Hyb}^{(3)} = 1]. \end{aligned}$$

This concludes the proof.

Claim 8 $\mathbb{P}[\text{Hyb}^{(3)} = 1] \leq tm \cdot 2^{-\alpha}$

Proof (Claim 8). Define **Event** as the event where **Abort** outputs 1 in $\text{Hyb}^{(3)}$. Additionally, define Event_j as the event where the j -th iteration triggers the abort, meaning that $sum^{(j)}[r_s..r_E]$ and $\mathbf{y}_{\text{ext}}[0..\alpha]$ have at least one equal component. Thus, we have $\text{Event} = \bigcup_{i \in [t]} \text{Event}_j$. Moreover, $\mathbb{P}[\text{Event}_j] \leq m \cdot 2^{-\alpha}$, as in every iteration j of $\text{Hyb}^{(3)}$, **Abort** compares component-wise two independent α -bit strings, one of which is uniformly random. By union bound,

$$\mathbb{P}[\text{Hyb}^{(3)} = 1] = \mathbb{P}[\text{Event}] \leq \sum_{i \in [t]} \mathbb{P}[\text{Event}_j] \leq tm \cdot 2^{-\alpha}.$$

This proves the claim.

To conclude the proof of [Theorem 14](#), we now combine [Claim 4](#), [5](#), [6](#), [7](#), and [8](#). Adding and subtracting the terms corresponding to all the hybrids, we get

$$\begin{aligned}
\mathbb{P}[\text{HonestAbort} = 1] &= \left(\mathbb{P}[\text{HonestAbort} = 1] - \mathbb{P}[\text{Hyb}^{(1)} = 1] \right) \\
&\quad + \left(\mathbb{P}[\text{Hyb}^{(1)} = 1] - \mathbb{P}[\text{Hyb}_0^{(2)} = 1] \right) \\
&\quad + \sum_{j^* \in [t]} \left(\mathbb{P}[\text{Hyb}_{j^*}^{(2)} = 1] - \mathbb{P}[\text{Hyb}_{j^*-1}^{(2)} = 1] \right) \\
&\quad + \left(\mathbb{P}[\text{Hyb}_t^{(2)} = 1] - \mathbb{P}[\text{Hyb}^{(3)} = 1] \right) \\
&\quad + \mathbb{P}[\text{Hyb}^{(3)} = 1] \\
&\leq t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}} + tm \cdot 2^{-\alpha}.
\end{aligned}$$

E.1 Completeness ([Theorem 4](#))

In the first part of the proof, we use [Theorem 14](#) to upper bound the probability that **Abort** outputs 1. Then, we use the almost homomorphic properties of the rounding function to prove that, if **Abort** outputs 0, then honestly generated proofs are always accepted. Consider the event $\text{Event} = \{\text{Abort}(\text{state}, \text{ch}) = 0\}$ in $\Sigma.\text{Comp}_\Sigma$. Then,

$$\begin{aligned}
\mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0] \\
&= \mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0, \text{Event}] + \mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0, \neg\text{Event}] \tag{23}
\end{aligned}$$

$$\leq \mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0, \text{Event}] + \mathbb{P}[\neg\text{Event}] \tag{24}$$

$$\leq \mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0, \text{Event}] + tm \cdot 2^{-\alpha} + t \cdot \epsilon_{\text{PRG}, \mathcal{K}} + \epsilon_{\text{LWR}}, \tag{25}$$

where [Eq. \(23\)](#) follows from the law of total probabilities, [Eq. \(24\)](#) follows from the observation that the joint probability is always smaller than the corresponding marginals, and [Eq. \(25\)](#) is true by [Theorem 14](#). In what follows, we prove that

$$\mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0, \text{Abort}(\text{state}, \text{ch}) = 0] = 0. \tag{26}$$

According to [Eq. \(25\)](#), this implies the theorem. If $\mathbb{P}[\text{Abort}(\text{state}, \text{ch}) = 0] = 0$, then [Eq. \(26\)](#) is true. Therefore, it suffices to prove [Eq. \(26\)](#) under the assumption that $\mathbb{P}[\text{Abort}(\text{state}, \text{ch}) = 0] > 0$. Via Bayes' Theorem, we get

$$\begin{aligned}
&\mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0, \text{Abort}(\text{state}, \text{ch}) = 0] \\
&= \mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0 | \text{Abort}(\text{state}, \text{ch}) = 0] \cdot \mathbb{P}[\text{Abort}(\text{state}, \text{ch}) = 0] \\
&\leq \mathbb{P}[\Sigma.\text{Comp}_\Sigma = 0 | \text{Abort}(\text{state}, \text{ch}) = 0], \tag{27}
\end{aligned}$$

where the latter probability is well defined because $\mathbb{P}[\text{Abort}(\text{state}, \text{ch}) = 0] > 0$. In what follows, we prove that [Eq. \(27\)](#) equals 0. The experiment $\Sigma.\text{Comp}_\Sigma$ outputs 0 only if at least one of these conditions is met:

1. $((\mathbf{X}, \mathbf{y}), \mathbf{k}) \notin \mathcal{R}$.
2. $resp = \perp$.
3. There exists $j \in [t], i \in \mathcal{I}^{(j)}$ such that $H(\mathbf{k}_i^{(j)}) \neq h_i^{(j)}$.
4. There exists $j \in [t], i \in \mathcal{I}^{(j)}$ such that $\lfloor \mathbf{z}_i^{(j)} \rfloor_{2^{p-\alpha}} \neq \mathbf{y}_i^{(j)}$.
5. There exists $j \in [t]$ such that $\mathbf{y} - \tilde{\mathbf{y}}^{(j)} \notin \{0, 1\}^m$, where $\tilde{\mathbf{y}}^{(j)} = \lfloor sum^{(j)} \rfloor_{2^{p-\alpha}} + \mathbf{y}_{i^*}^{(j)}$.

Item 1 is never true because the experiment $\Sigma.\text{Comp}_\Sigma$ generates $((\mathbf{X}, \mathbf{y}), \mathbf{k})$ with $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$, which only outputs statement-witness pairs in $\mathcal{R}$. **Item 2** cannot happen either, as **Eq. (27)** is under the condition that the prover does not abort. **Item 3** and **Item 4** are impossible as well in $\Sigma.\text{Comp}_\Sigma$, because $\Sigma.\text{Prove}_1$ generates $h_i^{(j)}$ and $\mathbf{y}_i^{(j)}$ precisely as $H(\mathbf{k}_i^{(j)})$ and $\lfloor \mathbf{z}_i^{(j)} \rfloor_{2^{p-\alpha}}$, respectively. To conclude the proof, it remains to prove **Item 5**. The latter argument combines the fact that the key is (pseudorandomly) additively shared with the almost homomorphic behaviour of the rounding function (**Proposition 1**, **Item 3**). For every $j \in [t]$, the experiment $\Sigma.\text{Comp}_\Sigma$ computes $\mathbf{y} - \tilde{\mathbf{y}}^{(j)}$ as follows

$$\begin{aligned}
& \mathbf{y} - \tilde{\mathbf{y}}^{(j)} \\
&= \mathbf{y} - \lfloor sum^{(j)} \rfloor_{2^{p-\alpha}} - \mathbf{y}_{i^*}^{(j)} \\
&= \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^{p-\alpha}} - \left[\sum_{i \in \mathcal{I}^{(j)}} \text{Output}(\mathbf{X}, \mathbf{k}_i, i) \right]_{2^{p-\alpha}} - \lfloor \text{Output}(\mathbf{X}, \mathbf{k}_{i^*}, i^*) \rfloor_{2^{p-\alpha}}. \quad (28)
\end{aligned}$$

We now want to use the almost homomorphic properties of the rounding, and rewrite **Eq. (28)** as $\lfloor \mathbf{x} + \mathbf{x}' \rfloor_{2^{p-\alpha}} - \lfloor \mathbf{x} \rfloor_{2^{p-\alpha}} - \lfloor \mathbf{x}' \rfloor_{2^{p-\alpha}}$. This would conclude the proof, as **Proposition 1**, **Item 3** guarantees that the latter always lies in $\{0, 1\}^m$. We define

$$\mathbf{x} = \sum_{i \in \mathcal{I}^{(j)}} \text{Output}(\mathbf{X}, \mathbf{k}_i, i), \quad \mathbf{x}' = \text{Output}(\mathbf{X}, \mathbf{k}_{i^*}, i^*).$$

Therefore, the lemma follows if we show that

$$\mathbf{x} + \mathbf{x}' = \mathbf{X}\mathbf{k}. \quad (29)$$

Here, we use the fact that the key is (pseudorandomly) additively shared. More formally, since $(\mathbf{k}_i)_{i \in [d]}$ is generated with $\text{Share}(\mathbf{k})$, we know that

$$\mathbf{k} = \sum_{i \in [d-1]} G_{\mathcal{K}}(\mathbf{k}_i) + \mathbf{k}_d. \quad (30)$$

Therefore, we can rewrite the right-hand side of **Eq. (29)** as

$$\mathbf{X}\mathbf{k} = \sum_{i \in [d-1]} \mathbf{X}G_{\mathcal{K}}(\mathbf{k}_i) + \mathbf{X}\mathbf{k}_d \quad (31)$$

$$= \sum_{i \in [d]} \text{Output}(\mathbf{X}, \mathbf{k}_i, i) \quad (32)$$

$$= \sum_{i \in \mathcal{I}^{(j)}} \text{Output}(\mathbf{X}, \mathbf{k}_i, i) + \text{Output}(\mathbf{X}, \mathbf{k}_{i^*}, i^*) \quad (33)$$

$$= \mathbf{x} + \mathbf{x}'. \quad (34)$$

where Eq. (31) uses Eq. (30) and the linearity of the matrix product, Eq. (32) uses the definition of **Output** and groups the summands together, Eq. (33) splits the summands isolating the index i^* , and Eq. (34) uses the definition of $\mathbf{x}$ and $\mathbf{x}'$. This proves Eq. (29), and therefore the theorem.

E.2 Average-Case Accepting Multi-HVZK (Theorem 5)

The proof follows by a sequence of hybrid experiments, moving from the real output to the simulated output. We denote by **Success** the event of a successful computation (i.e., the algorithm **Abort** outputs 0), and we denote by **Abort** the opposite event.

Before proceeding with the actual proof, we define some auxiliary tools. For pairs $(\beta, j), (\beta', j') \in [N] \times [t]$, we define the lexicographic relation $\leq_{\text{lex}}$, defined as

$$(\beta, j) \leq_{\text{lex}} (\beta', j') \iff \beta < \beta' \vee (\beta = \beta' \wedge j \leq j').$$

We define in Fig. 17 some helper functions that will make the proof easier to follow, and we summarize them below.

- A “batch” hash function $\mathbf{H}\langle\beta^*, j^*\rangle$ that, intuitively, uses $\mathbf{H}$ to compute the hash values of the list of keys provided as input. Additionally, given a set of indices i^* , $\mathbf{H}\langle\beta^*, j^*\rangle$ replaces some of the keys (depending on the parameters β^* and j^*) indexed by i^* and produces hash values of random keys instead.
- A “batch” sharing algorithm $\text{Share}\langle\beta^*, j^*\rangle$ that runs Share Nt times and produces a secret sharing of either the input value $\mathbf{k}$ or of a uniformly random key $\mathbf{k}^* \leftarrow \mathbb{Z}_Q^n$, depending on the parameters β^* and j^* .
- A “batch” output algorithm $\text{Output}\langle b \rangle$ that computes **Output** on every key share, except, when $b = 1$, for the ones indexed by i^* , for which the output value is computed by a special function **Calc**, described below.
- A function **Calc** that computes the share $\mathbf{y}_{i^*}$ given the computation of $\mathbf{z}_i \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i, i)$ for all the $i \neq i^*$, and the extended output $\mathbf{y}_{\text{ext}}$. Looking ahead, this computation does not always output the same as computing $\mathbf{y}_{i^*}$ directly as $\lfloor \mathbf{z}_i \rfloor_{2^{p-\alpha}}$; however, the cases in which the two outputs differ are the cases in which the **Abort** happens, which is never the case if we condition to **Success**.

For the sake of completeness, the descriptions of both the real experiment and the simulator are depicted in Fig. 18, in which we made explicit the steps of $\Sigma.\text{Prove}_1$. In particular, the view of the adversary in the real setting is given by the distribution

$$\text{Real}(\mathcal{D}_{\mathcal{X} \times \mathcal{W}}),$$

$H\langle\langle\beta^*, j^*\rangle\rangle(\text{keys}, i^*)$ where $\beta^* \in [N], j^* \in [t]$ $(\mathbf{k}_i^{(\beta,j)}, \rho_i^{(\beta,j)})_{(\beta,j,i) \in [N] \times [t] \times [d]} \leftarrow \text{keys}$ for $(\beta, j, i) \in [N] \times [t] \times [d]$: if $(\beta, j) \leq_{\text{lex}} (\beta^*, j^*) \vee i \neq i_{\beta,j}^*$: $h_i^{(\beta,j)} \leftarrow H(\mathbf{k}_i^{(\beta,j)}, \rho_i^{(\beta,j)})$ else : $\mathbf{k}^* \leftarrow \$_Z \mathbb{Z}_Q^n$ $h_i^{(\beta,j)} \leftarrow \$_H(\mathbf{k}^*)$ return $(h_i^{(\beta,j)})_{(\beta,j,i) \in [N] \times [t] \times [d]}$	$\text{Output}\langle\langle b \rangle\rangle(\mathbf{X}, \text{keys}, \mathbf{y}_{\text{ext}}, i^*)$ where $b \in \{0, 1\}$ $(\mathbf{k}_i^{(\beta,j)}, \rho_i^{(\beta,j)})_{(\beta,j,i) \in [N] \times [t] \times [d]} \leftarrow \text{keys}$ for $(\beta, j) \in [N] \times [t]$: for $i \in [d] \setminus \{b \cdot i_{\beta,j}^*\}$: $\mathbf{z}_i \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i, i)$ $\mathbf{y}_i^{(\beta,j)} \leftarrow \lfloor \mathbf{z}_i \rfloor_{2^{p-\alpha}}$ if $b = 1$: $\mathbf{y}_{i_{\beta,j}^*}^{(\beta,j)} \leftarrow \text{Calc}\left((\mathbf{z}_i)_{i \neq i_{\beta,j}^*}, \mathbf{y}_{\text{ext}}\right)$ return $(\mathbf{y}_i^{(\beta,j)})_{(\beta,j,i) \in [N] \times [t] \times [d]}$
$\text{Share}\langle\langle\beta^*, j^*\rangle\rangle(\mathbf{k})$ where $\beta^* \in [N], j^* \in [t]$ for $(\beta, j, i) \in [N] \times [t] \times [d]$: if $(\beta, j) \leq_{\text{lex}} (\beta^*, j^*)$: $(\mathbf{k}_i^{(\beta,j)})_{i \in [d]} \leftarrow \$_{\text{Share}}(\mathbf{k})$ else : $\mathbf{k}^* \leftarrow \$_Z \mathbb{Z}_Q^n$ $(\mathbf{k}_i^{(\beta,j)})_{i \in [d]} \leftarrow \$_{\text{Share}}(\mathbf{k}^*)$ return $(\mathbf{k}_i^{(\beta,j)})_{(\beta,j,i) \in [N] \times [t] \times [d]}$	$\text{Calc}\left((\mathbf{z}_i)_{i \neq i^*}, \mathbf{y}_{\text{ext}}\right)$ $\text{sum} \leftarrow \sum_{i \neq i^*} \mathbf{z}_i$ $\text{LHS} \leftarrow \mathbf{1}^m \parallel \mathbf{y}_{\text{ext}}[0..\alpha]$ $\text{RHS} \leftarrow \mathbf{0}^m \parallel \text{sum}[r_S..r_E]$ $\mathbf{c} \leftarrow \mathbf{1}^m - \lfloor \text{LHS} - \text{RHS} \rfloor_2$ $\mathbf{y}_{i^*} \leftarrow \lfloor \mathbf{y}_{\text{ext}} \rfloor_{2^{p-\alpha}} - \lfloor \text{sum} \rfloor_{2^{p-\alpha}} - \mathbf{c}$ return $\mathbf{y}_{i^*}$

Fig. 17. The helper functions. The order relation $\leq_{\text{lex}}$ is defined as $(\beta, j) \leq_{\text{lex}} (\beta^*, j^*)$ whenever $\beta < \beta^*$ or $\beta = \beta^*$ and $j \leq j^*$.

while the simulated view is output by

$$\text{Sim}_\omega(\mathcal{D}_{\mathcal{X} \times \mathcal{W}}).$$

The simulator tries ω times to produce a list of valid simulated transcripts, after which gives up and outputs $\perp$. In particular, for $\omega = 1$, only a single attempt is executed, which makes Sim_ω computationally close to **Real** only when conditioning on the event **Success** for both of them. We will show that, by increasing ω , the probability that Sim_ω outputs $\perp$ rapidly decreases, making the simulator efficient in expected runtime. Furthermore, for $\omega \rightarrow \infty$, the probability of $\perp$ goes to zero.

$\text{Real}((\mathbf{X}, \mathbf{y}), \mathbf{k})$	$\text{Sim}_\omega(\mathbf{X}, \mathbf{y})$
$\mathbf{y}_{ext} \leftarrow \lfloor \mathbf{X} \mathbf{k} \rfloor_{2^p}$ for $\beta \in [N]$: for $j \in [t]$: $(\mathbf{k}_1^{(\beta,j)}, \dots, \mathbf{k}_d^{(\beta,j)}) \leftarrow \text{Share}(\mathbf{k})$ for $i \in [d]$ $\mathbf{z}_i^{(\beta,j)} \leftarrow \text{Output}(\mathbf{X}, \mathbf{k}_i^{(\beta,j)}, i)$ $\mathbf{y}_i^{(\beta,j)} \leftarrow \lfloor \mathbf{z}_i^{(\beta,j)} \rfloor_{2^{p-\alpha}}$ $\rho_i^{(\beta,j)} \leftarrow \mathcal{P}_H$ $h_i^{(\beta,j)} \leftarrow H(\mathbf{k}_i^{(\beta,j)}, \rho_i^{(\beta,j)})$ $\text{firstmes}_\beta \leftarrow (\mathbf{y}_i^{(\beta,j)}, h_i^{(\beta,j)})_{i \in [d], j \in [t]}$ $ch_\beta := (\mathcal{I}^{(\beta,j)})_{j \in [t]} \leftarrow \mathcal{N}_{ch}$ $\text{keys}_\beta \leftarrow (\mathbf{k}_i^{(\beta,j)}, \rho_i^{(\beta,j)})_{(i \in [d], j \in [t])}$ $\text{state}_\beta \leftarrow (\mathbf{X}, \mathbf{y}_{ext}, \text{keys}_\beta)$ $\text{resp}_\beta \leftarrow \Sigma.\text{Prove}_2(\text{state}_\beta, ch_\beta)$ $\text{tr}_\beta = (\text{firstmes}_\beta, ch_\beta, \text{resp}_\beta)$ return $(\mathbf{X}, \mathbf{y}, (\text{tr}_\beta)_{\beta \in [N]})$	for $\cdot \in [\omega]$: $\mathbf{r} \leftarrow \mathbb{Z}_{2^\alpha}^n$ $\mathbf{y}_{ext} \leftarrow 2^\alpha \mathbf{y} + \mathbf{r}$ $i^* \leftarrow \mathbb{S} [d]^{N \times t}$ $(ch_\beta)_{\beta \in [N]} := ([d] \setminus \{i_{\beta,j}^*\})_{\beta \in [N], j \in [t]}$ $\text{keys} \leftarrow \text{Share}(\langle 0, 0 \rangle)(\mathbf{k})$ $\text{outs} \leftarrow \text{Output}(\langle 1 \rangle)(\mathbf{X}, \text{keys}, \mathbf{y}_{ext}, i^*)$ $\text{hashes} \leftarrow H(\langle 0, 0 \rangle)(\text{keys}, i^*)$ for $\beta \in [N]$: $\text{firstmes}_\beta \leftarrow (\text{outs}_\beta, \text{hashes}_\beta)$ $\text{state}_\beta \leftarrow (\mathbf{X}, \mathbf{y}_{ext}, \text{keys}_\beta)$ $\text{resp}_\beta \leftarrow \Sigma.\text{Prove}_2(\text{state}_\beta, ch_\beta)$ $\text{tr}_\beta \leftarrow (\text{firstmes}_\beta, ch_\beta, \text{resp}_\beta)$ if $\forall \beta \in [N], \text{resp}_\beta \neq \perp$: return $(\mathbf{X}, \mathbf{y}, (\text{tr}_\beta)_{\beta \in [N]})$ return $\perp$

Fig. 18. The real view and the simulated view, appearing in the proof of [Theorem 5](#).

The hybrid experiments are all depicted in [Figs. 19](#) and [20](#), in which we also highlight in a yellow box what changes from the previous experiment. We give an overview of the experiments below, along with an intuition of how to move from one of them to the next one.

Real($\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$): As stated before, this is the real experiment, in which we made explicit the steps of $\Sigma.\text{Prove}_1$.

- Hyb₀($\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$):** This is just a syntactic modification of the real experiment, in which we move some computation at the beginning and use the above “batch” functions instead of computing everything in sequence in a loop.
- Hyb₁($\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$):** Here we replace the “batch” function $H\langle\langle N, t \rangle\rangle$ with $H\langle\langle 0, 0 \rangle\rangle$, thus changing the distribution of the hash values for the indices i^* . Namely, for those indices, instead of computing the correct hash value, we compute the hash value for a uniform, unrelated key. By a hybrid argument via a simple reduction to the inner hash function H , we show that this is computationally close to the previous hybrid.
- Hyb₂($\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$):** Here we replace the “batch” function $\text{Output}\langle\langle 0 \rangle\rangle$ with $\text{Output}\langle\langle 1 \rangle\rangle$, thus changing the computation of the output values $y^{(\beta, j)}$ for the indices i^* . Namely, instead of computing such values by the usual rounding function, we use the **Calc** function, that simulates the value of the carry and uses it to compute $y_{i^*}^{(\beta, j)}$. We will show that, whenever **Abort** does not happen, the simulated carry is computed in the same way as the real one, and, therefore, the reconstructed output value $y_{i^*}^{(\beta, j)}$ is identically distributed to the real one as well. For this reason, when conditioning on **Success**, this experiment is identical to the previous one.
- Hyb₃($\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$):** Here we replace the “batch” function $\text{Share}\langle\langle N, t \rangle\rangle$ with $\text{Share}\langle\langle 0, 0 \rangle\rangle$, thus changing the distribution of the key shares $k_i^{(\beta, j)}$. Namely, now the key shares do not depend anymore on the original key k , but are sampled using a uniform, unrelated key. By a hybrid argument via a reduction to the inner secret sharing scheme **Share** (which, in turns, requires a reduction to the inner PRG), we show that this is computationally close to the previous hybrid.
- Hyb₄(**LWR.Real**(k)):** This is a syntactic modification of the previous experiment. Namely, in the previous experiment, the witness k was just used to compute y_{ext} ; therefore, here we replace the input $\mathcal{D}_{\mathcal{X} \times \mathcal{W}}$ with an LWR instance (X, y_{ext}) in which y_{ext} is given.
- Hyb₄(**LWR.Rand**()):** Here we apply a simple reduction to LWR to move from the real instance to a random one in which y_{ext} is a uniform vector.
- Sim₁(**LWR.Rand**()):** Finally, this is the simulator for $\omega = 1$, which, conditioning on **Success**, can be seen as a syntactic modification of the previous experiment. The only non-syntactic modification consists of how the value y_{ext} is sampled: in the previous case, y_{ext} is just a uniform vector, while, in this experiment, y is a uniform vector and y_{ext} is computed starting from y and uniformly sampling the missing bits; however, the distribution of y_{ext} turns out to be identical in the two experiments.
- Sim₁(**LWR.Real**(k)):** This is the last experiment of the chain of hybrid experiments, in which, by a simple reduction to LWR, we return back to the situation in which (X, y) comes from the real distribution.

The final step consists in switching from **Sim₁** to **Sim_∞**, where the simulator only generates successful views. The full proof of **Theorem 5** follows.

We start by proving the indistinguishability of the inner components.

Hyb₀(($\mathbf{X}, \mathbf{y}$), $\mathbf{k}$) $\mathbf{y}_{ext} \leftarrow \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^p}$ $i^* \leftarrow \$_{[d]^{N \times t}}$ $(ch_\beta)_{\beta \in [N]} := ([d] \setminus \{i_{\beta,j}^*\})_{\beta \in [N], j \in [t]}$ $\text{coins} \leftarrow \$_{\mathcal{P}_H^{N \cdot t \cdot d}}$ $\text{keys} \leftarrow \$_{\text{Share}(\langle N, t \rangle)(\mathbf{k}) \parallel \text{coins}}$ $\text{outs} \leftarrow \$_{\text{Output}(\langle 0 \rangle)(\mathbf{X}, \text{keys}, \mathbf{y}_{ext}, i^*)}$ $\text{hashes} \leftarrow \$_{H(\langle N, t \rangle)(\text{keys}, i^*)}$ for $\beta \in [N]$: $\quad \text{firstmes}_\beta \leftarrow (\text{outs}_\beta, \text{hashes}_\beta)$ $\quad \text{state}_\beta \leftarrow (\mathbf{X}, \mathbf{y}_{ext}, \text{keys}_\beta)$ $\quad \text{resp}_\beta \leftarrow \Sigma.\text{Prove}_2(\text{state}_\beta, ch_\beta)$ $\quad \text{tr}_\beta \leftarrow (\text{firstmes}_\beta, ch_\beta, \text{resp}_\beta)$ return $(\mathbf{X}, \mathbf{y}, (\text{tr}_\beta)_{\beta \in [N]})$	Hyb₁(($\mathbf{X}, \mathbf{y}$), $\mathbf{k}$) $\mathbf{y}_{ext} \leftarrow \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^p}$ $i^* \leftarrow \$_{[d]^{N \times t}}$ $(ch_\beta)_{\beta \in [N]} := ([d] \setminus \{i_{\beta,j}^*\})_{\beta \in [N], j \in [t]}$ $\text{coins} \leftarrow \$_{\mathcal{P}_H^{N \cdot t \cdot d}}$ $\text{keys} \leftarrow \$_{\text{Share}(\langle N, t \rangle)(\mathbf{k}) \parallel \text{coins}}$ $\text{outs} \leftarrow \$_{\text{Output}(\langle 0 \rangle)(\mathbf{X}, \text{keys}, \mathbf{y}_{ext}, i^*)}$ $\text{hashes} \leftarrow \$_{H(\langle 0, 0 \rangle)(\text{keys}, i^*)}$ for $\beta \in [N]$: $\quad \text{firstmes}_\beta \leftarrow (\text{outs}_\beta, \text{hashes}_\beta)$ $\quad \text{state}_\beta \leftarrow (\mathbf{X}, \mathbf{y}_{ext}, \text{keys}_\beta)$ $\quad \text{resp}_\beta \leftarrow \Sigma.\text{Prove}_2(\text{state}_\beta, ch_\beta)$ $\quad \text{tr}_\beta \leftarrow (\text{firstmes}_\beta, ch_\beta, \text{resp}_\beta)$ return $(\mathbf{X}, \mathbf{y}, (\text{tr}_\beta)_{\beta \in [N]})$
Hyb₂(($\mathbf{X}, \mathbf{y}$), $\mathbf{k}$) $\mathbf{y}_{ext} \leftarrow \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^p}$ $i^* \leftarrow \$_{[d]^{N \times t}}$ $(ch_\beta)_{\beta \in [N]} := ([d] \setminus \{i_{\beta,j}^*\})_{\beta \in [N], j \in [t]}$ $\text{coins} \leftarrow \$_{\mathcal{P}_H^{N \cdot t \cdot d}}$ $\text{keys} \leftarrow \$_{\text{Share}(\langle N, t \rangle)(\mathbf{k}) \parallel \text{coins}}$ $\text{outs} \leftarrow \$_{\text{Output}(\langle 1 \rangle)(\mathbf{X}, \text{keys}, \mathbf{y}_{ext}, i^*)}$ $\text{hashes} \leftarrow \$_{H(\langle 0, 0 \rangle)(\text{keys}, i^*)}$ for $\beta \in [N]$: $\quad \text{firstmes}_\beta \leftarrow (\text{outs}_\beta, \text{hashes}_\beta)$ $\quad \text{state}_\beta \leftarrow (\mathbf{X}, \mathbf{y}_{ext}, \text{keys}_\beta)$ $\quad \text{resp}_\beta \leftarrow \Sigma.\text{Prove}_2(\text{state}_\beta, ch_\beta)$ $\quad \text{tr}_\beta \leftarrow (\text{firstmes}_\beta, ch_\beta, \text{resp}_\beta)$ return $(\mathbf{X}, \mathbf{y}, (\text{tr}_\beta)_{\beta \in [N]})$	Hyb₃(($\mathbf{X}, \mathbf{y}$), $\mathbf{k}$) $\mathbf{y}_{ext} \leftarrow \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^p}$ $i^* \leftarrow \$_{[d]^{N \times t}}$ $(ch_\beta)_{\beta \in [N]} := ([d] \setminus \{i_{\beta,j}^*\})_{\beta \in [N], j \in [t]}$ $\text{coins} \leftarrow \$_{\mathcal{P}_H^{N \cdot t \cdot d}}$ $\text{keys} \leftarrow \$_{\text{Share}(\langle 0, 0 \rangle)(\mathbf{k}) \parallel \text{coins}}$ $\text{outs} \leftarrow \$_{\text{Output}(\langle 1 \rangle)(\mathbf{X}, \text{keys}, \mathbf{y}_{ext}, i^*)}$ $\text{hashes} \leftarrow \$_{H(\langle 0, 0 \rangle)(\text{keys}, i^*)}$ for $\beta \in [N]$: $\quad \text{firstmes}_\beta \leftarrow (\text{outs}_\beta, \text{hashes}_\beta)$ $\quad \text{state}_\beta \leftarrow (\mathbf{X}, \mathbf{y}_{ext}, \text{keys}_\beta)$ $\quad \text{resp}_\beta \leftarrow \Sigma.\text{Prove}_2(\text{state}_\beta, ch_\beta)$ $\quad \text{tr}_\beta \leftarrow (\text{firstmes}_\beta, ch_\beta, \text{resp}_\beta)$ return $(\mathbf{X}, \mathbf{y}, (\text{tr}_\beta)_{\beta \in [N]})$

Fig. 19. The first four hybrid experiments in the proof of [Theorem 5](#).

$\text{Hyb}_4(\mathbf{X}, \mathbf{y}_{\text{ext}})$
$i^* \leftarrow_{\$} [d]^{N \times t}$
$(ch_\beta)_{\beta \in [N]} := ([d] \setminus \{i_{\beta,j}^*\})_{\beta \in [N], j \in [t]}$
$\text{coins} \leftarrow_{\$} \mathcal{P}_H^{N \cdot t \cdot d}$
$\text{keys} \leftarrow_{\$} \text{Share}\langle\langle 0, 0 \rangle\rangle(\perp) \parallel \text{coins}$
$\text{outs} \leftarrow_{\$} \text{Output}\langle\langle 1 \rangle\rangle(\mathbf{X}, \text{keys}, \mathbf{y}_{\text{ext}}, i^*)$
$\text{hashes} \leftarrow_{\$} H\langle\langle 0, 0 \rangle\rangle(\text{keys}, i^*)$
for $\beta \in [N]$:
$\quad \text{firstmes}_\beta \leftarrow (\text{outs}_\beta, \text{hashes}_\beta)$
$\quad \text{state}_\beta \leftarrow (\mathbf{X}, \mathbf{y}_{\text{ext}}, \text{keys}_\beta)$
$\quad \text{resp}_\beta \leftarrow \Sigma.\text{Prove}_2(\text{state}_\beta, ch_\beta)$
$\quad \text{tr}_\beta \leftarrow (\text{firstmes}_\beta, ch_\beta, \text{resp}_\beta)$
return $(\mathbf{X}, \mathbf{y}, (\text{tr}_\beta)_{\beta \in [N]})$

Fig. 20. The last hybrid experiment in the proof of [Theorem 5](#).

Lemma 11. For every vector of keys $\text{keys} := \left(\mathbf{k}_i^{(\beta,j)} \right)_{(\beta,j,i) \in [N] \times [t] \times [d]}$, every vector of indices $i^* := \left(i_{\beta,j}^* \right)_{(\beta,j) \in [N] \times [t]}$ and every stateful quantum polynomial-time adversary $\mathbf{A}$,

$$|\mathbb{P}[\mathbf{A}(H\langle\langle N, t \rangle\rangle(\text{keys}, i^*))] - \mathbb{P}[\mathbf{A}(H\langle\langle 0, 0 \rangle\rangle(\text{keys}, i^*))]| \leq Nt\epsilon_{\text{Hide}}.$$

Furthermore, it is well defined $H\langle\langle N, t \rangle\rangle(\text{keys})$ such that, for all i^* ,

$$H\langle\langle N, t \rangle\rangle(\text{keys}) = H\langle\langle N, t \rangle\rangle(\text{keys}, i^*).$$

Proof. The proof proceeds by hybrid argument. Notice that, for $\beta, \beta^* \in [N], j \in [t]$, the condition $(\beta, j) \leq_{\text{lex}} (\beta^*, 0)$ implies that $\beta < \beta^*$ (since $j \geq 1$), or, equivalently, $\beta \leq \beta^* - 1$. Since $j \leq t$, this also implies $(\beta, j) \leq_{\text{lex}} (\beta^* - 1, t)$, which means that the condition in $H\langle\langle \beta^*, 0 \rangle\rangle$ and in $H\langle\langle \beta^* - 1, t \rangle\rangle$ is actually the same and the two experiments are identical. Furthermore, the condition $(\beta, j) \leq_{\text{lex}} (\beta^*, j^*)$ is never true for $\beta^* = 0$, making all the experiments $H\langle\langle 0, j^* \rangle\rangle$ for $j^* \in [t] \cup \{0\}$ identical. We now show that for $(\beta^*, j^*) \in [N] \times [t]$, the computational distance between $H\langle\langle \beta^*, j^* \rangle\rangle$ and $H\langle\langle \beta^*, j^* - 1 \rangle\rangle$ is bounded by ϵ_{Hide} ; the first part of the lemma will then follow by a nested hybrid argument: indeed, for every $\beta^* \in [N]$ and every stateful quantum polynomial-time adversary $\mathbf{A}$, an hybrid argument of t experiments shows that

$$|\mathbb{P}[\mathbf{A}(H\langle\langle \beta^*, t \rangle\rangle(\text{keys}, i^*))] - \mathbb{P}[\mathbf{A}(H\langle\langle \beta^*, 0 \rangle\rangle(\text{keys}, i^*))]| \leq t\epsilon_{\text{Hide}},$$

and, by $H\langle\langle \beta^*, 0 \rangle\rangle \equiv H\langle\langle \beta^* - 1, t \rangle\rangle$ we again have an hybrid arguments of N experiments.

Fix $(\beta^*, j^*) \in [N] \times [t]$. Then, the only difference between $H\langle\langle\beta^*, j^*\rangle\rangle$ and $H\langle\langle\beta^*, j^* - 1\rangle\rangle$ is how the hash $h_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ is computed. Namely, $h_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ is a commitment to $\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ in $H\langle\langle\beta^*, j^*\rangle\rangle$, and it is a commitment to an unrelated random value in $H\langle\langle\beta^*, j^* - 1\rangle\rangle$. The first part of the lemma follows by a simple reduction to the hiding property of H . The second part follows by realizing that all the hash values are computed from the real keys in $H\langle\langle N, t\rangle\rangle$ and, therefore, the function does not depend on i^* . $\square$

Lemma 12. *For every $\text{keys} := (\mathbf{k}_i^{(\beta, j)})_{(\beta, j, i) \in [N] \times [t] \times [d]}$, $i^* := (i_{\beta, j}^*)_{(\beta, j) \in [N] \times [t]}$, $\mathbf{X}, \mathbf{y}_{ext}$ and every stateful quantum polynomial-time adversary $\mathbf{A}$,*

$$\begin{aligned} & \mathbb{P}[\mathbf{A}(\text{Output}\langle\langle 0 \rangle\rangle)(\mathbf{X}, \text{keys}, \mathbf{y}_{ext}, i^*) \mid \text{Success}] \\ &= \mathbb{P}[\mathbf{A}(\text{Output}\langle\langle 1 \rangle\rangle)(\mathbf{X}, \text{keys}, \mathbf{y}_{ext}, i^*) \mid \text{Success}] \end{aligned}$$

Proof. First of all, notice that the only difference concerns the computation of the values $\mathbf{y}_{i_{\beta, j}^*}^{(\beta, j)}$ for $(\beta, j) \in [N] \times [t]$. To make notation easier, we focus on a single iteration, and we drop the indices β and j ; therefore, we focus on the computation $\mathbf{y}_{i^*}$.

In $\text{Output}\langle\langle 0 \rangle\rangle$, all the values $\mathbf{y}_i$ for $i \in [d]$ are computed by using the function Output to get $\mathbf{z}_i$ and then rounding, while, in $\text{Output}\langle\langle 1 \rangle\rangle$, the values $\mathbf{y}_{i^*}$ are computed using the special function Calc . Expanding the definition of Calc , we get that, in $\text{Output}\langle\langle 0 \rangle\rangle$, the values $\mathbf{y}$ and $\mathbf{y}_{i^*}$ are computed as

$$\mathbf{y} = \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^{p-\alpha}} \quad \text{and} \quad \mathbf{y}_{i^*} = \lfloor \mathbf{z}_{i^*} \rfloor_{2^{p-\alpha}}$$

respectively. Notice that $\mathbf{X}\mathbf{k} = \sum_{i=1}^d \mathbf{z}_i$ and, therefore, by recalling that $r_E = q - p + \alpha$,

$$\begin{aligned} \mathbf{y} &= \left\lfloor \sum_{i=1}^d \mathbf{z}_i \right\rfloor_{2^{p-\alpha}} \\ &= \left\lfloor \mathbf{z}_{i^*} + \sum_{i \neq i^*} \mathbf{z}_i \right\rfloor_{2^{p-\alpha}} \\ &= \lfloor \mathbf{z}_{i^*} + \text{sum} \rfloor_{2^{p-\alpha}} \\ &= \mathbf{y}_{i^*} + \lfloor \text{sum} \rfloor_{2^{p-\alpha}} + \lfloor \mathbf{z}_{i^*}[0..r_E] + \text{sum}[0..r_E] \rfloor_{2^{p-\alpha}}, \end{aligned}$$

where the last equality follows by [Item 2](#) of [Proposition 2](#). By letting

$$\mathbf{c}' := \lfloor \mathbf{z}_{i^*}[0..r_E] + \text{sum}[0..r_E] \rfloor_{2^{p-\alpha}},$$

and recalling that $\mathbf{y} = \lfloor \mathbf{y}_{ext} \rfloor_{2^{p-\alpha}}$, we get another way to compute $\mathbf{y}_{i^*}$, that is,

$$\mathbf{y}_{i^*} = \lfloor \mathbf{y}_{ext} \rfloor_{2^{p-\alpha}} - \lfloor \text{sum} \rfloor_{2^{p-\alpha}} - \mathbf{c}'.$$

On the other side, in $\text{Output}\langle\langle 1 \rangle\rangle$, the value $\mathbf{y}_{i^*}$ is computed directly as

$$\mathbf{y}_{i^*} = \lfloor \mathbf{y}_{ext} \rfloor_{2^{p-\alpha}} - \lfloor \text{sum} \rfloor_{2^{p-\alpha}} - \mathbf{c},$$

and, therefore, we only need to show that, when **Success** happens, the two ways of computing the carry are equivalent, i.e.,

$$\mathbf{c} = \mathbf{c}'.$$

Since $\lfloor \cdot \rfloor$ acts component-wise, we show that this holds for every component of the m -sized vectors that we are considering; namely, assuming that $\mathbf{y}_{ext}, \text{sum}, \mathbf{y}, \mathbf{z}_{i^*}$ are all scalars, we want to show that

$$1 - \lfloor 2^\alpha + \mathbf{y}_{ext}[0..\alpha] - \text{sum}[r_S..r_E] \rfloor_2 = \lfloor \mathbf{z}_{i^*}[0..r_E] + \text{sum}[0..r_E] \rfloor_{2^{p-\alpha}}, \quad (35)$$

where the left side of the above is how $\mathbf{c}$ is computed and the right side is exactly $\mathbf{c}'$. We split the proof in three cases.

– *Case* $\mathbf{y}_{ext}[0..\alpha] - \text{sum}[r_S..r_E] > 0$. In this case,

$$\begin{aligned} 2^\alpha &< 2^\alpha + \mathbf{y}_{ext}[0..\alpha] - \text{sum}[r_S..r_E] \\ &\leq 2^\alpha + \mathbf{y}_{ext}[0..\alpha] \\ &< 2^\alpha + 2^\alpha, \end{aligned}$$

hence

$$\lfloor 2^\alpha + \mathbf{y}_{ext}[0..\alpha] - \text{sum}[r_S..r_E] \rfloor_2 = 1$$

and the left side of [Eq. \(35\)](#) is 0. On the other side, by $\mathbf{y}_{ext}[0..\alpha] = \mathbf{z}[r_S..r_E]$ and [Item 4 of Proposition 2](#),

$$\mathbf{y}_{ext}[0..\alpha] > \text{sum}[r_S..r_E] \implies \mathbf{z}[0..r_E] > \text{sum}[0..r_E].$$

In this case, $\mathbf{z}_{i^*} = \mathbf{z} - \text{sum}$, and the same happens when only considering the r_E LSBs, which means that the right side of the equality simplifies as

$$\begin{aligned} \lfloor \mathbf{z}_{i^*}[0..r_E] + \text{sum}[0..r_E] \rfloor_{2^{p-\alpha}} &= \lfloor \mathbf{z}[0..r_E] - \text{sum}[0..r_E] + \text{sum}[0..r_E] \rfloor_{2^{p-\alpha}} \\ &= \lfloor \mathbf{z}[0..r_E] \rfloor_{2^{p-\alpha}} \\ &= 0. \end{aligned}$$

– *Case* $\mathbf{y}_{ext}[0..\alpha] - \text{sum}[r_S..r_E] < 0$. This case is very similar to the previous one, except that the carry assumes value 1. Namely, we get that

$$0 < 2^\alpha + \mathbf{y}_{ext}[0..\alpha] - \text{sum}[r_S..r_E] < 2^\alpha,$$

and the left side of [Eq. \(35\)](#) simplifies as

$$1 - \lfloor 2^\alpha + \mathbf{y}_{ext}[0..\alpha] - \text{sum}[r_S..r_E] \rfloor_2 = 1 - 0 = 1.$$

On the other side, by $\mathbf{y}_{ext}[0..\alpha] = \mathbf{z}[r_S..r_E]$ and [Item 4](#) of [Proposition 2](#), again,

$$\mathbf{y}_{ext}[0..\alpha] < \text{sum}[r_S..r_E] \implies \mathbf{z}[0..r_E] < \text{sum}[0..r_E].$$

In this case, $\mathbf{z}_{i^*} = 2^{p-\alpha} + \mathbf{z} - \text{sum}$, the same happens when only considering the r_E LSBs, which means that the right side of the equality simplifies as

$$\begin{aligned} & \lfloor \mathbf{z}_{i^*}[0..r_E] + \text{sum}[0..r_E] \rfloor_{2^{p-\alpha}} \\ &= \lfloor 2^{p-\alpha} + \mathbf{z}[0..r_E] - \text{sum}[0..r_E] + \text{sum}[0..r_E] \rfloor_{2^{p-\alpha}} \\ &= \lfloor 2^{p-\alpha} + \mathbf{z}[0..r_E] \rfloor_{2^{p-\alpha}} \\ &= 1. \end{aligned}$$

- *Case $\mathbf{y}_{ext}[0..\alpha] = \text{sum}[r_S..r_E]$.* In this case, it may happen that $\mathbf{c} \neq \mathbf{c}'$ with non-negligible probability. However, this is also the case that triggers $\mathbf{Abort} = 1$ and, therefore, we don't need the condition $\mathbf{c} = \mathbf{c}'$.

Finally, the values sum and $\mathbf{y}_{ext}$ are computed in the same way in both experiments and, since they are the only ones influencing the event **Abort**, the probability of **Success** is the same in both experiments. The claim follows by extending the above reasoning to all components of the involved vectors. $\square$

Now we show, via a sequence of hybrid experiments, that $\text{Hyb}_2(\mathcal{D}_{\mathcal{X} \times \mathcal{W}})$ and $\text{Sim}_1(\mathcal{D}_{\mathcal{X}})$ are computationally close. Looking forward, we will use this fact, with an additional lemma, to show that, when **Abort** does not happen, **Real** and Sim_1 are computationally close.

Lemma 13. *For every $\mathbf{X} \in \mathbb{Z}_Q^{m \times n}$, $\mathbf{k} \in \mathbb{Z}_Q^n$, $\mathbf{y} = \lfloor \mathbf{X}\mathbf{k} \rfloor$, and every stateful quantum polynomial-time adversary $\mathbf{A}$,*

$$|\mathbb{P}[\mathbf{A}(\text{Hyb}_2((\mathbf{X}, \mathbf{y}), \mathbf{k}))] - \mathbb{P}[\mathbf{A}(\text{Hyb}_3((\mathbf{X}, \mathbf{y}), \mathbf{k}))]| \leq N\epsilon_{\text{PRG}}.$$

Proof. For any $(\beta^*, j^*) \in [N] \times [t]$, let $\text{Hyb}^* \langle \beta^*, j^* \rangle$ be the same as Hyb_2 (resp., Hyb_3) except for the fact that we replace $\text{Share} \langle N, t \rangle$ (resp., $\text{Share} \langle 0, 0 \rangle$) with $\text{Share} \langle \beta^*, j^* \rangle$. In particular, we get that $\text{Hyb}^* \langle N, t \rangle \equiv \text{Hyb}_2$ and $\text{Hyb}^* \langle 0, 0 \rangle \equiv \text{Hyb}_3$, since the batch parameters of Share are the only difference between the two experiments. As in the proof for [Lemma 11](#), we only need to prove that, for any $(\beta^*, j^*) \in [N] \times [t]$,

$$|\mathbb{P}[\mathbf{A}(\text{Hyb}^* \langle \beta^*, j^* \rangle((\mathbf{X}, \mathbf{y}), \mathbf{k}))] - \mathbb{P}[\mathbf{A}(\text{Hyb}^* \langle \beta^*, j^* - 1 \rangle((\mathbf{X}, \mathbf{y}), \mathbf{k}))]| \leq \epsilon_{\text{PRG}}$$

and then conclude by nested hybrid argument. We proceed by reduction to the PRG $G_{\mathcal{K}}$. Notice that, in both experiments, the key share $\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ is never revealed, nor used anywhere. Indeed, $\mathbf{y}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ is computed by using all the other shares, $h_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ is the commitment to a uniformly sampled and independent value $\mathbf{k}^*$, and, being $ch_{\beta^*, j^*} = [d] \setminus \{i_{\beta^*, j^*}^*\}$, share $\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ will never make it into *resp.* We divide the proof in two cases.

Case $i_{\beta^*, j^*}^* = d$. In this case, $\mathbf{k}_1^{(\beta^*, j^*)}, \dots, \mathbf{k}_{d-1}^{(\beta^*, j^*)}$ are generated in the same way in both experiments: in particular, they are randomly sampled, independently of the key $\mathbf{k}$. Therefore, in this case, the two experiments are identical.

Case $i_{\beta^*, j^*}^* < d$. This case is slightly more complicated, because we now give to the adversary the value

$$\mathbf{k}_d^{(\beta^*, j^*)} = \mathbf{k} - \sum_{i=1}^{d-1} G_{\mathcal{K}} \left(\mathbf{k}_i^{(\beta^*, j^*)} \right),$$

which depends on $\mathbf{k}$. However, in this case, one of the seeds $\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ is never revealed, and a simple reduction to $G_{\mathcal{K}}$ shows that we can replace the PRG generated share $G_{\mathcal{K}}(\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)})$ with a uniformly sampled one $\tilde{\mathbf{k}}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$. More in detail, we define a reduction R that, upon input a value $\hat{\mathbf{k}}$, runs exactly as $\text{Hyb}^* \langle \beta^*, j^* \rangle$, with the only difference of setting $\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)} \leftarrow \hat{\mathbf{k}}$. Then, R perfectly simulates $\text{Hyb}^* \langle \beta^*, j^* - 1 \rangle$ if $\hat{\mathbf{k}}$ comes from the PRG, and perfectly simulates $\text{Hyb}^* \langle \beta^*, j^* \rangle$ if $\hat{\mathbf{k}}$ is uniformly sampled. Then, the advantage of any stateful quantum polynomial-time adversary A' distinguishing between R with PRG input and R with real input is bounded by ϵ_{PRG} . A final step is missing, to move from the situation where the shares are sampled from the original key $\mathbf{k}$ to the situation where they are sampled by $\text{Sample}()$. Namely, after the reduction R, the shares are still sampled by sampling $\mathbf{k}_1^{(\beta^*, j^*)}, \dots, \mathbf{k}_{d-1}^{(\beta^*, j^*)}$ first and then letting

$$\mathbf{k}_d^{(\beta^*, j^*)} = \mathbf{k} - \mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)} - \sum_{i \in [d-1] \setminus \{i_{\beta^*, j^*}^*\}} G_{\mathcal{K}} \left(\mathbf{k}_i^{(\beta^*, j^*)} \right).$$

It is easy to see that this is equivalent to sampling all the shares except for $\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$, and then letting

$$\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)} = \mathbf{k} - \mathbf{k}_d^{(\beta^*, j^*)} - \sum_{i \in [d-1] \setminus \{i_{\beta^*, j^*}^*\}} G_{\mathcal{K}} \left(\mathbf{k}_i^{(\beta^*, j^*)} \right).$$

However, now we can use the same reasoning of the case $i_{\beta^*, j^*}^* = d$ since we don't reveal $\mathbf{k}_{i_{\beta^*, j^*}^*}^{(\beta^*, j^*)}$ anywhere and this is the only share that functionally depends on the key $\mathbf{k}$. □

Lemma 14. *For uniform $\mathbf{k}$,*

$$\text{Hyb}_3(\mathcal{D}_{\mathcal{X} \times \mathcal{W}}) \equiv \text{Hyb}_4(\text{LWR.Real}(\mathbf{k})).$$

Proof. The only difference between Hyb_3 and Hyb_4 is what is given as input and how $\mathbf{y}_{\text{ext}}$ is generated. Namely, the only two places where Hyb_3 uses the

input $\mathbf{k}$ are when $\mathbf{y}_{ext}$ is computed and when $\mathbf{keys}$ is computed. However, in the computation of $\mathbf{keys}$, $\mathbf{k}$ is never used, since $\text{Share}\langle\langle 0, 0 \rangle\rangle$ never uses its input, while the value $\mathbf{y}_{ext}$ is computed exactly as in the real case. Therefore, if we give $\mathbf{y}_{ext}$ as input directly, we don't need $\mathbf{k}$ anymore, which is what happens in Hyb_4 . $\square$

Lemma 15. *For uniform $\mathbf{k}$, and every stateful quantum polynomial-time adversary A ,*

$$|\mathbb{P}[A(\text{Hyb}_4(\text{LWR.Real}(\mathbf{k})))] - \mathbb{P}[A(\text{Hyb}_4(\text{LWR.Rand}()))]| \leq \epsilon_{\text{LWR}}.$$

Proof. Assume not. Then, there exists A such that

$$|\mathbb{P}[A(\text{Hyb}_4(\text{LWR.Real}(\mathbf{k})))] - \mathbb{P}[A(\text{Hyb}_4(\text{LWR.Rand}()))]| > \epsilon_{\text{LWR}},$$

which, in turns, means that there exists $A' := A \circ \text{Hyb}_4$ such that

$$|\mathbb{P}[A'(\text{LWR.Real}(\mathbf{k}))] - \mathbb{P}[A'(\text{LWR.Rand}())]| > \epsilon_{\text{LWR}},$$

thus breaking the LWR assumption. $\square$

Lemma 16.

$$\text{Hyb}_4(\text{LWR.Rand}()) \equiv \text{Sim}_1(\text{LWR.Rand}()).$$

Proof. The only modification is that Sim_1 , instead of receiving $(\mathbf{X}, \mathbf{y}_{ext})$, receives $(\mathbf{X}, \mathbf{y})$ and uniformly samples the missing α bits per component to get $\mathbf{y}_{ext}$. Since both samples $(\mathbf{X}, \mathbf{y}_{ext})$ and $(\mathbf{X}, \mathbf{y})$ come from the random distributions of the corresponding LWR instances, the value of $\mathbf{y}_{ext}$ in the two experiments is identically distributed. $\square$

Lemma 17. *For uniform $\mathbf{k}$, and every stateful quantum polynomial-time adversary A ,*

$$|\mathbb{P}[A(\text{Sim}_1(\text{LWR.Rand}()))] - \mathbb{P}[A(\text{Sim}_1(\text{LWR.Real}(\mathbf{k})))]| \leq \epsilon_{\text{LWR}}.$$

Proof. Assume not. Then, there exists A such that

$$|\mathbb{P}[A(\text{Sim}_1(\text{LWR.Rand}()))] - \mathbb{P}[A(\text{Sim}_1(\text{LWR.Real}(\mathbf{k})))]| > \epsilon_{\text{LWR}},$$

which, in turns, means that there exists $A' := A \circ \text{Sim}_1$ such that

$$|\mathbb{P}[A'(\text{LWR.Rand}())] - \mathbb{P}[A'(\text{LWR.Real}(\mathbf{k}))]| > \epsilon_{\text{LWR}},$$

thus breaking the LWR assumption. $\square$

The following lemma is needed in order to step from the general indistinguishability of experiments to the indistinguishability of experiments conditioning on some non-negligible event happening.

Lemma 18. *Let $\mathcal{D}, \mathcal{D}'$ be two distributions over a set $\mathcal{X}$, let $\mathcal{S} \subseteq \mathcal{X}$, and let $\mathbf{E}$ (resp. $\mathbf{E}'$) be the event that $x \in \mathcal{S}$ when $x \leftarrow \mathcal{D}$ (resp. $x \leftarrow \mathcal{D}'$). Assume that $\mathbb{P}[\mathbf{E}] > 0$, $\mathbb{P}[\mathbf{E}'] > 0$, and, for some $\epsilon \in [0, 1]$ and for every stateful quantum polynomial-time adversary $\mathbf{A}$,*

$$|\mathbb{P}[\mathbf{A}(\mathcal{D}) = 1] - \mathbb{P}[\mathbf{A}(\mathcal{D}') = 1]| \leq \epsilon.$$

Finally, assume that $x \in \mathcal{S}$ is an efficiently verifiable statement. Then, for every stateful quantum polynomial-time adversary $\mathbf{A}'$,

$$|\mathbb{P}[\mathbf{A}'(\mathcal{D}) = 1 \mid \mathbf{E}] - \mathbb{P}[\mathbf{A}'(\mathcal{D}') = 1 \mid \mathbf{E}']| \leq \min \left\{ \frac{2\epsilon}{\mathbb{P}[\mathbf{E}]}, \frac{2\epsilon}{\mathbb{P}[\mathbf{E}']} \right\}.$$

Furthermore, $|\mathbb{P}[\mathbf{E}] - \mathbb{P}[\mathbf{E}']| \leq \epsilon$.

Proof. First of all, we show that

$$|\mathbb{P}[\mathbf{E}] - \mathbb{P}[\mathbf{E}']| \leq \epsilon.$$

Indeed, assume that this is not the case, and consider an adversary $\mathbf{A}''$ that outputs 1 if $x \in \mathcal{S}$ and 0 otherwise (this adversary exists because the statement $x \in \mathcal{S}$ is efficiently verifiable). Then,

$$\begin{aligned} & |\mathbb{P}[\mathbf{E}] - \mathbb{P}[\mathbf{E}']| \\ &= |\mathbb{P}[\mathbf{A}''(\mathcal{D}) = 1] - \mathbb{P}[\mathbf{A}''(\mathcal{D}') = 1]| \\ &\leq \epsilon, \end{aligned}$$

where the last step comes directly from the hypothesis.

From the above, we have that $\mathbb{P}[\mathbf{E}'] \leq \mathbb{P}[\mathbf{E}] + \epsilon$. Let $\mathbf{B} : \mathcal{X} \rightarrow \mathcal{X} \cup \{\perp\}$ be the function that maps every $x \in \mathcal{S}$ to $\perp$ and behaves like the identity otherwise. Then, for every stateful quantum polynomial-time adversary $\mathbf{A}'$,

$$\begin{aligned} & |\mathbb{P}[\mathbf{A}'(\mathcal{D}) = 1] - \mathbb{P}[\mathbf{A}'(\mathcal{D}') = 1]| \\ &\geq |\mathbb{P}[\mathbf{A}'(\mathbf{B}(\mathcal{D})) = 1] - \mathbb{P}[\mathbf{A}'(\mathbf{B}(\mathcal{D}')) = 1]| \\ &= |\mathbb{P}[\mathbf{A}'(\mathbf{B}(\mathcal{D})) = 1 \wedge \mathbf{E}] - \mathbb{P}[\mathbf{A}'(\mathbf{B}(\mathcal{D}')) = 1 \wedge \mathbf{E}']| \\ &= |\mathbb{P}[\mathbf{E}] \mathbb{P}[\mathbf{A}'(\mathbf{B}(\mathcal{D})) = 1 \mid \mathbf{E}] - \mathbb{P}[\mathbf{E}'] \mathbb{P}[\mathbf{A}'(\mathbf{B}(\mathcal{D}')) = 1 \mid \mathbf{E}']| \\ &= |\mathbb{P}[\mathbf{E}] \mathbb{P}[\mathbf{A}'(\mathcal{D}) = 1 \mid \mathbf{E}] - \mathbb{P}[\mathbf{E}'] \mathbb{P}[\mathbf{A}'(\mathcal{D}') = 1 \mid \mathbf{E}']| \\ &\geq \mathbb{P}[\mathbf{E}] |\mathbb{P}[\mathbf{A}'(\mathcal{D}) = 1 \mid \mathbf{E}] - \mathbb{P}[\mathbf{A}'(\mathcal{D}') = 1 \mid \mathbf{E}']| - \epsilon, \end{aligned}$$

and, by turning around the inequality and letting $\mathbf{A} = \mathbf{A}'$,

$$\begin{aligned} & |\mathbb{P}[\mathbf{A}'(\mathcal{D}) = 1 \mid \mathbf{E}] - \mathbb{P}[\mathbf{A}'(\mathcal{D}') = 1 \mid \mathbf{E}']| \\ &\leq \frac{|\mathbb{P}[\mathbf{A}(\mathcal{D}) = 1] - \mathbb{P}[\mathbf{A}(\mathcal{D}') = 1]| + \epsilon}{\mathbb{P}[\mathbf{E}]} \\ &\leq \frac{2\epsilon}{\mathbb{P}[\mathbf{E}]}. \end{aligned}$$

The proof for

$$|\mathbb{P}[\mathbf{A}'(\mathcal{D}) = 1 \mid \mathbf{E}] - \mathbb{P}[\mathbf{A}'(\mathcal{D}') = 1 \mid \mathbf{E}']| \leq \frac{2\epsilon}{\mathbb{P}[\mathbf{E}']}$$

is analogous. $\square$

The following lemma puts everything together, proving that **Real** and **Sim**₁ are computationally close when **Abort** does not happen.

Lemma 19. *For every stateful quantum polynomial-time adversary $\mathbf{A}$,*

$$\begin{aligned} & |\mathbb{P}[\mathbf{A}(\mathbf{Real}(\mathcal{D}_{\mathcal{X} \times \mathcal{W}})) \mid \mathbf{Success}] - \mathbb{P}[\mathbf{A}(\mathbf{Sim}_1(\mathcal{D}_{\mathcal{X}})) \mid \mathbf{Success}]| \\ & \leq Nt\epsilon_{\text{Hide}} + \frac{2Nt\epsilon_{\text{PRG}} + 4\epsilon_{\text{LWR}}}{1 - mNt2^{-\alpha}}. \end{aligned}$$

Proof. First of all, notice that the only difference between **Real** and **Hyb**₀ is where the keys, outputs, hash values, and challenges are computed. Namely, the real experiment computes each of the above in its own loop, while **Hyb**₀ computes everything in batch at the beginning of the experiment. Furthermore, the challenge i^* is given as input to **Output** $\langle\langle 0 \rangle\rangle$ and **H** $\langle\langle N, t \rangle\rangle$, but none of them actually uses the indices. Therefore, the two experiments are identical:

$$\mathbf{Real}(\mathcal{D}_{\mathcal{X} \times \mathcal{W}}) \equiv \mathbf{Hyb}_0(\mathcal{D}_{\mathcal{X} \times \mathcal{W}}). \quad (36)$$

Moving on, the event **Success** does not depend nor affects the distribution of hashes and, therefore,

$$\begin{aligned} & |\mathbb{P}[\mathbf{A}(\mathbf{Hyb}_0(\mathcal{D}_{\mathcal{X} \times \mathcal{W}})) \mid \mathbf{Success}] - \mathbb{P}[\mathbf{A}(\mathbf{Hyb}_1(\mathcal{D}_{\mathcal{X} \times \mathcal{W}})) \mid \mathbf{Success}]| \\ & = |\mathbb{P}[\mathbf{A}(\mathbf{Hyb}_0(\mathcal{D}_{\mathcal{X} \times \mathcal{W}}))] - \mathbb{P}[\mathbf{A}(\mathbf{Hyb}_1(\mathcal{D}_{\mathcal{X} \times \mathcal{W}}))]| \\ & \leq Nt\epsilon_{\text{Hide}}, \end{aligned} \quad (37)$$

where the last inequality is a consequence of **Lemma 11**. Furthermore, by applying **Lemmas 13** to **17** and the triangular inequality, we have that

$$|\mathbb{P}[\mathbf{A}(\mathbf{Hyb}_2(\mathcal{D}_{\mathcal{X} \times \mathcal{W}}))] - \mathbb{P}[\mathbf{A}(\mathbf{Sim}_1(\mathcal{D}_{\mathcal{X}}))]| \leq Nt\epsilon_{\text{PRG}} + 2\epsilon_{\text{LWR}}.$$

Let **Success'** be the event that **Abort** does not happen in **Sim**₁($\mathcal{D}_{\mathcal{X}}$). Then, by **Lemma 18**, the above inequality becomes

$$\begin{aligned} & |\mathbb{P}[\mathbf{A}(\mathbf{Hyb}_2(\mathcal{D}_{\mathcal{X} \times \mathcal{W}})) \mid \mathbf{Success}] - \mathbb{P}[\mathbf{A}(\mathbf{Sim}_1(\mathcal{D}_{\mathcal{X}})) \mid \mathbf{Success}']| \\ & \leq \frac{2Nt\epsilon_{\text{PRG}} + 4\epsilon_{\text{LWR}}}{\mathbb{P}[\mathbf{Success}']}. \end{aligned} \quad (38)$$

Notice that the event **Abort** happens whenever the first α bits of one of the components of the vector $\mathbf{y}_{\text{ext}}$ match the same bits of one of the components of any of the *sum* vectors generated in any of the Nt iterations. Let **Abort** _{i} be the event where **Abort** happens and is caused by the i -th component of the above

vectors. Since these α bits of $\mathbf{y}_{ext}$ are uniform in $\text{Sim}_1(\mathcal{D}_{\mathcal{X}})$, the probability that the α bits of a single component cause **Abort**_{*i*} is

$$\mathbb{P}[\mathbf{Abort}_i] = \frac{Nt}{2^\alpha},$$

and, by the union bound,

$$\begin{aligned} \mathbb{P}[\mathbf{Abort}] &= \mathbb{P}\left[\bigvee_{i \in [m]} \mathbf{Abort}_i\right] \\ &\leq \sum_{i \in [m]} \mathbb{P}[\mathbf{Abort}_i] \\ &= m \frac{Nt}{2^\alpha}. \end{aligned}$$

Therefore, by complementing the above event, we get that

$$\mathbb{P}[\mathbf{Success}'] \geq 1 - mNt2^{-\alpha}. \quad (39)$$

Finally, the lemma follows by plugging together [Eqs. \(36\) to \(39\)](#) and [Lemma 12](#) and observing that, since $\text{Output}\langle\langle 0 \rangle\rangle \equiv \text{Output}\langle\langle 1 \rangle\rangle$ when **Abort** does not happen, $\text{Hyb}_1 \equiv \text{Hyb}_2$ under the same condition. $\square$

To conclude the proof, we just replace Sim_1 with Sim_ω , which tries to sample valid transcripts for ω before giving up, thus amplifying the probability of success.

Proof (of [Theorem 5](#)). It suffices to show that, for every $\omega > 0$ and every stateful quantum polynomial-time adversary A ,

$$|\mathbb{P}[A(\text{Sim}_1(\mathcal{D}_{\mathcal{X}})) \mid \neg \mathbf{Abort}] - \mathbb{P}[A(\text{Sim}_\omega(\mathcal{D}_{\mathcal{X}}))]| \leq 2(mNt2^{-\alpha})^\omega,$$

and the proof will follow by the triangular inequality and by the fact that

$$\lim_{\omega \rightarrow \infty} 2(mNt2^{-\alpha})^\omega = 0.$$

In each attempt, the simulator Sim_ω discards everything and samples everything from scratch, causing the event **Abort** with probability at most $mNt2^{-\alpha}$. Therefore, the probability that, for ω consecutive attempts, **Abort** happens is upper bounded by

$$(mNt2^{-\alpha})^\omega.$$

If the event **Abort** does not happen, Sim_1 and Sim_ω output a sample from the same distribution. The lemma follows by

$$\begin{aligned} &|\mathbb{P}[A(\text{Sim}_1(\mathcal{D}_{\mathcal{X}})) \mid \neg \mathbf{Abort}] - \mathbb{P}[A(\text{Sim}_\omega(\mathcal{D}_{\mathcal{X}}))]| \\ &\leq |\mathbb{P}[A(\text{Sim}_1(\mathcal{D}_{\mathcal{X}})) \mid \neg \mathbf{Abort}] - \mathbb{P}[A(\text{Sim}_\omega(\mathcal{D}_{\mathcal{X}})) \mid \neg \mathbf{Abort}] \mathbb{P}[\neg \mathbf{Abort}]| \\ &\quad + \mathbb{P}[\mathbf{Abort}] \end{aligned}$$

$$\begin{aligned}
 &\leq |\mathbb{P}[\mathbf{A}(\text{Sim}_1(\mathcal{D}_{\mathcal{X}})) \mid \neg \mathbf{Abort}] - \mathbb{P}[\mathbf{A}(\text{Sim}_\omega(\mathcal{D}_{\mathcal{X}})) \mid \neg \mathbf{Abort}] (1 - \mathbb{P}[\mathbf{Abort}])| \\
 &\quad + \mathbb{P}[\mathbf{Abort}] \\
 &\leq 2\mathbb{P}[\mathbf{Abort}] \leq 2(mNt2^{-\alpha})^\omega.
 \end{aligned}$$

This concludes the proof. $\square$

E.3 Special Soundness (Theorem 6)

We begin with an outline of the proof intuition, and later provide a formal treatment. First of all, we isolate the event where the adversary succeeds in producing $s = 2$ accepting proofs for the same commitment and different challenges. Otherwise, there would be nothing to extract. Next, we observe that having 2 such transcripts provides the extractor with a complete sharing of a candidate witness $\mathbf{k}$. Therefore, extraction can be finalized by reconstructing $\mathbf{k}$ from its sharing. Finally, we prove that this extraction strategy always provides a witness for the given statement. To do it, we rely on the validity of each transcript with respect to the same statement and first message, and on the almost homomorphism of the rounding function.

Our extractor always reconstructs the sharing in the same way: $d - 1$ shares from the first transcript, and the remaining share from the second. Notably, there can be many different ways to reconstruct the sharing: we could have also considered an extractor taking $d - i$ shares from the first transcript, and the remaining i shares from the second. In principle, all of them could result in a different witness candidate. Interestingly, all such cases would lead to a valid witness, for two main reasons. First, despite the opened key shares $\mathbf{k}_{i,1}$ and $\mathbf{k}_{i,2}$ can be distinct, they all verify with respect to the same first message, and in particular they all correspond to the same output shares $\mathbf{y}_i$. Moreover, our witness relation is defined at the output level, and therefore the specific choice of the reconstruction is not relevant as long as the output shares stay the same. For the formal analysis, we consider the extractor $\mathbf{E}$ defining special soundness described in Fig. 21.

In order to prove $(s, \epsilon_{\Sigma, \text{SS}})$ -special soundness for Σ , we need to upper bound with $\epsilon_{\Sigma, \text{SS}}$ the quantity

$$\mathbb{P}[\Sigma.\text{SS}(\mathbf{A}, \mathbf{E}) = 0].$$

where $\Sigma.\text{SS}(\mathbf{A}, \mathbf{E})$ is the experiment defining special soundness (cf. Fig. 1). Let $\mathbf{ASucc}$ be the event where the adversary succeeds in creating s transcripts having distinct challenges and correctly verifying, *i.e.*

$$\mathbf{ASucc} = \left\{ \begin{array}{l} ch_1 \neq ch_2 \\ \Sigma.\text{Verify}((\mathbf{X}, \mathbf{y}), \text{tr}_c) = 1 \ \forall c \in [2] \end{array} \middle| \begin{array}{l} ((\mathbf{X}, \mathbf{y}), \text{mtr}) \leftarrow^s \mathbf{A}(), \\ \mathbf{k} \leftarrow^s \mathbf{E}((\mathbf{X}, \mathbf{y}), \text{mtr}) \end{array} \right\},$$

where the abbreviation mtr denotes the s transcripts produced by $\mathbf{A}$, *i.e.*

$$\begin{aligned}
 \text{mtr} &= (\text{firstmes}, (ch_c, \text{resp}_c)_{c \in \{1,2\}}) \\
 &= ((\mathbf{y}_i^{(j)}, h_i^{(j)})_{i \in [d], j \in [t]}, ((\mathcal{I}_c^{(j)}), (\mathbf{k}_{i,c}^{(j)})_{i \in \mathcal{I}_c^{(j)}})_{j \in [t], c \in \{1,2\}},
 \end{aligned}$$

$\mathbf{E}((\mathbf{X}, \mathbf{y}), firstmes, (ch_c, resp_c)_{c \in [s]})$ <pre> $j^* \leftarrow 0$ for $j \in [t]$ for $c \in \{1, 2\}$ $(\mathcal{I}_c^{(j)}) \leftarrow ch_c$ $(\mathbf{k}_{i,c}^{(j)})_{i \in \mathcal{I}_c^{(j)}, j \in [t]} \leftarrow resp_c$ if $\mathcal{I}_1^{(j)} \neq \mathcal{I}_2^{(j)}$ $j^* = j$ if $j^* = 0$ return $\perp$ for $i \in [d]$ if $i \in \mathcal{I}_1^{(j^*)}$ $\mathbf{k}_i \leftarrow \mathbf{k}_{i,1}^{(j^*)}$ else $\mathbf{k}_i \leftarrow \mathbf{k}_{i,2}^{(j^*)}$ $\mathbf{k} \leftarrow \sum_{i \in [d-1]} G_{\mathcal{K}}(\mathbf{k}_i) + \mathbf{k}_d$ return $\mathbf{k}$ </pre>

Fig. 21. The extractor $\mathbf{E}$.

and tr_c denotes the c -th transcript $(\text{firstmes}, (ch_c, \text{resp}_c))$. Then,

$$\mathbb{P}[\Sigma.\text{SS}(\mathbf{A}, \mathbf{E}) = 0] = \mathbb{P}[\Sigma.\text{SS}(\mathbf{A}, \mathbf{E}) = 0, \text{ASucc}]. \quad (40)$$

If $\mathbb{P}[\text{ASucc}] = 0$, then the theorem follows, as [Eq. \(40\)](#) is always smaller than $\mathbb{P}[\text{ASucc}]$. Otherwise, we can use Bayes' Theorem and rewrite [Eq. \(40\)](#) as

$$\mathbb{P}[\Sigma.\text{SS}(\mathbf{A}, \mathbf{E}) = 0] = \mathbb{P}[\Sigma.\text{SS}(\mathbf{A}, \mathbf{E}) = 0 | \text{ASucc}] \cdot \mathbb{P}[\text{ASucc}]. \quad (41)$$

In the rest of this proof, we show that

$$\mathbb{P}[\Sigma.\text{SS}(\mathbf{A}, \mathbf{E}) = 0 | \text{ASucc}] = 0. \quad (42)$$

According to [Eq. \(41\)](#), this implies the theorem. Intuitively, [Eq. \(42\)](#) means that conditioned on the two challenges being distinct and the transcripts all accepting, then the extractor always outputs a valid witness. [Eq. \(42\)](#) comes from the following sequence of remarks.

1. Conditioning on ASucc guarantees that $ch_1 \neq ch_2$. Since each challenge ch_c consists of t subsets $(\mathcal{I}_c^{(j)})_{j \in [t]}$, then there must be at least one index j^* in which the subsets differ, *i.e.* $\mathcal{I}_1^{(j^*)} \neq \mathcal{I}_2^{(j^*)}$. Since those are two distinct subsets of $[d]$ with $d-1$ elements, then $\mathcal{I}_1^{(j^*)} \cup \mathcal{I}_2^{(j^*)} = [d]$. This implies that the extractor $\mathbf{E}$ is well defined and does not abort.
2. Using [Item 1](#), we know that $\mathbf{E}$ produces d values $(\mathbf{k}_1, \dots, \mathbf{k}_d)$ and computes $\mathbf{k} = \sum_{i \in [d-1]} G_{\mathcal{K}}(\mathbf{k}_i) + \mathbf{k}_d$. In other words, $(\mathbf{k}_1, \dots, \mathbf{k}_d)$ is a sharing of the extracted key $\mathbf{k}$.
3. By combining [Item 2](#), the almost key homomorphism of the rounding function ([Proposition 1](#), [Item 3](#)), and the fact that the two first messages are equal, we derive

$$\lfloor \mathbf{X}\mathbf{k} \rfloor_{2^{q-\alpha}} - \tilde{\mathbf{y}}_1^{(j^*)} \in \{0, 1\}, \quad (43)$$

where $\tilde{\mathbf{y}}_1^{(j^*)}$ denotes the value $\tilde{\mathbf{y}}^{(j^*)}$ reconstructed during the verification of tr_1 . The latter equation follows a reasoning similar to the one made for proving [Theorem 4](#), which we detail below. We define

$$x = \sum_{i \in \mathcal{I}^{(j^*)}} \text{Output}(\mathbf{X}, \mathbf{k}_{i,1}^{(j^*)}, i), \quad x' = \text{Output}(\mathbf{X}, \mathbf{k}_{i^*,2}^{(j^*)}, i^*).$$

Using [2](#), we can rewrite

$$\begin{aligned} \mathbf{X}\mathbf{k} &= \sum_{i \in [d-1]} \mathbf{X}G_{\mathcal{K}}(\mathbf{k}_i) + \mathbf{X}\mathbf{k}_d \\ &= \sum_{i \in [d]} \text{Output}(\mathbf{X}, \mathbf{k}_i, i) \\ &= \sum_{i \in \mathcal{I}^{(j^*)}} \text{Output}(\mathbf{X}, \mathbf{k}_{i,1}^{(j^*)}, i) + \text{Output}(\mathbf{X}, \mathbf{k}_{i^*,2}^{(j^*)}, i^*) \\ &= \mathbf{x} + \mathbf{x}', \end{aligned}$$

where $i^* = [d] \setminus \mathcal{I}^{(j^*)}$. Therefore, the almost key homomorphism of the rounding ([Proposition 1](#), [Item 3](#)) implies that

$$\lfloor \mathbf{X}\mathbf{k} \rfloor_{2^{q-\alpha}} - \lfloor \mathbf{x} \rfloor_{2^{q-\alpha}} - \lfloor \mathbf{x}' \rfloor_{2^{q-\alpha}} \in \{0, 1\}^m. \quad (44)$$

To conclude the proof of the remark, we use two main observations. First, we denote with $sum_1^{(j^*)}$ the partial sum computed in the verification of tr_1 , and rewrite

$$\lfloor \mathbf{x} \rfloor_{2^{q-\alpha}} = \left\lfloor \sum_{i \in \mathcal{I}^{(j^*)}} \text{Output}(\mathbf{X}, \mathbf{k}_{i,1}^{(j^*)}, i) \right\rfloor_{2^{q-\alpha}} = \lfloor sum_1^{(j^*)} \rfloor_{2^{q-\alpha}}. \quad (45)$$

Next, we use the fact that the first message is the same for both transcripts, and they are both accepting. This implies that

$$\lfloor \mathbf{x}' \rfloor_{2^{q-\alpha}} = \left\lfloor \text{Output}(\mathbf{X}, \mathbf{k}_{i^*,2}^{(j^*)}, i^*) \right\rfloor_{2^{q-\alpha}} = \mathbf{y}_{i^*} \quad (46)$$

where $\mathbf{y}_{i^*}$ is the i^* -th output share appearing in *firstmes*. [Eq. \(45\)](#) and [Eq. \(46\)](#) imply that

$$\lfloor \mathbf{x} \rfloor_{2^{q-\alpha}} + \lfloor \mathbf{x}' \rfloor_{2^{q-\alpha}} = \lfloor sum_1^{(j^*)} \rfloor_{2^{q-\alpha}} + \mathbf{y}_{i^*} = \tilde{\mathbf{y}}_1^{(j^*)}. \quad (47)$$

Combining [Eq. \(47\)](#) and [Eq. \(44\)](#), the remark follows.

4. Conditioning on ASucc guarantees that verification succeeds, and in particular that

$$\mathbf{y} - \tilde{\mathbf{y}}_1^{(j^*)} \in \{0, 1\}^m.$$

5. [Item 3](#) and [4](#) imply that $\mathbf{k}$ is a valid witness for $(\mathbf{X}, \mathbf{y})$, as

$$\mathbf{y} - \lfloor \mathbf{X}\mathbf{k} \rfloor_{2^{p-\alpha}} = (\mathbf{y} - \tilde{\mathbf{y}}_1^{(j^*)}) - (\lfloor \mathbf{X}\mathbf{k} \rfloor_{2^{p-\alpha}} - \tilde{\mathbf{y}}_1^{(j^*)}) \in \{-1, 0, +1\}^m.$$

This concludes the proof.

E.4 Quantum Computationally Unique Response ([Theorem 7](#))

This proof is very similar to the one provided for Picnic1 in Corollary 26 of [\[47\]](#). To prove quantum computationally unique responses, we must argue about the event where an adversary provides two different responses for the same first message and challenge, and about the detection of a measurement in the computational basis. Notably, the response of our sigma protocol only contains preimages of the hash function. Therefore, the QCUR property reduces to the collapsingness of H on each of the $(d-1)t$ opened values. Using the bound on the concurrent composition of collapsing functions (Lemma 4, [\[51\]](#)), the theorem follows.

E.5 Unpredictable Commitments (**Theorem 8**)

To prove commitment unpredictability, we must upper bound the following quantity

$$\mathbb{E} \left(\max_{firstmes^* \in \mathcal{N}_{com}} \Pr[firstmes = firstmes^*] \right),$$

where the average ranges over the choice of the statement and witness pair, and the probability inside is taken over $firstmes \leftarrow \Sigma.\text{Prove}_1(x, w)$. In what follows, we actually ignore the average and provide an upper bound for every statement and witness pair. Notably, for every statement $((\mathbf{X}, \mathbf{y}), \mathbf{k})$ and every first message $firstmes^*$, the probability that $\Sigma.\text{Prove}_1$ generates exactly $firstmes^*$ as a commitment is upper-bounded by the probability that all the corresponding hash values are the same. In particular, this holds true for the $t(d-1)$ values generated with the PRG. Formally,

$$\Pr[firstmes = firstmes^*] \leq \Pr[h_i^{(j)} = h_i^{*(j)} \quad \forall (i, j) \in [d-1] \times [t]].$$

We now want to leverage the inherent collision resistance of collapsing hash functions. For $N \in [(d-1)t]$ and for every subset of N indices $\mathcal{S} \subseteq [d-1] \times [t]$, consider the event where only the key shares in $\mathcal{S}$ are different, *i.e.*

$$\text{Event}_{\mathcal{S}} = \{\mathbf{k}_i^{(j)} \neq \mathbf{k}_i^{*(j)} \quad \forall (i, j) \in \mathcal{S} \text{ and } \mathbf{k}_i^{(j)} = \mathbf{k}_i^{*(j)} \text{ otherwise}\},$$

and let $\text{Event}_{i,j}$ be the event where the specific couple (i, j) yields distinct key shares, *i.e.*

$$\text{Event}_{i,j} = \{\mathbf{k}_i^{(j)} \neq \mathbf{k}_i^{*(j)}\}.$$

Using the law of total probabilities and the definition of conditional probability, we get

$$\begin{aligned} \Pr[firstmes = firstmes^*] &\leq \Pr[h_i^{(j)} = h_i^{*(j)} \quad \forall (i, j) \in [d-1] \times [t]] \\ &= \sum_{N \in [(d-1)t]} \sum_{\substack{\mathcal{S} \subseteq [(d-1)] \times [t] \\ |\mathcal{S}|=N}} \Pr[h_i^{(j)} = h_i^{*(j)} \quad \forall (i, j) \in [d-1] \times [t], \text{Event}_{\mathcal{S}}] \\ &= \sum_{N \in [(d-1)t]} \sum_{\substack{\mathcal{S} \subseteq [(d-1)] \times [t] \\ |\mathcal{S}|=N}} \Pr[h_i^{(j)} = h_i^{*(j)} \quad \forall (i, j) \in [d-1] \times [t] \mid \text{Event}_{\mathcal{S}}] \\ &\quad \cdot \Pr[\text{Event}_{\mathcal{S}}]. \end{aligned} \tag{48}$$

In what follows, we analyze the terms of [Eq. \(48\)](#) separately. Starting with $\Pr[\text{Event}_{\mathcal{S}}]$, we remark that $d-1$ shares per iteration correspond to $(d-1)t$ independently random sampled seeds for the PRG. Therefore, $\Pr[\text{Event}_{\mathcal{S}}]$ satisfies

$$\Pr[\text{Event}_{\mathcal{S}}] \leq \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^N \cdot \left(\frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^{(d-1)t-N}.$$

We now study the term $\Pr[h_i^{(j)} = h_i^{*(j)} \mid \forall (i, j) \in [d-1] \times [t] \mid \mathbf{Event}_S]$. Using the independence of the key shares and the fact that equal key shares correspond to equal hash, we can rewrite it as

$$\begin{aligned} & \Pr[h_i^{(j)} = h_i^{*(j)} \mid \forall (i, j) \in [d-1] \times [t] \mid \mathbf{Event}_S] \\ &= \prod_{(i,j) \in \mathcal{S}} \Pr[h_i^{(j)} = h_i^{*(j)} \mid \mathbf{Event}_{i,j}] \end{aligned} \quad (49)$$

$$\leq (2\epsilon_{\text{Coll}})^N, \quad (50)$$

where the last inequality holds because collapsing hash functions are inherently collision-resistant (*cf.* Lemma 24 in [82]). Notably, the terms in Eq. (49) correspond to finding a collision for the hash function. According to Lemma 24 of [82], each of them can be upper bounded by $2\epsilon_{\text{Coll}}$, yielding Eq. (50). Going back to Eq. (48), this yields

$$\begin{aligned} & \Pr[\text{firstmes} = \text{firstmes}^*] \leq \Pr[h_i^{(j)} = h_i^{*(j)} \mid \forall (i, j) \in [d-1] \times [t]] \\ &= \sum_{N \in [(d-1)t]} \sum_{\substack{S \subseteq [(d-1)] \times [t] \\ |S|=N}} \Pr[h_i^{(j)} = h_i^{*(j)} \mid \forall (i, j) \in [d-1] \times [t] \mid \mathbf{Event}_S] \\ & \quad \cdot \Pr[\mathbf{Event}_S] \\ &\leq \sum_{N \in [(d-1)t]} \sum_{\substack{S \subseteq [(d-1)] \times [t] \\ |S|=N}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^N \cdot \left(\frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^{(d-1)t-N} (2\epsilon_{\text{Coll}})^N \\ &= \sum_{N \in [(d-1)t]} \binom{(d-1)t}{N} \cdot \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^N \cdot \left(\frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^{(d-1)t-N} (2\epsilon_{\text{Coll}})^N \\ &\leq \left(2\epsilon_{\text{Coll}} \left(1 - \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right) + \frac{1}{|\mathcal{P}_{\mathcal{K}}|}\right)^{(d-1)t}, \end{aligned}$$

where the last inequality is true by the binomial theorem. The theorem follows.

F Albrecht et al.'s estimator security estimates

Table 9. Security estimates (in bits) against lattice attacks with $m = \infty$ using [5] (Commit 2bfb768). Entries marked as ∞ indicate a security greater than 1000 bits.

NIST Security	q	(p', n)	coded BKW	uSVP	BDD	BDD hybrid	BDD mitm hybrid	dual	Size (Bytes) $d = 256$
L1	128	(2, 6)	392.60	144.01	144.88	144.88	299.20	159.01	7 817
		(3, 9)	521.01	138.45	138.45	138.45	292.76	154.22	6 510
		(4, 12)	649.33	135.67	135.67	135.67	265.81	145.74	6 919
	112	(2, 6)	344.41	128.73	129.60	129.60	258.52	142.62	7 164
		(3, 10)	456.82	135.12	135.12	135.12	283.64	150.84	6 401
		(4, 13)	569.14	129.62	129.84	129.84	254.05	139.00	6 692
	96	(2, 7)	296.19	128.73	129.60	129.60	258.52	142.62	7 164
		(3, 11)	392.60	128.73	128.73	128.73	265.55	143.65	6 183
		(4, 16)	585.19	135.67	135.67	135.67	265.81	145.74	6 919
	80	(2, 10)	328.34	149.01	149.88	149.88	312.95	164.02	8 035
		(3, 14)	408.67	135.12	135.12	135.12	283.64	150.84	6 401
		(4, 19)	569.15	134.56	134.56	134.56	262.48	144.30	6 874
L3	128	(2, 9)	521.01	205.31	206.18	206.18	469.51	222.33	17 563
		(3, 14)	649.34	203.36	203.39	203.39	483.65	224.59	14 949
		(4, 19)	905.82	202.52	201.12	201.12	461.21	219.93	16 084
	112	(2, 10)	456.82	200.29	200.87	200.87	454.93	217.02	17 237
		(3, 16)	681.41	203.36	203.39	203.39	483.65	224.59	14 949
		(4, 21)	∞	196.38	194.98	194.98	443.12	213.47	15 743
	96	(2, 12)	488.92	205.31	206.18	206.18	469.51	222.33	17 563
		(3, 18)	681.41	196.94	196.95	196.95	463.85	217.30	14 622
		(4, 25)	873.77	201.12	199.17	199.17	459.47	217.20	15 947
	80	(2, 14)	408.67	200.29	200.87	200.87	454.93	217.02	17 237
		(3, 22)	649.34	200.01	200.02	200.02	473.98	221.20	14 782
		(4, 29)	∞	194.15	192.75	192.75	435.72	210.74	15 609
L5	128	(2, 12)	649.33	266.83	267.42	267.42	648.64	284.75	31 200
		(3, 19)	905.82	268.51	269.38	269.38	684.09	296.82	26 842
		(4, 25)	∞	261.83	259.55	259.55	647.39	283.87	28 330
	112	(2, 14)	569.14	271.87	272.74	272.74	663.87	290.08	31 636
		(3, 21)	∞	260.39	261.26	261.26	658.08	288.10	26 300
		(4, 29)	∞	263.19	262.91	262.91	654.53	287.90	28 598
	96	(2, 16)	585.19	266.83	267.42	267.42	648.64	284.75	31 200
		(3, 25)	873.77	265.15	266.02	266.02	673.66	293.42	26 627
		(4, 33)	∞	259.39	257.03	257.03	640.07	281.14	28 147
	80	(2, 19)	569.15	264.31	264.90	264.90	641.03	282.49	30 985
		(3, 29)	∞	257.03	257.90	257.90	647.73	284.47	26 080
		(4, 40)	∞	261.83	259.55	259.55	647.39	283.87	28 330